Traveler - Web Application

It's Possible Tech

2024/2025

1211503 - João Henrique da Costa Pereira

Traveler - Web Application

It's Possible Tech

2024/2025

1211503 - João Henrique da Costa Pereira

Licenciatura em Engenharia Informática

Junho de 2025

Orientador ISEP: Patricia Alves

Supervisor Externo: José Silva

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, 2024/2025

Resumo

A forma como as agências de viagem apresentam as propostas e os orçamentos aos seus clientes tem, atualmente, um impacto cada vez mais significativo nas decisões dos consumidores finais. Nos dias de hoje, é bastante comum encontrar agências que recorrem a documentos PDF ou até mesmo a impressões em papel para apresentar as propostas ao cliente, o que acaba por dificultar a comunicação e tornar a experiência menos envolvente, levando a taxas de conversão muito baixas. Neste contexto, foi desenvolvido o *Traveler*, uma aplicação web que procura modernizar o processo de orçamentação e apresentação das propostas, transformando-o numa experiência digital personalizada e ajustada ao perfil de cada cliente.

Este relatório documenta a abordagem seguida ao longo do estágio curricular, realizado na empresa It's Possible Tech. A solução desenvolvida inclui um Backoffice para a gestão de propostas por parte dos agentes de viagem e um Frontoffice destinado à visualização das propostas pelos clientes (não documentado neste relatório). O desenvolvimento da aplicação assentou numa arquitetura modular baseada em boas práticas de engenharia de software. Entre as principais funcionalidades implementadas destacam-se a autenticação com Google, envio de propostas por email através de SMTP, duplicação de propostas, e a criação automatizada de itinerários com suporte de inteligência artificial. O projeto foi conduzido com recurso à metodologia Gitflow e envolveu várias fases: levantamento de requisitos, desenho arquitetónico (C4 e 4+1), implementação de funcionalidades organizadas por user stories, testes funcionais, validação iterativa com o supervisor e deploy para um ambiente de produção.

Os objetivos propostos foram plenamente atingidos, resultando numa aplicação funcional, escalável e adaptada às necessidades operacionais da empresa. A solução demonstra-se vantajosa na redução de tarefas repetitivas, no aumento da eficiência interna e na melhoria da experiência do cliente. Foram ainda identificadas oportunidades de melhoria, como a integração com sistemas de reservas em tempo real (GDS), suporte multilingue e melhorias no módulo de apoio ao cliente.

Palavras-chave: Turismo, Orçamentação Digital, Experiência do Cliente, Propostas Interativas, Eficiência Operacional

Abstract

The way travel agencies present travel proposals and budgets to their clients currently has an increasingly significant impact on the decisions of end consumers. Nowadays, it is still quite common to find agencies that rely on PDF documents or even printed versions to present proposals to clients, which often hinders communication and makes the experience less engaging, leading to very low conversion rates. In this context, *Traveler* was developed, a web application that aims to modernize the budgeting and proposal presentation process, transforming it into a digital experience that is personalized and tailored to each client's profile.

This report documents the approach followed throughout the curricular internship carried out at the company It's Possible Tech. The developed solution includes a Backoffice for travel agents to manage proposals, and a Frontoffice for clients to view their proposals (not documented in this report). The application was built on a modular architecture based on software engineering best practices. Among the main features implemented are Google authentication, proposal delivery via SMTP email, proposal duplication, and the automated generation of itineraries supported by artificial intelligence. The project was conducted using the Gitflow methodology and included several phases: requirements gathering, architectural design (C4 and 4+1), implementation of features organized by user stories, functional testing, iterative validation with the supervisor, and deployment to a production environment.

The defined objectives were fully achieved, resulting in a functional, scalable application adapted to the company's operational needs. The solution proves advantageous in reducing repetitive tasks, increasing internal efficiency, and enhancing the client experience. Improvement opportunities were also identified, such as integration with real-time booking systems (GDS), multilingual support, and enhancements to the customer support module.

Agradecimentos

Quero expressar o meu sincero agradecimento à Professora Patrícia Alves, pela orientação e pelos conselhos valiosos ao longo deste projeto.

Quero expressar o meu agradecimento ao meu orientador de estágio, José Silva, pela confiança e pela forma como me guiou ao longo deste percurso. A sua disponibilidade e partilha de conhecimento foram fundamentais para o meu crescimento pessoal e profissional.

Aos meus pais, agradeço por tudo o que fizeram e continuam a fazer por mim. O vosso apoio, mesmo nos momentos menos fáceis, foi o que me deu força para chegar até aqui.

Um agradecimento muito especial à Inês pela presença constante, pelas palavras certas nos momentos difíceis e por me lembrar, sempre, que desistir não é opção.

Por fim, aos meus amigos, que me acompanharam ao longo deste percurso, o meu obrigado. Tornaram este caminho mais leve, mais divertido, e cheio de boas memórias.

Conteúdo

Agradecimentos **vii**

Lista de Figuras **xiii**

Lista de Tabelas **xv**

Lista de Algorítmos **xv**

Lista de Código **xv**

Lista de Acrónimos **xvii**

1 Introdução **1**
1.1 Enquadramento/Contexto 1
1.2 Descrição do Problema 1
1.2.1 Objetivos 1
1.2.2 Abordagem 2
1.2.3 Contributos 2
1.2.4 Planeamento do Trabalho - Backoffice 2
1.3 Estrutura do Relatório 3

2 Conceitos e Trabalhos Relacionados **5**
2.1 Trabalhos Relacionados 5
2.1.1 Ferramentas Especializadas em Criação de Propostas de Viagem Personalizadas 5
2.1.2 Plataformas All-in-One para Agências 6
2.1.3 Análise Comparativa 7
2.1.4 Síntese e Enquadramento com o Projeto Desenvolvido 7
2.2 Tecnologias utilizadas 8
2.3 Tecnologias Existentes vs Tecnologias Utilizadas 8
2.3.1 Base de dados relacional e não relacional 9
2.3.2 Node.js vs .NET 9
2.3.3 Angular VS React VS NextJS 10

3 Análise do Problema **13**
3.1 Domínio do Problema 13
3.1.1 Glossário e Modelo Domínio - Backoffice 14
3.2 Requisitos funcionais e não funcionais 15
3.2.1 Gestão de Utilizadores e Propostas de Viagem - Backoffice 15
3.3 Requisitos Não Funcionais 17

4 Desenho da Solução **19**
4.1 Arquitetura da Aplicação 19
4.1.1 Nível 1 - Vista Lógica 19
4.1.2 Nível 2 - Vista Lógica 20
4.1.3 Nível 2 - Vista Física 21
4.1.4 Nível 3 - Vista Implementação 22
Vista Implementação - Backend 22
Vista Implementação - Frontend 23

5 Implementação da Solução **25**
5.1 Descrição da implementação 25
5.1.1 US01 - Como utilizador da aplicação quero fazer login no sistema utilizando as minhas credenciais 27
5.1.2 US02 - Como utilizador da aplicação quero aceder apenas à minha dashboard 30
5.1.3 US05 - Como Administrador/Travel Designer quero poder criar packs para associar às propostas de viagem 32
5.1.4 US06 - Como Administrador/Travel Designer quero criar uma nova proposta de viagem 35
5.1.5 US07 – Enviar proposta por email com recurso a SMTP 48
5.2 Deploy da Aplicação 52
5.2.1 Links da Aplicação 52
5.3 Testes 52
5.4 Avaliação da Solução 53

6 Conclusões **55**
6.1 Objetivos Concretizados 55
6.2 Limitações e trabalho futuro 56
6.3 Apreciação Final 56

A Anexos **59**
A.1 Diagrama de Sequência & Interface – Viajantes 59
A.2 Diagrama de Sequência & Interface – Destinos 61
A.3 Diagrama de Sequência & Interface – Elementos Extra 63
A.4 Diagrama de Sequência & Interface – Hotéis 64
A.5 Diagrama de Sequência & Interface – Voos 68
A.6 Diagrama de Sequência & Interface – Transferes 71
A.7 Diagrama de Sequência & Interface – Atividades 73
A.8 Diagrama de Sequência & Interface – Seguros 76
A.9 Diagrama de Sequência & Interface – Celebrações 79
A.10 Diagrama de Sequência & Interface – Itinerários 82

Lista de Figuras

1.1 Diagrama de Gantt - Planeamento do Trabalho 3

3.1 Diagrama de classes do backoffice da aplicação 15
3.2 Diagrama de Casos de Uso - Backoffice 17

4.1 Backoffice: Vista Lógica - Nível 1 19
4.2 Backoffice: Vista Lógica - Nível 2 20
4.3 Backoffice: Vista Fisica 21
4.4 TravelerBE: Vista Implementação 22
4.5 TravelerFE: Vista Implementação 23

5.1 Arquitetura do Backend - MVC Adaptado 25
5.2 Arquitetura do Frontend - Arquitetura modular e baseado em componentes reutilizáveis 26
5.3 US01: Diagrama de Sequência do Frontend 28
5.4 US01: Diagrama de Sequência do Backend 29
5.5 Interface de Login 30
5.6 Dashboard do Administrador 31
5.7 Dashboard do Travel Designer 31
5.8 Dashboard da Equipa de Suporte 32
5.9 Página de gestão de packs 32
5.10 US05: Diagrama de Sequência do Frontend 33
5.11 US05: Diagrama de Sequência do Backend 34
5.12 Botão que dá início ao processo de criação de uma proposta 35
5.13 Diagrama de estados das propostas de viagem 36
5.14 US7 - Diagrama de sequência (Frontend) 50
5.15 US7 - Diagrama de sequência (Backend) 51

A.1 Diagrama de sequência (frontend) – Gestão de viajantes 59
A.2 Diagrama de sequência (backend) – Gestão de viajantes 60
A.3 Interface da página de viajantes 60
A.4 Diagrama de sequência (frontend) – Gestão de destinos 61
A.5 Diagrama de sequência (backend) – Gestão de destinos 62
A.6 Interface da página de destinos 62
A.7 Diagrama de sequência (frontend) – Gestão de elementos extra 63
A.8 Diagrama de sequência (backend) – Gestão de elementos extra 64
A.9 Interface da página de elementos extra 64
A.10 US06.4 - Diagrama de sequência (frontend) 65
A.11 US06.4 - Diagrama de sequência (backend) 66
A.12 Interface da página de Hotéis: Seleção de um pack 67
A.13 Formulário para adicionar hotel 67
A.14 Interface da página de Hotéis: Seleção de um pack 68

A.15 Formulário para adicionar hotel 68
A.16 US06.5 - Diagrama de sequência (frontend) 69
A.17 US06.5 - Diagrama de sequência (backend) 70
A.18 Interface da página de Transferes: Seleção de um pack 71
A.19 Formulário para adicionar transfere 71
A.20 US06.6 - Diagrama de sequência (frontend) 72
A.21 US06.6 - Diagrama de sequência (backend) 73
A.22 Interface da página de Atividades: Seleção de um pack 73
A.23 Lista das Atividades 74
A.24 Formulário de Atividade 74
A.25 US06.7 - Diagrama de sequência (frontend) 75
A.26 US06.7 - Diagrama de sequência (backend) 76
A.27 Interface da página de Seguros: Seleção de um pack 76
A.28 Seleção de um Seguro 77
A.29 US06.8 - Diagrama de sequência (frontend) 78
A.30 US06.8 - Diagrama de sequência (backend) 79
A.31 Interface da página de Celebrações: Seleção de um pack 79
A.32 Listagem de Celebrações 80
A.33 Formulário para adicionar/editar celebração 80
A.34 US06.9 - Diagrama de sequência (frontend) 81
A.35 US06.9 - Diagrama de sequência (backend) 82
A.36 Interface da página de Itinerários: Seleção de um pack 82
A.37 Botão para criar itinerário com IA 83
A.38 Exemplo Itinerário criado com IA 83
A.39 US06.11 - Diagrama de sequência (frontend) 84
A.40 US06.11 - Diagrama de sequência (backend) 85

Lista de Tabelas

2.1 Comparação entre Plataformas All-in-One e Aplicação Desenvolvida 7
2.2 Comparação entre bases de dados relacionais e não relacionais 9
2.3 Comparação entre Node.js e .NET 10

3.1 Glossário Backoffice 14
3.2 User Stories da componente de Backoffice 15

6.1 Concretização dos objetivos do projeto 55

Lista de Acrónimos

ACID	(Atomicity, Consistency, Isolation, and Durability).
AI	Artificial Intelligence.
API	Application Programming Interface.
API REST	API Representational State Transfer.
AWS	Amazon Web Services.
CLI	Command Line Interface.
CPU	Central Processing Unit.
CRM	Gestão de Relacionamento com o Cliente.
CSS	Cascading Style Sheets.
DNS	Domain Name System.
ERP	Enterprise Resource Planning.
FURPS+	Functionality, Usability, Reliability, Performance, and Supportability.
GDS	Global Distribution System.
HTML	HyperText Markup Language.
HTTP	HyperText Transfer Protocol.
IA	Inteligência Artificial.
IATA	International Air Transport Association.
JSON	JavaScript Object Notation.
JWT	JSON Web Token.
LLM	Large Language Model.
MVP	Minimum Viable Product.
PWA	Progressive Web Apps.
SMTP	Simple Mail Transfer Protocol.
SPA	Singe Page Application.
SQL	Structured Query Language.
SSL	Secure Sockets Layer.

UX	User Experience.

Capítulo 1

Introdução

1.1 Enquadramento/Contexto

No contexto do turismo, a forma como um orçamento de viagem é apresentado pode influenciar a decisão do cliente. Nos dias de hoje, é bastante comum encontrar agências que recorrem a documentos PDF ou até mesmo a impressões em papel para apresentar as propostas ao cliente, o que acaba por dificultar a comunicação e tornar a experiência menos envolvente. Este tipo de abordagem pode afastar o cliente e reduzir a eficácia da proposta, já que, atualmente, com tanta oferta, é difícil destacar um negócio que se limite a prestar o serviço ao cliente, sem tornar apelativo e personalizado o próprio processo da prestação. As abordagens tradicionais comprometem a experiência de quem recebe a proposta. Como resultado, é comum observar taxas de conversão mais baixas e atrasos nos pagamentos, fatores que, em conjunto, impactam negativamente a rentabilidade e a sustentabilidade do negócio.

1.2 Descrição do Problema

Atualmente, muitas agências de viagens ainda recorrem a documentos em PDF ou mesmo a folhas impressas para apresentar propostas de viagem aos seus clientes. Estes métodos prejudicam a interatividade e o relacionamento entre a agência e o cliente. A equipa da It's Possible Tech identificou a necessidade de encontrar uma solução que permitisse aumentar as taxas de conversão e reduzir os atrasos nos pagamentos.

Neste contexto, surge o Traveler, uma aplicação que promete, em primeiro lugar, simplificar o trabalho dos gestores de viagens, permitindo-lhes criar novas propostas com apenas alguns cliques. Em segundo lugar, pretende-se apresentar essas propostas de forma totalmente interativa, através de um website personalizado que proporciona uma experiência de maior proximidade com o utilizador. Esta solução, especialmente num mundo cada vez mais digital, revela-se extremamente vantajosa, pois faz com que os clientes se sintam valorizados ao receberem uma proposta num formato adaptado às suas expectativas.

1.2.1 Objetivos

O principal objetivo deste projeto consiste em aumentar não só a produtividade das agências de viagens, como também a sua receita. Para alcançar este propósito, foram definidos os seguintes objetivos específicos:

- Desenvolver um backoffice que permita aos agentes de viagens criar propostas de forma rápida e eficiente, e que possibilite aos administradores das agências acompanhar a produtividade dos seus colaboradores;
- Construir um frontoffice intuitivo, onde os clientes possam aceder à proposta de viagem de forma interativa e personalizada;
- Garantir a segurança da aplicação, tendo em conta que as agências lidam com dados sensíveis dos clientes (como o número do cartão de cidadão e contactos telefónicos), prevenindo assim qualquer tipo de ataque informático;
- Documentar detalhadamente todo o processo de desenvolvimento, incluindo os testes realizados e os resultados obtidos.

1.2.2 Abordagem

A proposta preconizada consistirá em desenvolver uma aplicação web que resolverá as limitações dos métodos tradicionais de apresentação de orçamentos. Para isso, a solução dividir-se-á em dois ambientes principais:

Frontoffice: Voltado para os clientes, este ambiente permitirá que visualizem os orçamentos de viagens de forma dinâmica e envolvente. A implementação do frontoffice ficará a cargo de outro estagiário, sendo desenvolvida em colaboração em equipa para garantir a integração com o restante do sistema.

Backoffice: Destinado às empresas, este ambiente possibilitará a gestão completa dos orçamentos enviados. O backoffice otimizará o fluxo de trabalho e contribuirá para a melhoria na eficiência corporativa.

Do ponto de vista tecnológico, a implementação do backoffice contará com um backend criado em `Node.js` e `Express`, e uma base de dados em `MongoDB`. O frontend será desenvolvido utilizando `React`, `Next.js` e `Tailwind CSS`.

1.2.3 Contributos

A plataforma desenvolvida tem como principal contributo a substituição dos métodos tradicionais utilizados na elaboração e partilha de orçamentos de viagem, eliminando a dependência de ficheiros em formato PDF e de documentos estáticos. Esta modernização promove uma maior sustentabilidade ambiental, ao reduzir significativamente o consumo de papel.

Outro ponto bastante positivo, é a possibilidade de duplicar propostas o que torna o processo de criação de uma nova proposta ainda mais eficiente. Para consultores de viagens que trabalham com comissões, qualquer melhoria que torne o processo de realizar uma nova proposta e enviar para o cliente, traduz-se numa oportunidade de aumentar os seus rendimentos.

Por outro lado, a automação dos processos internos contribui para minimizar erros associados a procedimentos manuais, o que promove uma gestão mais fiável e eficiente dentro das agências de viagens.

1.2.4 Planeamento do Trabalho - Backoffice

- **Início do estágio:** 03/03/2025

- **Levantamento de Requisitos e Planeamento:** 03/03/2025 - 14/03/2025
- **Design, Arquitetura e Documentação:** 05/03/2025 - 21/03/2025
- **Desenvolvimento do Backend:** 21/03/2025 - 06/06/2025
- **Desenvolvimento do Frontend:** 28/03/2025 - 06/06/2025
- **Integração e Testes Finais:** 07/06/2025 - 14/06/2025
- **Validação e Lançamento do Minimum Viable Product (MVP):** 15/06/2025 - 30/06/2025
- **Desenvolvimento do Relatório de Estágio:** 03/03/2025 - 06/07/2025
- **Fim do estágio:** 30/06/2025

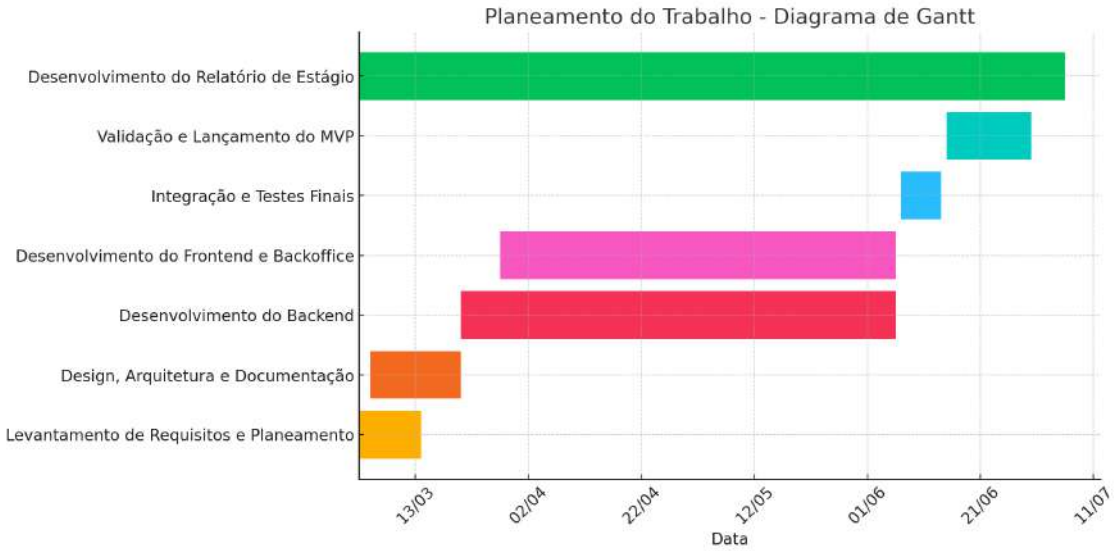

Figura 1.1: Diagrama de Gantt - Planeamento do Trabalho

1.3 Estrutura do Relatório

O presente relatório encontra-se organizado em seis capítulos, estruturados da seguinte forma:

No Capítulo 1, é apresentada a introdução do projeto, fornecendo uma visão geral sobre o mesmo. São abordados o contexto, os objetivos gerais e a motivação subjacente à proposta desenvolvida.

O Capítulo 2 incide sobre o estado da arte, incluindo uma breve pesquisa das tecnologias adotadas, trabalhos relacionados e a justificação das decisões tecnológicas tomadas ao longo do projeto.

No Capítulo 3, é realizada a análise do problema, recorrendo a artefactos como o glossário e o modelo de domínio. São igualmente definidos os requisitos funcionais e não funcionais da aplicação.

O Capítulo 4 é dedicado ao desenho da solução, onde se apresenta a arquitetura proposta para dar resposta ao problema identificado no capítulo anterior.

No Capítulo 5, é descrita a implementação da aplicação, com o detalhe das principais *user stories*, dos fluxos principais da solução e dos processos de avaliação realizados.

Por fim, o Capítulo 6 apresenta as conclusões do trabalho. São discutidos os objetivos alcançados, as principais limitações encontradas e uma reflexão final sobre o projeto desenvolvido.

Capítulo 2

Conceitos e Trabalhos Relacionados

Neste capítulo procede-se a uma análise das tecnologias já existentes, com foco em duas vertentes fundamentais: a gestão de propostas de viagem e a respetiva apresentação ao cliente. São exploradas as abordagens adotadas por diferentes plataformas, discutindo-se a visão subjacente a cada uma delas e avaliando em que medida os seus objetivos e funcionalidades se alinham com o contexto e as necessidades do projeto em desenvolvimento.

Posteriormente, é apresentada uma tabela comparativa que sintetiza as principais diferenças e semelhanças entre essas soluções existentes e a aplicação desenvolvida no âmbito deste trabalho.

Seguidamente, procede-se à análise das tecnologias e ferramentas utilizadas no desenvolvimento da aplicação. Importa referir que estas tecnologias foram previamente definidas no âmbito do projeto, não tendo sido objeto de escolha por parte do autor, o que condicionou algumas das decisões técnicas ao longo da implementação.

Por fim, realiza-se uma comparação final entre o ecossistema tecnológico utilizado com outras tecnologias também presentes no setor da informática.

2.1 Trabalhos Relacionados

A análise de ferramentas existentes no setor das agências de viagens é essencial para compreender o posicionamento do trabalho desenvolvido durante o estágio face às soluções atuais do mercado. Nos últimos anos, surgiram várias plataformas digitais que procuram apoiar os agentes de viagem na criação de propostas personalizadas, gestão de itinerários e automatização de processos administrativos e comerciais. Estas soluções podem ser agrupadas em duas categorias principais: ferramentas especializadas na criação de propostas e plataformas all-in-one para gestão global da agência.

2.1.1 Ferramentas Especializadas em Criação de Propostas de Viagem Personalizadas

Travefy[1] é uma plataforma especializada na criação de itinerários e propostas de viagem altamente visuais e personalizáveis. Direcionada a agentes de viagem independentes, pequenas agências e operadores turísticos que valorizam a estética e a experiência do cliente durante a fase de pré-venda, a aplicação permite a construção de itinerários interativos que integram fotografias, mapas, cronogramas detalhados e notas personalizadas, acessíveis tanto em formato web como PDF.[1]

[1] `https://travefy.com/`

Entre as suas funcionalidades mais relevantes destacam-se:

- Uma interface intuitiva com suporte drag-and-drop para montagem rápida de itinerários;
- Integração com bases de dados de destinos e fornecedores;
- Possibilidade de colaboração direta com os clientes através da partilha de links editáveis.

O principal trunfo do Travefy, à semelhança da aplicação analisada neste relatório, reside na sua capacidade de converter propostas de viagem em experiências visuais envolventes e apelativas. Todavia, a ausência de funcionalidades de reserva direta e de integração com sistemas de gestão mais robustos limita o seu potencial em contextos empresariais de maior escala ou com exigências operacionais mais complexas.[1]

2.1.2 Plataformas All-in-One para Agências

O **Lemax**[2] é uma plataforma completa de gestão para operadores turísticos e agências de viagens. É concebida para suportar todo o ciclo operacional da agência, incluindo:

- Sistema de reservas online com integração Global Distribution System (GDS);
- Gestão de inventário e contratos com fornecedores;
- Gestão de Relacionamento com o Cliente (CRM) incorporado e automatização de propostas;
- Funcionalidades financeiras e contabilidade;
- Dashboards analíticos para tomada de decisão.

[2]

O público-alvo do Lemax são operadores turísticos de média a grande dimensão.[2] A plataforma diferencia-se pela profundidade funcional e integração com sistemas de reservas em tempo real.[2] Como limitação, destaca-se o custo de implementação e a complexidade técnica associada.

O **TravelWorks**[3] desenvolvida pela **Pc Voyages**[4] é outra solução all-in-one, voltada para a gestão global de agências de viagens. Tal como o Lemax, oferece módulos de propostas, reservas, faturação, relatórios e CRM, mas destaca-se pela sua flexibilidade modular, permitindo que a agência escolha apenas os componentes necessários à sua operação.[3]

Entre os seus pontos fortes estão:

- Integração com múltiplos fornecedores e GDS;
- Gestão de contas, orçamentos e receitas centralizada;
- Plataforma multilingue e multi-moeda.

[3]

[2] `https://lemax.net/`
[3] `https://travelworkssolution.com/`
[4] `https://pcvoyages.com/`

Apesar da sua abrangência, a TravelWorks pode ser menos indicada para pequenas agências com recursos técnicos limitados, dadas as exigências de formação e adaptação do sistema.[3]

2.1.3 Análise Comparativa

Ao comparar ferramentas especializadas como o Travefy com plataformas completas como Lemax e TravelWorks, observa-se um claro *trade-off* entre flexibilidade e abrangência.

A Tabela 2.1 abaixo apresenta as principais diferenças entre estas abordagens, considerando critérios como custo, integração de dados em tempo real, complexidade técnica e objetivo funcional.

Tabela 2.1: Comparação entre Plataformas All-in-One e Aplicação Desenvolvida

Critério	**Plataformas All-in-One (Juniper, Lemax, TravelWorks)**	**Ferramentas Especializadas em Criação de Propostas de Viagem Personalizadas (Aplicação desenvolvida, Travefy**
Acesso a dados em tempo real	Sim – através de integração com GDS e Application Programming Interface (API)s	Não – dados inseridos manualmente
Criação de propostas interativas	Sim, com base em dados atualizados	Sim, com foco em storytelling e personalização
Automatização de reservas	Sim – permite reservas diretas	Não – proposta conceptual, sem sistema de reservas
Integração com fornecedores	Elevada (GDS, APIs de hotéis, voos, etc.)	Não existe integração externa
Personalização visual e editorial	Limitada pela estrutura dos dados automáticos	Elevada – controle total sobre layout e conteúdo
Custo de implementação	Elevado – licenciamento, formação, manutenção	Reduzido – desenvolvimento interno ou modular
Complexidade técnica	Alta – requer infraestrutura e conhecimento GDS	Média/baixa – foco na simplicidade e User Experience (UX)
Tempo de resposta ao cliente	Rápido – dados prontos e preços confirmáveis	Médio – depende da preparação manual da proposta
Público-alvo ideal	Grandes agências e operadores com grande volume	Pequenas/médias agências, agentes freelancers
Objetivo principal	Eficiência e escalabilidade	Inspiração e personalização

2.1.4 Síntese e Enquadramento com o Projeto Desenvolvido

A aplicação desenvolvida no âmbito deste estágio posiciona-se como uma solução leve e personalizável para a criação de propostas de viagem, sem dependência de integração com sistemas externos. Esta abordagem procura simplificar o processo de proposta e facilitar a adaptação a diferentes tipos de pacotes ou destinos, oferecendo maior controlo ao utilizador. Embora não ofereça funcionalidades como acesso em tempo real a preços ou reservas, permite uma apresentação profissional e personalizável que se alinha bem com o perfil de agências de pequena e média dimensão ou agentes freelancers.

Deste modo, o projeto propõe-se a preencher uma lacuna existente entre ferramentas especializadas e plataformas all-in-one, ao oferecer um meio termo entre estas ferramentas.

2.2 Tecnologias utilizadas

Desenvolvimento fullstack é o processo de desenvolvimento do frontend e do backend das aplicações. Qualquer aplicação tem uma componente de frontend (voltado para o utilizador) e um componente de backend (base de dados e lógica). O frontend contém a interface do utilizador e o código relacionado às interações dos utilizadores com a aplicação. O backend contém todo o código necessário para a execução da aplicação, incluindo integrações com bases de dados, comunicação com outras aplicações e processamento de dados.

No âmbito do desenvolvimento desta aplicação fullstack, foi-me imposto o uso de um conjunto específico de tecnologias que cobrem tanto o lado do cliente (frontend) como o lado do servidor (backend).

No backend, a aplicação foi desenvolvida com recurso ao Node.js, um ambiente de execução de JavaScript no lado do servidor. O Node.js permite a construção de aplicações com elevada capacidade de resposta, recorrendo a uma arquitetura assíncrona e orientada a eventos.

Sobre esta base, foi utilizado o Express.js, uma framework para o Node.js. O Express simplifica a criação de servidores web e APIs RESTful.

Como base de dados foi utilizado o MongoDB, uma base de dados NoSQL orientada a documentos. Ao contrário das bases de dados relacionais, o MongoDB não requer um esquema rígido, o que permite uma maior flexibilidade na estruturação dos dados ao longo do tempo.

No que diz respeito ao frontend, recorreu-se ao React, este é baseado no conceito de componentes reutilizáveis e permite a construção de interfaces de utilizador, utilizando um DOM virtual para otimizar a renderização (React, 2024).

Para complementar esta tecnologia, foi utilizado o Next.js, uma framework que expande as capacidades do React, permitindo funcionalidades avançadas como a renderização do lado do servidor (SSR) e a geração estática de páginas (SSG). O Next.js facilita também a gestão de rotas e a integração com APIs tornando-se uma excelente escolha para aplicações modernas (Next.js, 2024).

Em termos de estilo e design visual, a aplicação foi construída com Tailwind CSS, uma framework utilitária de Cascading Style Sheets (CSS) que permite aplicar estilos diretamente nas classes HyperText Markup Language (HTML).

Finalmente, o projeto inteiro foi desenvolvido com TypeScript, um derivado do JavaScript.

2.3 Tecnologias Existentes vs Tecnologias Utilizadas

Nesta secção procede-se à comparação entre as tecnologias utilizadas no desenvolvimento do projeto e outras tecnologias existentes. Serão apresentadas algumas distinções relevantes, de forma a justificar as escolhas tecnológicas efetuadas.

2.3.1 Base de dados relacional e não relacional

As bases de dados relacionais organizam os dados em tabelas. Cada linha da tabela representa um registo único, e cada coluna representa um atributo específicos dos dados. A principal característica de uma base de dados relacional é a possibilidade de criar relações entre estas tabelas, permitindo que os dados sejam ligados e organizados de forma estruturada. O SQL (Structured Query Language) é a linguagem usada para interagir com uma base de dados relacional. [4] Exemplos de bases de dados relacionais incluem o Oracle[5], MySQL[6], Microsoft SQL Server[7] e PostgreSQL[8]. [4]

As bases de dados não relacionais, também conhecidas como base de dados NoSQL(Não apenas SQL) seguem uma abordagem mais flexivel. Em vez de utilizarem tabelas tradicionais, recorrem a diferentes modelos de dados, como pares chave-valor, documentos JSON, colunas ou grafos. Esta flexibilidade permite que as bases de dados não relacionais tratem diferentes tipos de dados e grandes volumes de dados tornando-as adequadas para Big Data e aplicações web em tempo real. [4] Exemplos de bases de dados não relacionais incluem o MongoDB[9] e o Redis[10]. [4]

Tabela 2.2: Comparação entre bases de dados relacionais e não relacionais

Critério	**Base de Dados Relacional (SQL)**	**Base de Dados Não Relacional (NoSQL)**
Modelo de Dados	Baseado em tabelas (linhas e colunas)	Modelos flexíveis: documento, chave-valor, gráfico, coluna-família
Esquema	Estrutura rígida e pré-definida (schema fixo)	Esquema dinâmico e flexível
Linguagem de Consulta	Structured Query Language (SQL)	Linguagens específicas ou APIs próprias
Escalabilidade	Vertical (potenciar um único servidor)	Horizontal (distribuição por vários servidores)
Integridade	Elevada integridade e consistência de dados ((Atomicity, Consistency, Isolation, and Durability) (ACID))	Maior tolerância a inconsistências temporárias (eventual consistency)
Casos de Uso Comuns	Aplicações empresariais, sistemas de gestão, Enterprise Resource Planning (ERP)	Big Data, aplicações web em tempo real, redes sociais

2.3.2 Node.js vs .NET

https://www.techmagic.co/blog/node-js-vs-net-what-to-choose

[5] https://www.oracle.com/
[6] https://www.mysql.com/
[7] https://www.microsoft.com/pt-pt/sql-server
[8] https://www.postgresql.org/
[9] https://www.mongodb.com/
[10] https://redis.io/software/

Node.js e .NET são duas frameworks amplamente utilizadas no desenvolvimento de aplicações web. Para efeitos desta comparação, optou-se por utilizar o .NET, uma vez que já tinha sido previamente explorado no âmbito do projeto da unidade curricular de LAPR5.

O Node.js é uma plataforma de execução baseada em JavaScript, orientada a eventos, o que a torna particularmente eficiente em aplicações com elevada carga de entrada/saída (I/O). [5]

Por outro lado, o .NET é uma framework de desenvolvimento de software, utilizada para desenvolver uma ampla variedade de aplicações. Além disso, o .NET integra-se perfeitamente com ferramentas de desenvolvimento modernas e serviços na *cloud*, tornando-se uma boa escolha para o desenvolvimento de software moderno.[5]

O .NET proporciona uma arquitetura flexível que permite aos programadores criar aplicações web modulares. Isto possibilita uma manutenção e atualizações mais simples, bem como tempos de desenvolvimento mais curtos.[5]

A Tabela 2.3 apresenta uma comparação direta entre estas duas tecnologias em diferentes critérios.

Tabela 2.3: Comparação entre Node.js e .NET

Critério	**Node.js**	**.NET**
Modelo de execução	Assíncrono, baseado em event loop e I/O não bloqueante	Compilado com suporte a multithreading e execução paralela
Desempenho	Ótimo para operações I/O-bound; menos eficiente em CPU-bound	Excelente desempenho em tarefas intensivas de Central Processing Unit (CPU)
Escalabilidade	Alta escalabilidade horizontal com clusters	Escalabilidade vertical e horizontal com suporte nativo
Produtividade	Rápido para prototipagem e desenvolvimento com JavaScript full-stack	Ferramentas robustas (ex.: Visual Studio), ideais para grandes equipas
Ecossistema	Grande número de pacotes via NPM[11]; comunidade ativa	Ecossistema maduro com NuGet[12] e forte suporte da Microsoft
Casos de uso	Aplicações web em tempo real, microserviços, API Representational State Transfer (API REST)	Aplicações empresariais, serviços financeiros, IoT, integrações complexas
Segurança	Requer configuração manual de várias medidas de segurança	Funcionalidades de segurança nativas e suporte institucional

2.3.3 Angular VS React VS NextJS

O Angular é uma *framework* completa de *frontend*, desenvolvido para a criação de aplicações dinâmicas de página única (Singe Page Application (SPA)). Baseia-se em *TypeScript*, um superset de *JavaScript*, permitindo uma verificação de tipos robusta e maior escalabilidade. [6]

Principais Características:

- Ligação bidirecional de dados (*Two-way data binding*);[6]

- Injeção de dependências;[6]
- Roteamento integrado e gestão de formulários;[6]
- Ferramenta de linha de comandos (Command Line Interface (CLI)) abrangente para estruturação de projetos;[6]
- Programação reativa com recurso à biblioteca RxJS;[6]
- Suporte para Aplicações Web Progressivas (Progressive Web Apps (PWA)).[6]

O React é uma biblioteca JavaScript popular desenvolvida e mantida pela *Meta Platforms, Inc.*[13]

Capítulo 3

Análise do Problema

Neste capítulo, descreve-se o processo de análise que sustentou o desenvolvimento da solução proposta no âmbito do projeto. Explica-se como o trabalho foi estruturado e justificam-se as decisões tomadas ao longo das várias etapas que culminaram na construção da aplicação.

Começa-se por introduzir o problema e os principais desafios que motivaram a sua abordagem. Em seguida, são identificados os requisitos funcionais e não funcionais considerados mais relevantes para a definição da estrutura lógica do sistema.

3.1 Domínio do Problema

A gestão de propostas e orçamentos constitui um elemento central para a eficiência das agências de viagens, sobretudo em contextos onde a personalização do atendimento ao cliente é valorizada como um fator diferenciador.

Clientes que valorizam um serviço personalizado, com propostas adaptadas às suas preferências e apresentadas de forma visualmente apelativa, tendem a ter uma maior probabilidade de conversão em vendas.

A capacidade de uma agência para criar propostas de forma rápida, sem comprometer a personalização e a qualidade da experiência oferecida ao cliente, representa um fator estratégico de competitividade. Este equilíbrio entre eficiência operacional e excelência na apresentação comercial é especialmente relevante em mercados onde a atenção ao detalhe e a experiência do utilizador são tão valorizadas quanto o próprio conteúdo da viagem.

Na Tabela 3.1 apresenta-se o glossário dos principais conceitos utilizados no backoffice da aplicação, bem como o respetivo modelo de domínio 3.1.

3.1.1 Glossário e Modelo Domínio - Backoffice

Tabela 3.1: Glossário Backoffice

TEA	Descrição
Backoffice	Área da aplicação destinada à gestão interna, utilizada por administradores ou colaboradores para configurar, monitorizar e operar funcionalidades empresariais não acessíveis ao utilizador final.
Admin	Abreviatura de "Administrador", pessoa que tem privilégios dentro da plataforma e que é responsável por gerir os utilizadores.
Travel Designer	Colaborador responsável pela criação, personalização e gestão de propostas de viagem adaptadas às necessidades dos clientes.
Support Team	Colaborador responsável por prestar apoio aos utilizadores da aplicação, esclarecendo dúvidas, resolvendo problemas e garantindo o bom funcionamento da plataforma.
Group	Conjunto de utilizadores do sistema que partilham o acesso e utilização da aplicação de forma colaborativa, geralmente associados a uma mesma entidade ou equipa.
Proposal	Documento ou interface interativa que apresenta uma sugestão de viagem personalizada, criada com base nas necessidades e preferências do cliente.
Pack	Conjunto de itens agrupados com o objetivo de oferecer diferentes opções de viagem dentro de uma proposta, permitindo variações de preço, serviços e nível de personalização.
Travelers	Conjunto de pessoas para as quais a proposta de viagem é elaborada.
Destination	Local ou conjunto de locais para os quais a proposta de viagem é elaborada.
Hotel	Estadia ou conjunto de estadias incluídas na proposta de viagem.
Flight	Voo ou conjunto de voos incluídos na proposta de viagem.
TripElements	Conjunto de elementos complementares à viagem principal.
Transfer	Serviços de transporte.
Activity	Atividades adicionais incluídas na proposta de viagem.
Insurance	Seguro de viagem.
Itinerary	Organização cronológica da viagem.
Celebration	Eventos comemorativos.
Extra	Informação complementar ou apontamentos adicionais.

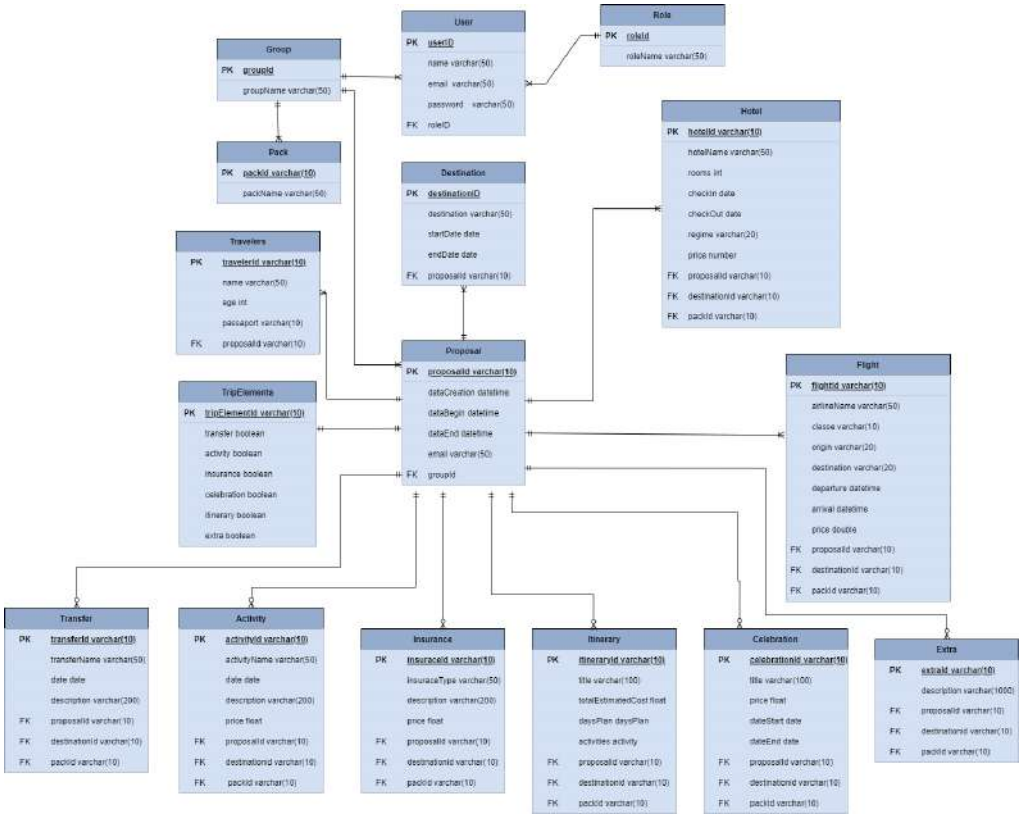

Figura 3.1: Diagrama de classes do backoffice da aplicação

3.2 Requisitos funcionais e não funcionais

Este projeto tem como objetivos principais as seguintes funcionalidades:

- Permitir aos utilizadores (Administradores e Travel Designers) a possibilidade de criar propostas de viagens de acordo com as exigências dos clientes
- Permitir aos clientes verificar as suas propostas de viagem de forma interativa

3.2.1 Gestão de Utilizadores e Propostas de Viagem - Backoffice

Nesta secção estão descritas as principais User Stories que orientaram a conceção e implementação da componente de backoffice da aplicação (ver Tabela 3.2).

Tabela 3.2: User Stories da componente de Backoffice

User Story	Papel	Descrição
US00	Todos os Utilizadores	Fazer login com a conta Google para acesso facilitado à aplicação
US01	Todos os Utilizadores	Fazer login na aplicação com credenciais válidas
US02	Todos os Utilizadores	Aceder apenas à dashboard correspondente ao meu perfil

User Story	Papel	Descrição
US03	Administrador	Criar, editar e remover utilizadores para gerir os acessos ao sistema
US04	Administrador	Convidar utilizadores para o meu grupo de trabalho dentro da aplicação
US05	Administrador, Travel Designer	Criar, editar e remover packs para associar às propostas de viagem
US06	Administrador, Travel Designer	Criar uma nova proposta de viagem para iniciar o planeamento personalizado
US06.1	Administrador, Travel Designer	Adicionar, editar, remover viajantes a uma proposta
US06.2	Administrador, Travel Designer	Adicionar, editar, remover destinos a uma proposta
US06.3	Administrador, Travel Designer	Adicionar, editar, remover elementos de viagem extra (Transfer, Atividades, Seguros, Celebrações, Itinerários, Extras)
US06.4	Administrador, Travel Designer	Adicionar, editar, remover hotéis a uma proposta
US06.5	Administrador, Travel Designer	Adicionar, editar, remover voos a uma proposta
US06.6	Administrador, Travel Designer	Adicionar, editar, remover transfers a uma proposta (Apenas se selecionado nos elementos de viagem extra)
US06.7	Administrador, Travel Designer	Adicionar, editar, remover atividades a um destino específico da proposta (Apenas se selecionado nos elementos de viagem extra)
US06.8	Administrador, Travel Designer	Adicionar, editar, remover seguros a um destino específico da proposta (Apenas se selecionado nos elementos de viagem extra)
US06.9	Administrador, Travel Designer	Adicionar, editar, remover celebrações a um destino específico da proposta (Apenas se selecionado nos elementos de viagem extra)
US06.10	Administrador, Travel Designer	Adicionar, editar, remover um itinerário a um destino específico da proposta (Apenas se selecionado nos elementos de viagem extra)
US06.11	Administrador, Travel Designer	Criar um itinerário com recurso à inteligência artificial(Artificial Intelligence (AI)) (Apenas se selecionado nos elementos de viagem extra)
US06.12	Administrador, Travel Designer	Adicionar, editar, remover extras a um destino específico da proposta (Apenas se selecionado nos elementos de viagem extra)
US7	Administrador, Travel Designer	Rever a proposta e enviar para o cliente (A.C: envio por Email com Simple Mail Transfer Protocol (SMTP))
US8	Administrador	Listar todas as propostas criadas por um grupo

User Story	Papel	Descrição
US9	Administrador, Travel Designer	Duplicar uma proposta já criada e submetida
US10	Administrador, Travel Designer	Visualizar todas as propostas do grupo ao qual pertenço

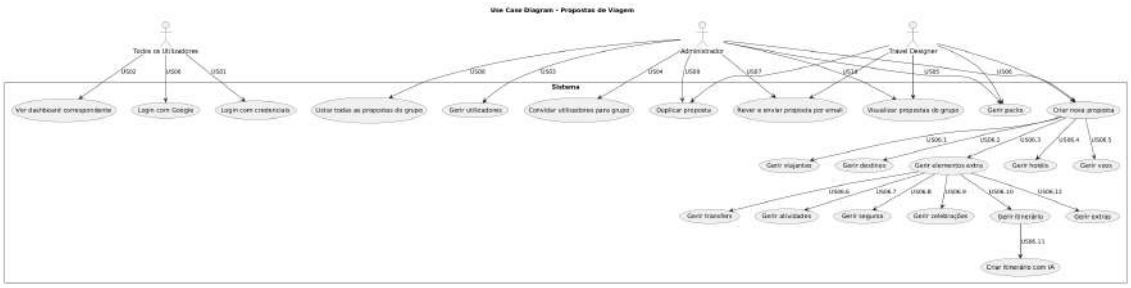

Figura 3.2: Diagrama de Casos de Uso - Backoffice

3.3 Requisitos Não Funcionais

Além dos requisitos funcionais, foram definidos diversos requisitos não funcionais com base no modelo Functionality, Usability, Reliability, Performance, and Supportability (FURPS+), que categoriza os requisitos em: Funcionalidade, Usabilidade, Confiabilidade, Desempenho, Suportabilidade e outras restrições técnicas. Abaixo apresenta-se a descrição dos principais requisitos não funcionais considerados neste projeto.

Funcionalidade

- Garantir a segurança dos dados sensíveis através de autenticação com tokens JSON Web Token (JWT) armazenados em cookies com flag `HttpOnly`.
- Garantir a integridade dos dados processados e persistidos no sistema.
- Suportar diferentes perfis de utilizador (Administrador, Travel Designer e Cliente), com permissões distintas.

Usabilidade

- A interface deve ser intuitiva, clara e fácil de utilizar, mesmo para utilizadores com pouca experiência técnica.
- Navegação fluida entre páginas e formulários, com feedback visual consistente.
- Validação imediata de dados introduzidos no frontend para evitar erros e melhorar a experiência do utilizador.

Confiabilidade

- A aplicação deve garantir disponibilidade contínua e comportamento estável.
- Os dados submetidos devem ser validados tanto no frontend como no backend.

- Os erros devem ser tratados e reportados de forma clara ao utilizador, sem comprometer o funcionamento global do sistema.

Desempenho

- O tempo de resposta da aplicação deve ser reduzido, mesmo em cenários com elevado número de propostas.
- O carregamento de páginas e a comunicação com a API devem ser otimizados para proporcionar uma experiência fluida.

Suportabilidade

- Adotou-se uma arquitetura modular e escalável, baseada em boas práticas como a *Clean Architecture*.
- O código foi estruturado de forma a facilitar a manutenção, testes e adição de novas funcionalidades.
- A aplicação é compatível com os principais navegadores modernos.

Capítulo 4

Desenho da Solução

Neste capítulo, é apresentado o desenho da solução proposta para responder ao problema identificado no capítulo anterior anterior. A arquitetura do sistema é descrita com recurso aos modelos C4 e 4+1, permitindo uma visão estruturada dos diferentes componentes e das suas interações. Pretende-se, com esta abordagem, definir uma base arquitetónica sólida, escalável e bem integrada na restante plataforma, de forma a garantir a sua eficácia e manutenção futura.

4.1 Arquitetura da Aplicação

4.1.1 Nível 1 - Vista Lógica

Neste nível, identificamos os agentes que interagem com as plataformas. Este diagrama evidencia a relação dos intervientes com as plataformas. Como o projeto é dividido entre um backoffice (destinado à agência de viagens). Foram criados o diagrama de vista lógica nível 1 para o backoffice 4.1. Este diagrama mostra as interações das várias interfaces de utilizador com a aplicação.

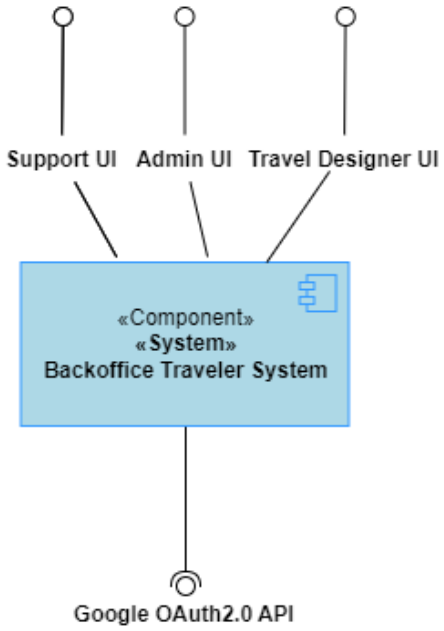

Figura 4.1: Backoffice: Vista Lógica - Nível 1

4.1.2 Nível 2 - Vista Lógica

A arquitetura apresentada refere-se ao Backoffice da aplicação Traveler Web Application, uma vez que a parte do *Frontoffice* foi desenvolvida por outro estagiário.

O sistema do Backoffice é composto por três contentores principais: `TravelerFE`, `TravelerBE` e `Database`, conforme ilustrado na Figura 4.2.

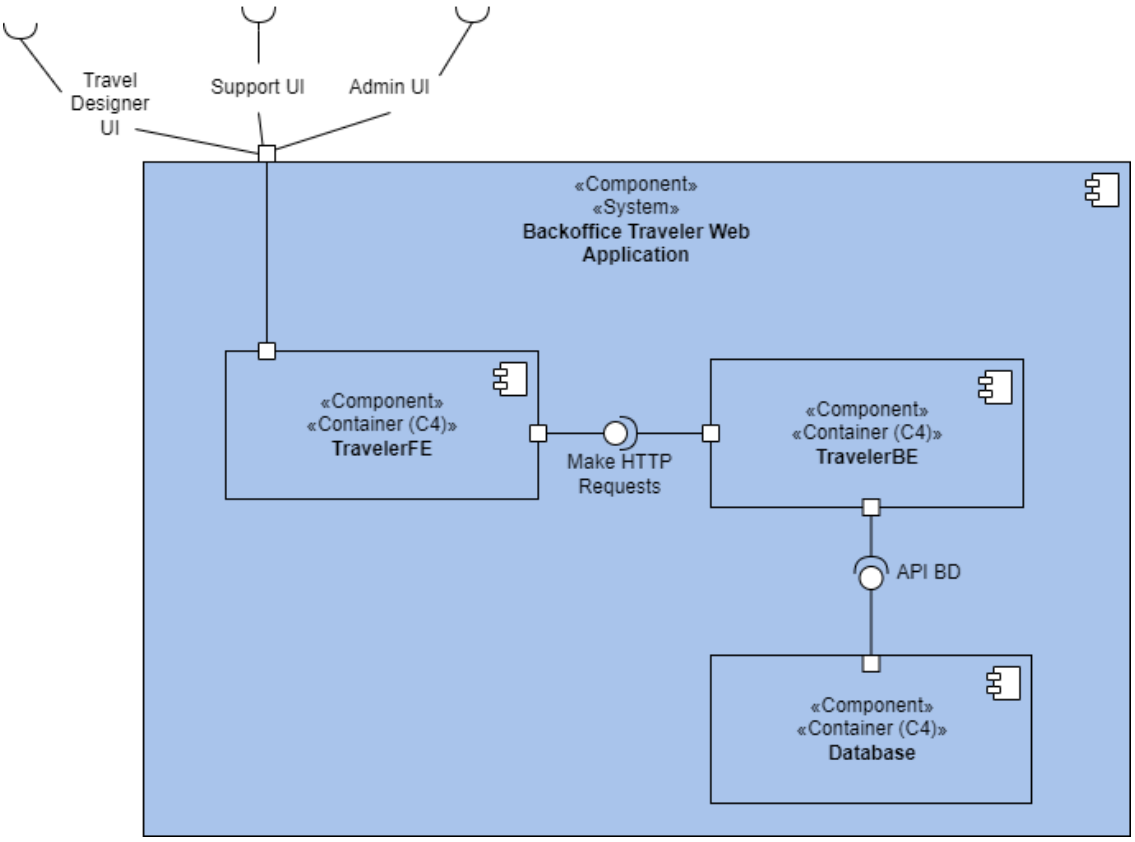

Figura 4.2: Backoffice: Vista Lógica - Nível 2

- **TravelerFE** representa o *frontend* do Backoffice, responsável pela interação com os utilizadores com base em três interfaces distintas: *Travel Designer UI*, *Support UI* e *Admin UI*. Este contentor envia pedidos HyperText Transfer Protocol (HTTP) ao `TravelerBE`, funcionando como o ponto de entrada dos utilizadores da organização.
- **TravelerBE** corresponde ao *backend*, responsável por processar a lógica de negócio do sistema. Este contentor expõe uma API REST que recebe os pedidos do `TravelerFE`, executa as operações necessárias e comunica com a base de dados. É responsável por garantir a consistência e integridade da informação transmitida ao frontend.
- **Database** é o contentor responsável pela persistência dos dados da aplicação. Todos os acessos à base de dados são realizados exclusivamente através do `TravelerBE`, assegurando o controlo de acesso, isolamento e integridade dos dados.

4.1.3 Nível 2 - Vista Física

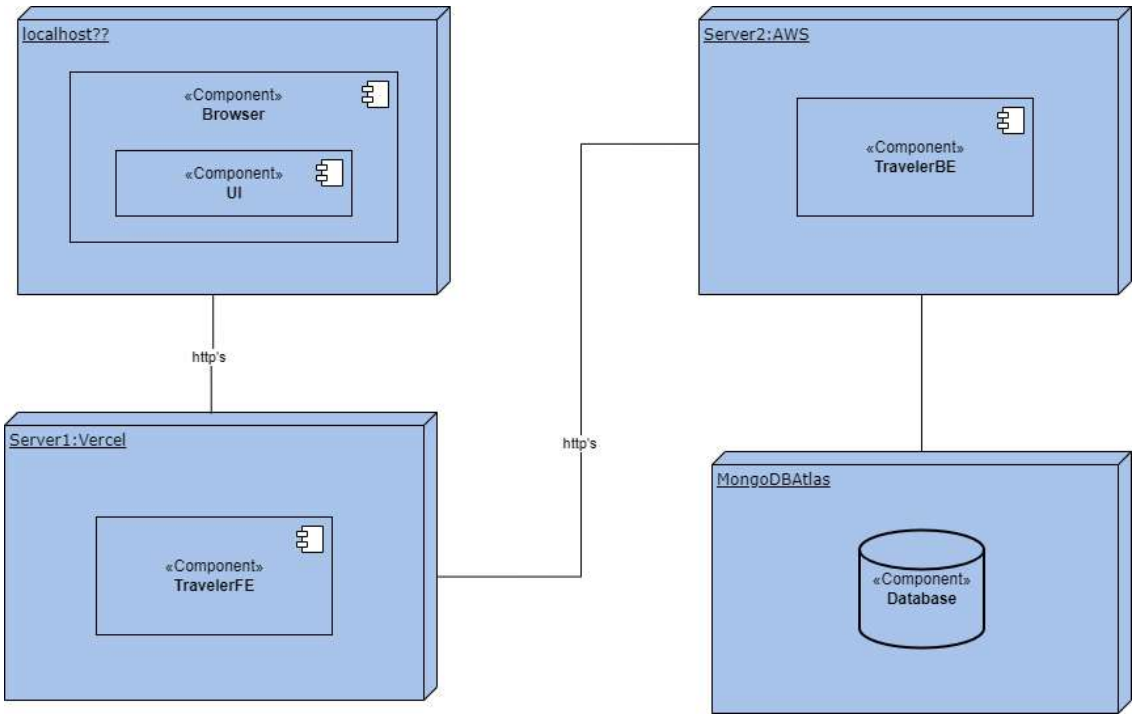

Figura 4.3: Backoffice: Vista Fisica

A Figura 4.3 representa a vista física da aplicação web Traveler, ilustrando a distribuição dos principais componentes do sistema nos diferentes nós de infraestrutura. Esta arquitetura baseia-se num modelo cliente-servidor, onde os utilizadores interagem com a aplicação através de um Browser.

O **navegador** comunica via protocolo HTTPS com o frontend da aplicação, alojado na plataforma Vercel. Este componente, denominado `TravelerFE`, é responsável pela renderização da interface e pela gestão da interação com o backend.

O **frontend** comunica, igualmente por HTTPS, com o backend da aplicação com o componente `TravelerBE`, que se encontra alojado num servidor na AWS. Este componente trata da lógica de negócio, e contém toda a lógica operacional do sistema.

Finalmente, o **backend** interage com a base de dados hospedada em MongoDB Atlas, responsável pelo armazenamento persistente de dados relacionados com utilizadores, propostas, itinerários e outras entidades da aplicação.

4.1.4 Nível 3 - Vista Implementação

Vista Implementação - Backend

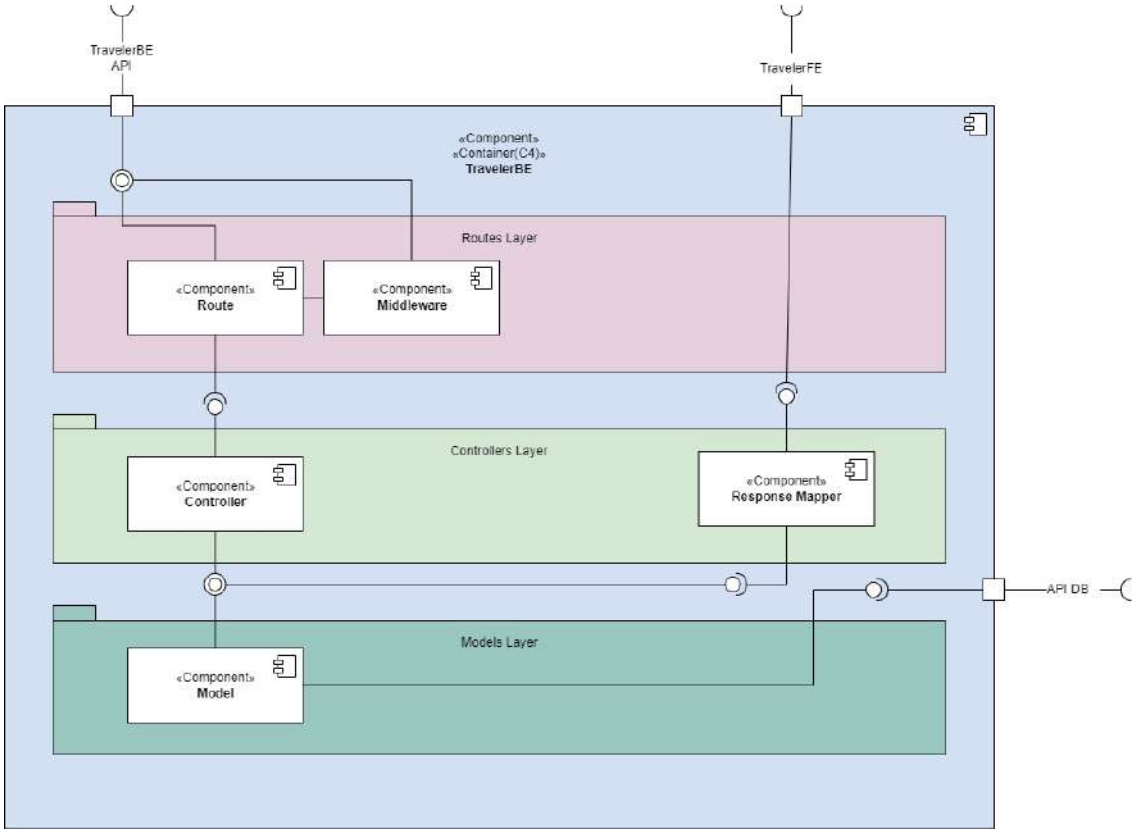

Figura 4.4: TravelerBE: Vista Implementação

A Figura 4.4 representa a vista de implementação do componente **TravelerBE**, responsável pela lógica de negócio e tratamento dos pedidos no backend da aplicação. Esta componente segue uma arquitetura modular em camadas (*layered architecture*), promovendo a separação de responsabilidades e facilitando a manutenção e evolução do sistema.

A primeira camada **Routes Layer** é responsável por expor os pontos de entrada da API, tratando os pedidos HTTP recebidos através de rotas bem definidas. Esta camada inclui também `middlewares`, utilizados para executar operações transversais como autenticação, validação de dados e controlo de permissões, antes da passagem do pedido às camadas inferiores.

A segunda camada **Controllers Layer** centraliza a lógica de negócio da aplicação. Os `controllers` recebem os dados processados pelas rotas e executam as operações correspondentes, de forma a interagir com os modelos de dados e coordenar o fluxo das operações. Esta camada inclui ainda um componente `Response Mapper`, responsável por normalizar e estruturar as respostas devolvidas ao frontend, de forma a assegurar consistência na comunicação entre componentes.

Por fim, a camada inferior **Models Layer** encapsula os `models`, que representam as entidades do domínio e estabelecem a ligação com a base de dados. É nesta camada que se realiza

a persistência dos dados, através de operações de leitura e escrita sobre a base de dados *MongoDB*, representado no diagrama como `API DB`.

Vista Implementação - Frontend

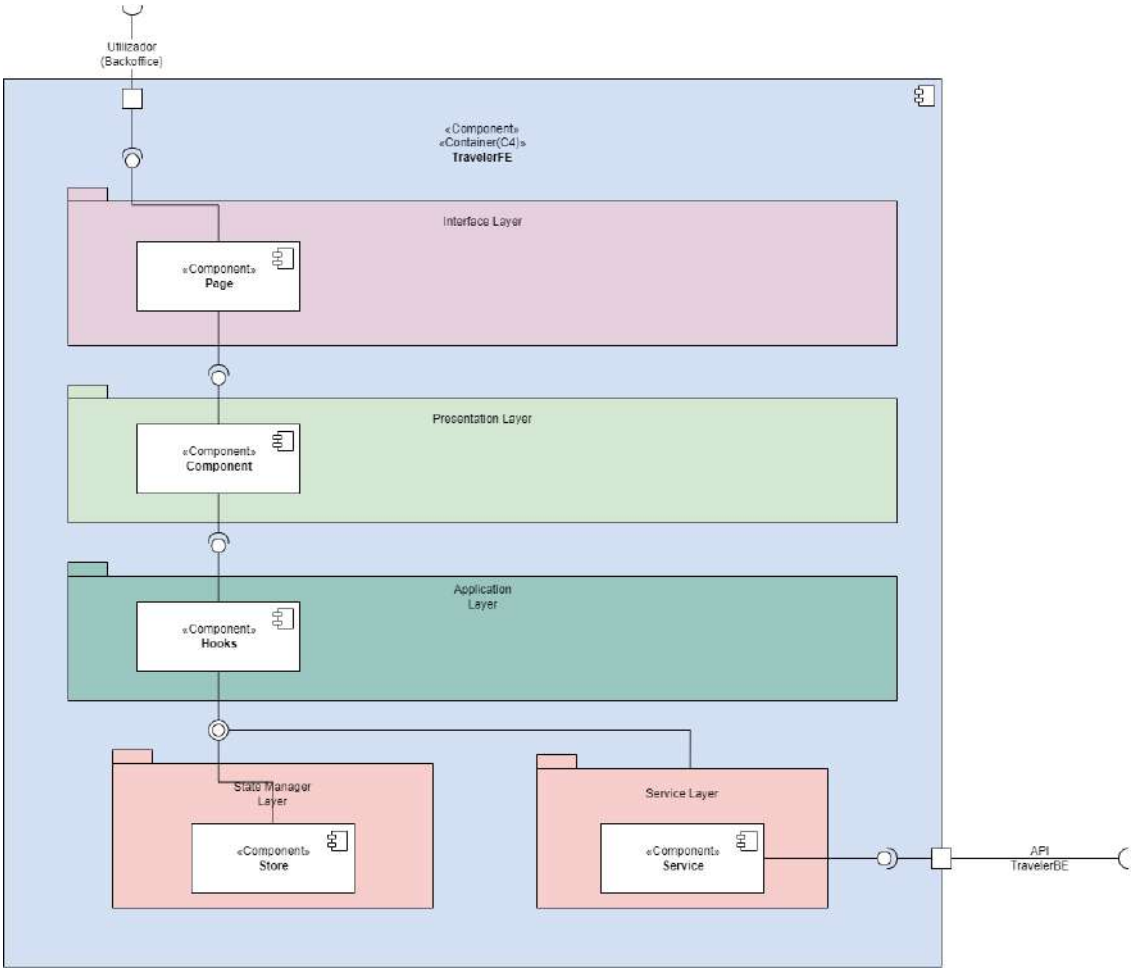

Figura 4.5: TravelerFE: Vista Implementação

A Figura 4.5 representa a vista de implementação da componente *TravelerFE*, correspondente ao frontend dO Backoffice da aplicação Traveler. Esta vista mostra a organização interna do sistema em diferentes camadas (*layers*), refletindo a separação de responsabilidades entre apresentação, gestão de estado, lógica de aplicação e comunicação com o backend.

A camada superior, **Interface Layer**, corresponde ao ponto de entrada da aplicação, sendo composta por páginas que definem as rotas e estruturam os ecrãs apresentados ao utilizador.

A **Presentation Layer** agrupa os componentes reutilizáveis que compõem a interface visual da aplicação. É nesta camada que se encontra o HTML que define a estrutura e apresentação das páginas, sendo responsável pela composição dos elementos visuais exibidos ao utilizador.

A **Application Layer** contém os *hooks* personalizados, responsáveis por coordenar a lógica entre os serviços (responsáveis pela comunicação com o backend) e a store (responsável pelo

estado da aplicação). Esta camada abstrai a complexidade da gestão de dados e fornece uma interface simplificada para as camadas superiores.

A **State Manager Layer** inclui a store da aplicação, implementada com recurso à biblioteca Zustand. Esta camada mantém o estado partilhado entre componentes, bem como as funções necessárias para o seu acesso e atualização.

A **Service Layer** encapsula a lógica de comunicação com o backend, sendo responsável por enviar pedidos HTTP para a API do *TravelerBE* e por tratar as respostas recebidas. Através desta camada, os dados são obtidos, criados, atualizados ou eliminados conforme necessário.

Capítulo 5

Implementação da Solução

Este capítulo descreve a implementação da solução proposta, detalhando as principais componentes desenvolvidas ao longo do projeto. A abordagem adotada dividiu o trabalho em duas partes complementares: uma dedicada ao *backoffice*, orientada para a gestão interna da aplicação, e outra ao *frontoffice*, focada na visualização das propostas por parte do cliente final. Apesar de funcionarem de forma independente, ambas as aplicações foram desenvolvidas com base numa estrutura comum, garantindo consistência arquitetónica, reutilização de componentes e coerência na experiência geral da solução.

5.1 Descrição da implementação

Com base na análise e desenho do problema, foram reunidas as condições necessárias para o desenvolvimento do projeto. Este processo tem como objetivo garantir a qualidade de acordo com os requisitos estabelecidos. Neste capítulo, serão apresentados os processos de engenharia das funcionalidades e os modelos desenvolvidos.

A aplicação backend foi desenvolvida com base numa arquitetura do tipo MVC (Model View Controller) adaptado, onde a separação de responsabilidades é feita em três camadas principais: *Routes*, *Controller* e *Model*. Os controladores acumulam funções típicas de serviços, tratando simultaneamente a lógica de negócio e a orquestração entre as rotas e os modelos. A persistência de dados é assegurada através do *MongoDB*, utilizando o *ODM Mongoose* (ver Figura 5.1).

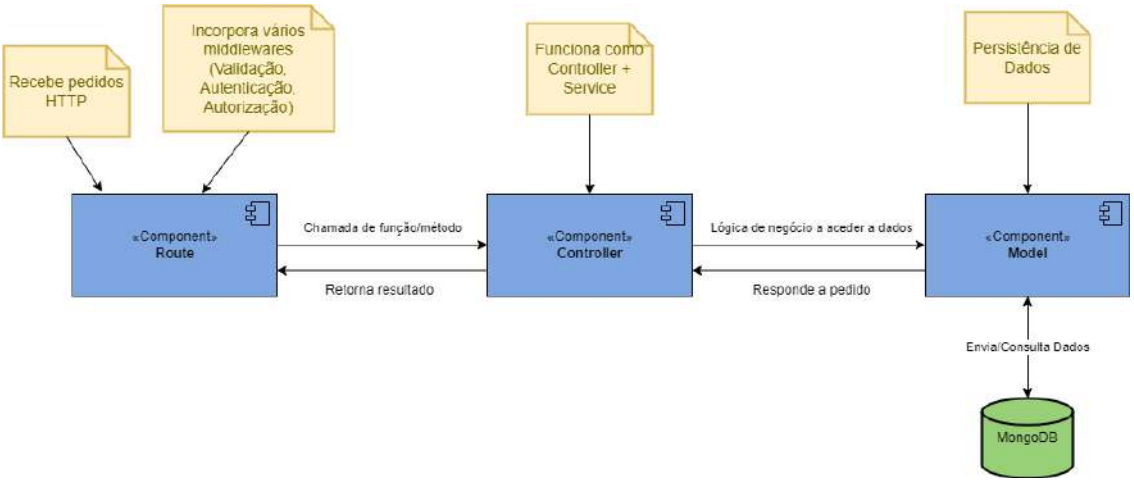

Figura 5.1: Arquitetura do Backend - MVC Adaptado

O frontend foi desenvolvida com recurso ao *React + Next.js*, seguindo uma arquitetura modular e baseada em componentes reutilizáveis. A estrutura da aplicação organiza-se em diretórios com responsabilidades bem definidas, permitindo uma separação clara entre apresentação, estado e lógica de acesso a dados.

O diretório *pages* contém a definição das rotas e páginas da aplicação, conforme o paradigma de roteamento baseado em ficheiros do Next.js. Os componentes reutilizáveis de interface encontram-se na pasta *components*, promovendo a reutilização dos componentes. Para encapsular a lógica reutilizável, foram criados *hooks* personalizados, permitindo, por exemplo, abstrair chamadas a serviços ou interações com o estado global.

A comunicação com o backend é realizada através de funções definidas na pasta *services*, responsáveis por encapsular os endpoints e isolar a lógica de acesso a dados. O estado global da aplicação é gerido com a biblioteca *Zustand*[1], cuja configuração se encontra no diretório *store*, permitindo um controlo simples, e reativo do estado partilhado entre componentes (ver Figura 5.2).

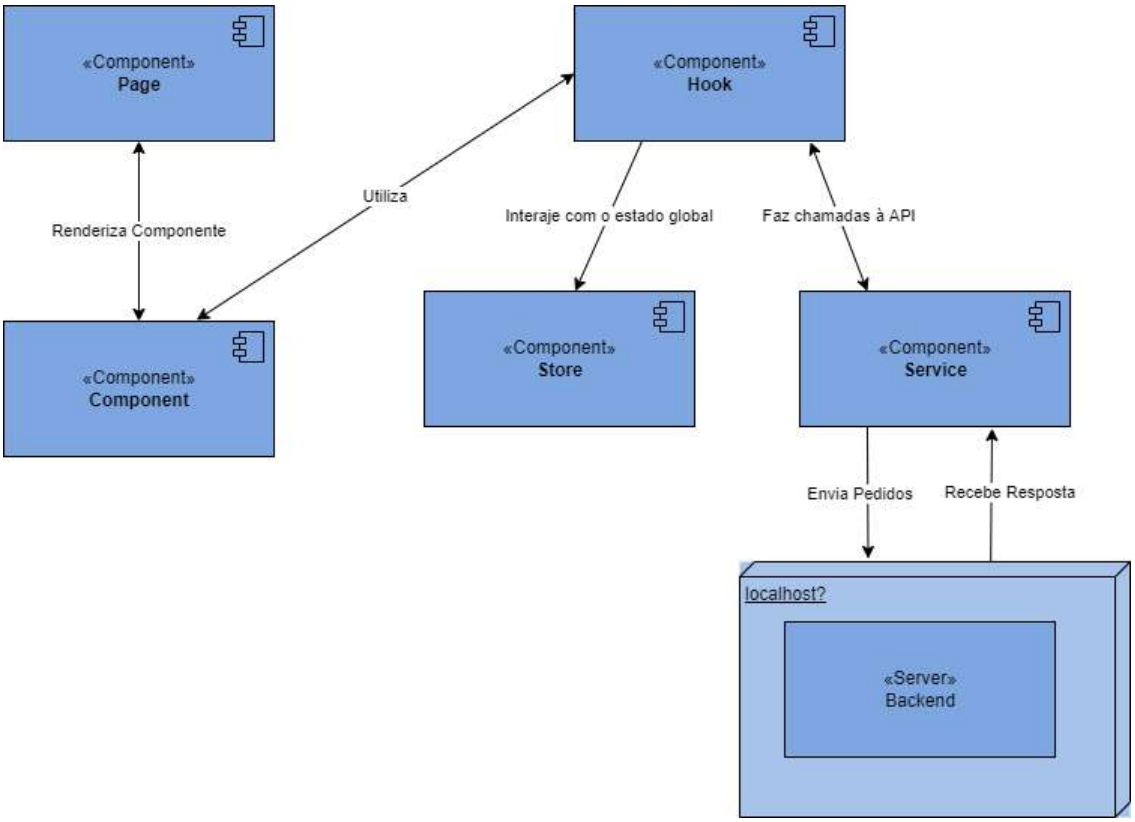

Figura 5.2: Arquitetura do Frontend - Arquitetura modular e baseado em componentes reutilizáveis

Devido à existência de várias *user stories*, nas secções seguintes serão documentadas apenas aquelas que se revelaram mais importantes para o projeto: quer pelo seu impacto funcional, pelo grau de complexidade, ou pela integração com APIs externas.

[1] `https://zustand-demo.pmnd.rs/`

5.1.1 US01 - Como utilizador da aplicação quero fazer login no sistema utilizando as minhas credenciais

Descrição: Esta user story descreve o processo de autenticação de um utilizador na aplicação através do fornecimento das suas credenciais (email e palavra-passe). O processo envolve tanto o frontend como o backend, sendo composto por múltiplas etapas de validação e resposta.

Fluxo de Interação (Frontend): O utilizador inicia o processo introduzindo o seu email e palavra-passe no formulário de login. A interface de utilizador (*UI*) invoca o *FrontendService*, que é responsável por enviar os dados para o backend através de uma requisição HTTP POST para o endpoint `/login`. Esta comunicação é implementada com a biblioteca *Axios*, embora tal detalhe esteja abstraído no diagrama de sequência.

O *FrontendService* processa a resposta do servidor:

- Em caso de sucesso (status 200), o token de autenticação (JWT) é enviado pelo backend num *cookie* HTTP-only e armazenado automaticamente pelo navegador.
- Em caso de erro (status `401` ou `404`), é apresentada uma mensagem de erro correspondente ao utilizador.

Fluxo de Interação (Backend): O backend expõe o endpoint `/login` na *AuthRoute*, que delega a lógica ao *AuthController*. Este, por sua vez, invoca o *UserModel* para procurar o utilizador na base de dados com base no email fornecido.

Os seguintes cenários são possíveis:

- **Utilizador não encontrado:** é devolvido um erro `404`.
- **Palavra-passe incorreta:** a comparação falha e é devolvido um erro `401`.
- **Autenticação bem-sucedida:** é gerado um *accessToken* (JWT), que é enviado ao cliente num cookie com as flags `HttpOnly`, `Secure` e `SameSite`, garantindo maior segurança.

Diagramas de Sequência: Dois diagramas de sequência foram elaborados:

- **Frontend:** representa a interação entre o utilizador, a UI e o serviço responsável por enviar a requisição para o backend (ver Figura 5.3).
- **Backend:** detalha a lógica de autenticação desde a receção da requisição até à validação dos dados e envio seguro do token (ver Figura 5.4).

Notas Técnicas:

- A comunicação entre frontend e backend é feita via HTTP, com o método `POST`.
- A biblioteca `axios`[2] é utilizada no *FrontendService* para realizar a chamada.
- O token JWT é armazenado no cliente em cookies com flag `HttpOnly`, protegendo contra ataques XSS.

[2] `https://axios-http.com/docs/intro`

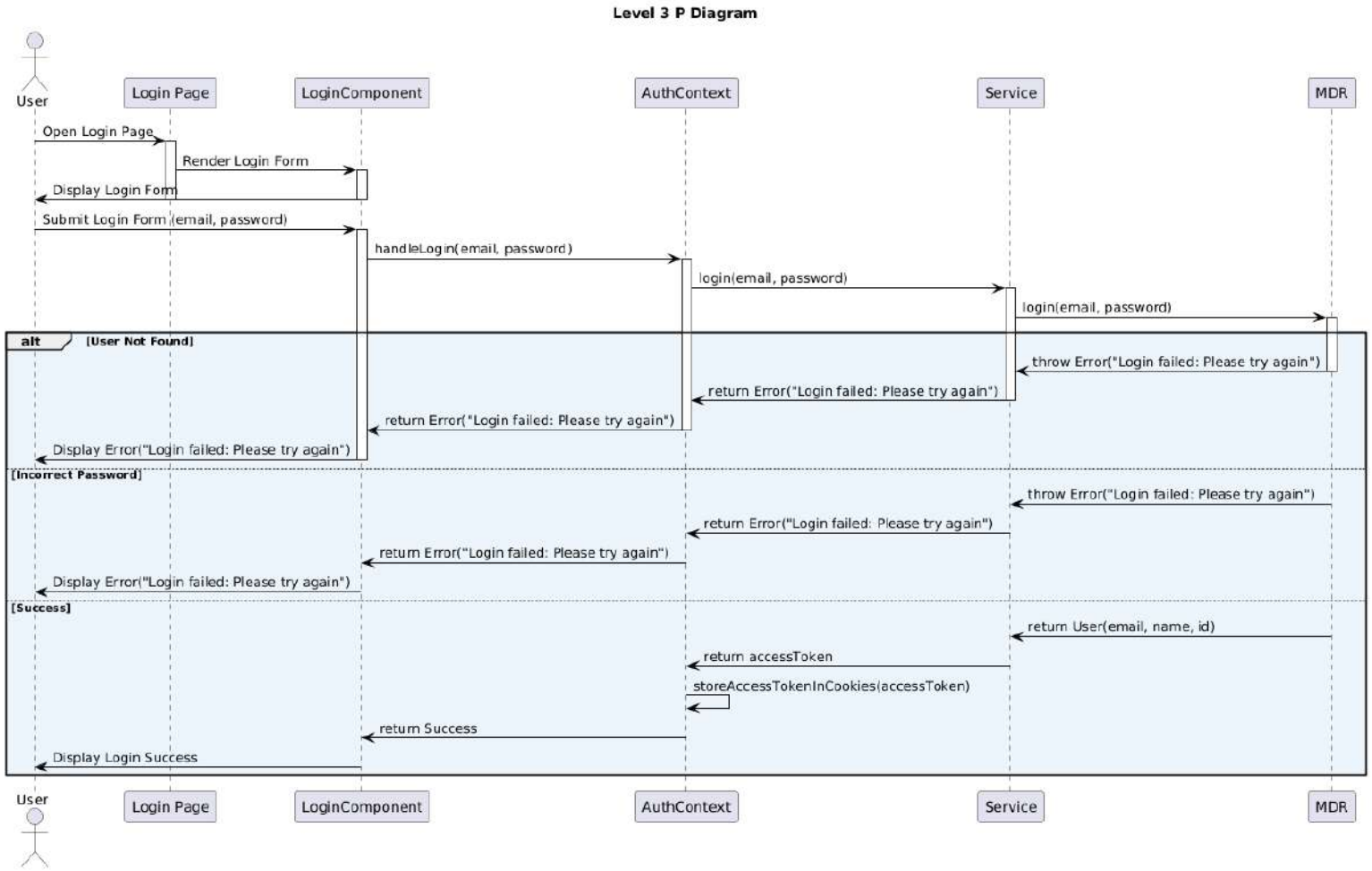

Figura 5.3: US01: Diagrama de Sequência do Frontend

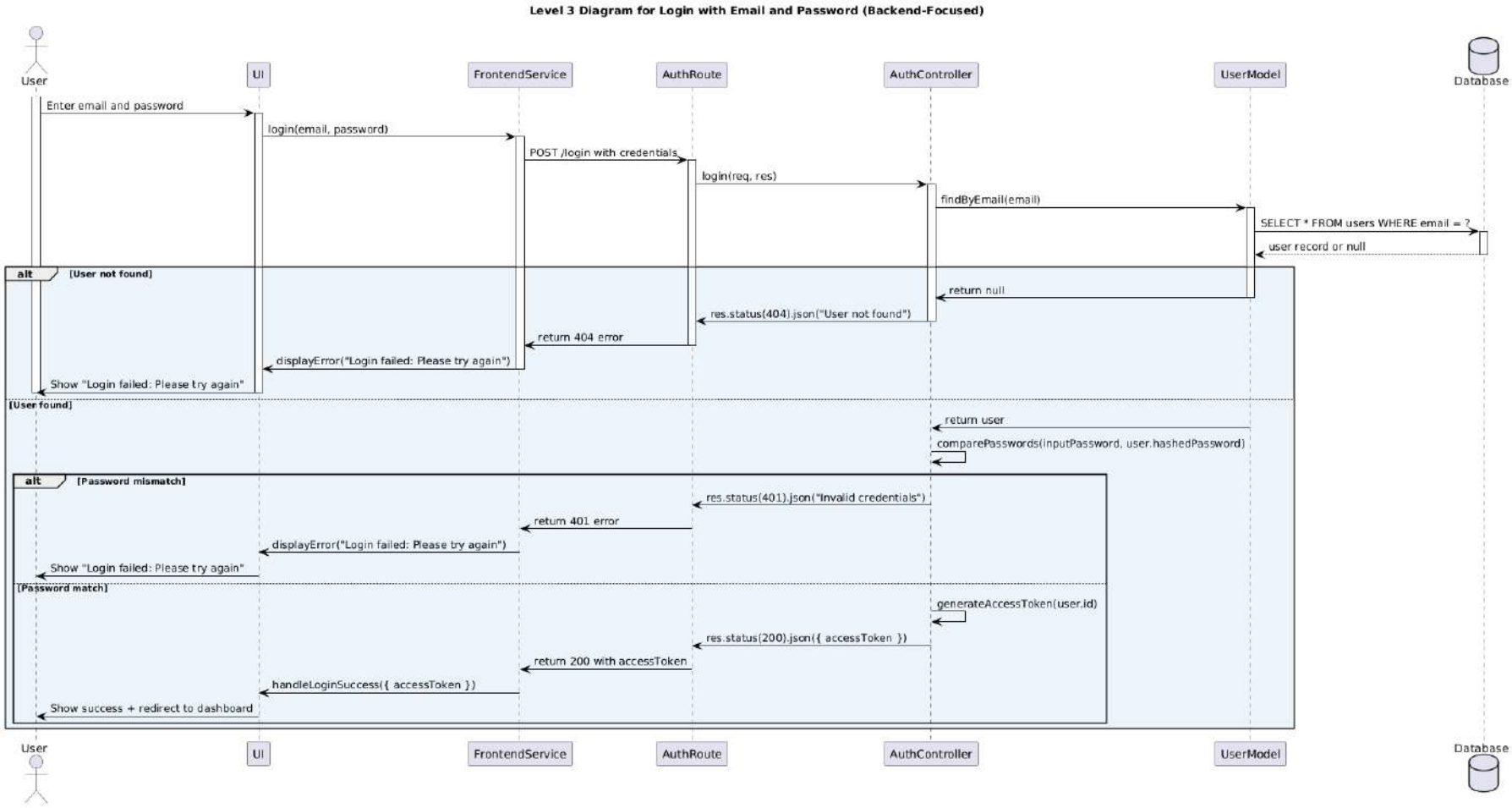

Figura 5.4: US01: Diagrama de Sequência do Backend

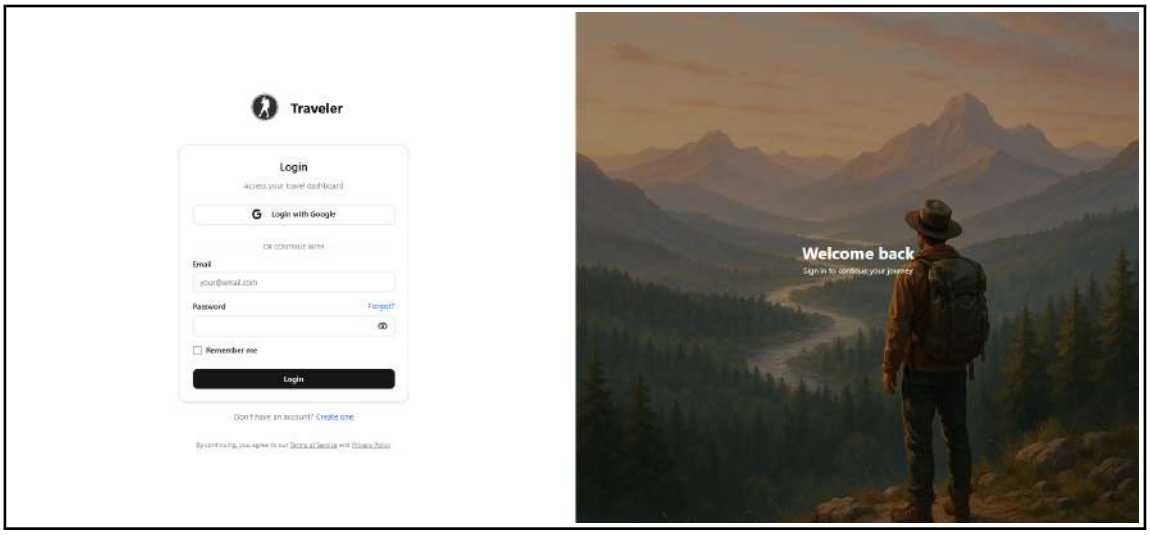

Figura 5.5: Interface de Login

5.1.2 US02 - Como utilizador da aplicação quero aceder apenas à minha dashboard

Descrição: Esta user story assegura que, após o login, cada utilizador é redirecionado exclusivamente para a sua *dashboard* correspondente, de acordo com o seu perfil de acesso. Esta funcionalidade garante isolamento, segurança e relevância da informação apresentada ao utilizador.

Perfis de Utilizador e Dashboards:

- **Administrador**
  - Visualização e gestão de todas as propostas existentes.
  - Acesso a todos os utilizadores e permissões.
  - Acesso a estatísticas globais do sistema.
- **Travel Designer**
  - Visualização das suas propostas atribuídas ou criadas.
  - Acesso a ferramentas de criação e edição de propostas.
  - Acesso à página de gestão de packs.
- **Support Team**
  - Acesso a histórico de apoio ao cliente.
  - Ferramentas de acompanhamento de pedidos de suporte.

Comportamento Esperado:

1. Após login, o sistema identifica o papel do utilizador (via token ou sessão).
2. O utilizador é redirecionado automaticamente para a dashboard correspondente.

3. É impedido o acesso a outras dashboards por controlo de permissões e *guards*.

Notas Técnicas:

- O controlo de acesso é garantido no frontend através de `Route Guards` e no backend por validação de permissões no JWT.
- A rota do dashboard é a mesma para cada role.
- O backend devolve o papel do utilizador no login, permitindo a renderização da interface adequada.

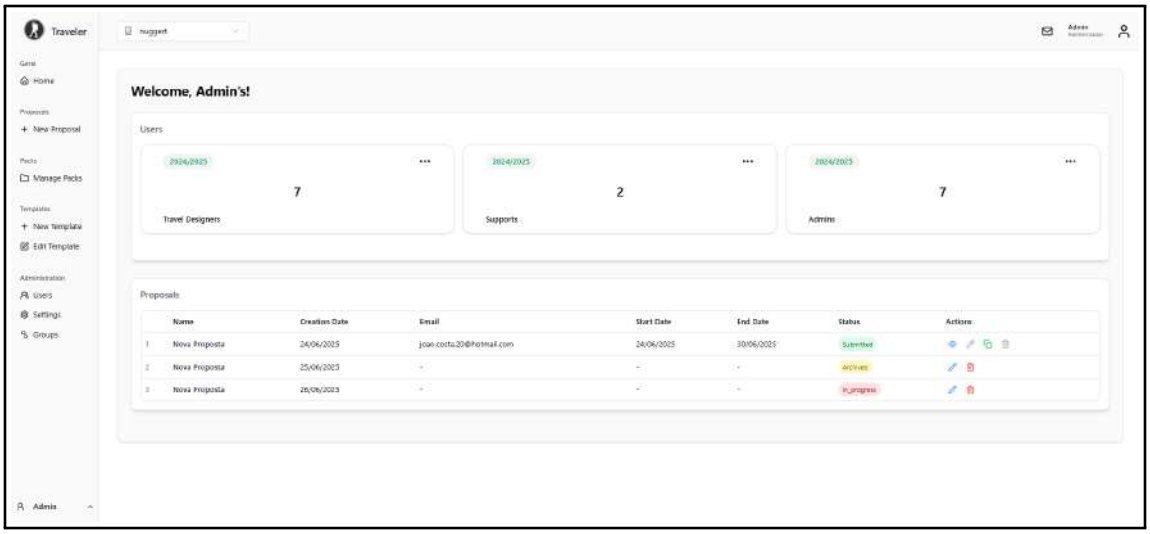

Figura 5.6: Dashboard do Administrador

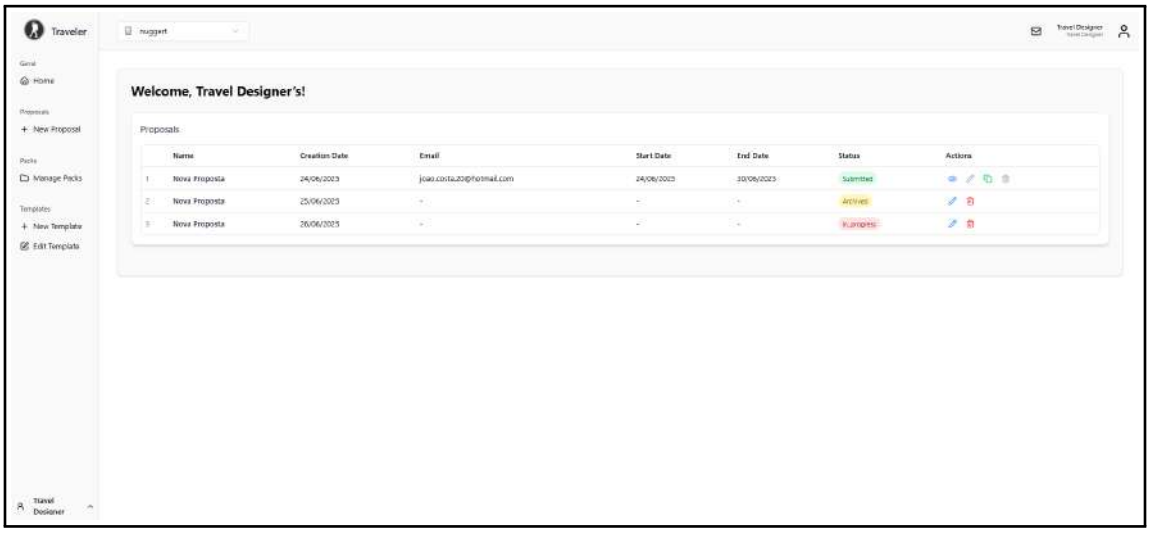

Figura 5.7: Dashboard do Travel Designer

Figura 5.8: Dashboard da Equipa de Suporte

5.1.3 US05 - Como Administrador/Travel Designer quero poder criar packs para associar às propostas de viagem

Descrição: Esta user story permite que utilizadores com perfil de Administrador ou Travel Designer criem múltiplos *packs* personalizados (por exemplo: *Low-cost*, *Standard*, *Premium*) dentro de uma proposta de viagem. Cada pack representa uma alternativa da proposta, com combinações diferentes de serviços e preços.

Fluxo de Interação (Frontend): O utilizador define os packs diretamente na página de configuração da proposta. A UI permite criar, editar ou remover packs, indicando o nome do pack. O *FrontendService* é responsável por atualizar o estado local (via Zustand) e, ao avançar para o próximo passo, submete os dados para o backend via `POST` para `/packs`.

Figura 5.9: Página de gestão de packs

Fluxo de Interação (Backend): O backend expõe endpoints RESTful para gerir packs, incluindo criação, edição e remoção. A rota `/packs` delega ao *PackController*, que lida com a validação dos dados e interage com o *PackModel*.

São considerados os seguintes cenários:

- **Criação bem-sucedida:** o pack é guardado com sucesso para ser utilizado em futuras propostas.
- **Validação inválida:** é devolvido um erro 400.
- **Edição ou remoção:** controladas por permissões, apenas disponíveis a perfis autorizados.

Diagramas de Sequência: Dois diagramas de sequência foram elaborados:

- **Frontend:** ilustra a interação entre o utilizador, UI, estado local e o envio final dos dados (ver Figura 5.10).
- **Backend:** representa a lógica completa do tratamento dos dados dos packs após submissão (ver Figura 5.11).

Notas Técnicas:

- O frontend utiliza Zustand para guardar temporariamente os packs antes da submissão.
- O backend valida os dados, incluindo se o utilizador está autorizado a criar/modificar/apagar os packs.

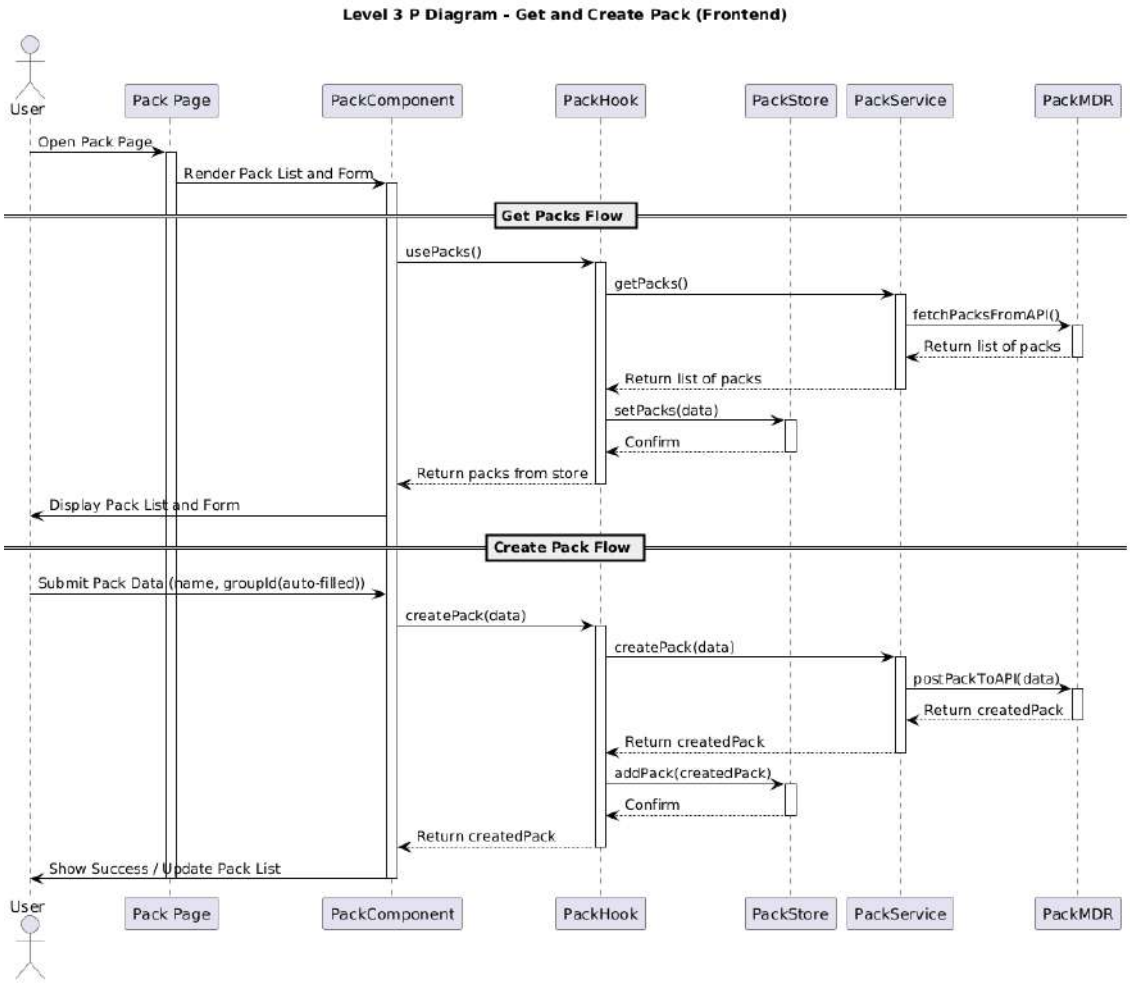

Figura 5.10: US05: Diagrama de Sequência do Frontend

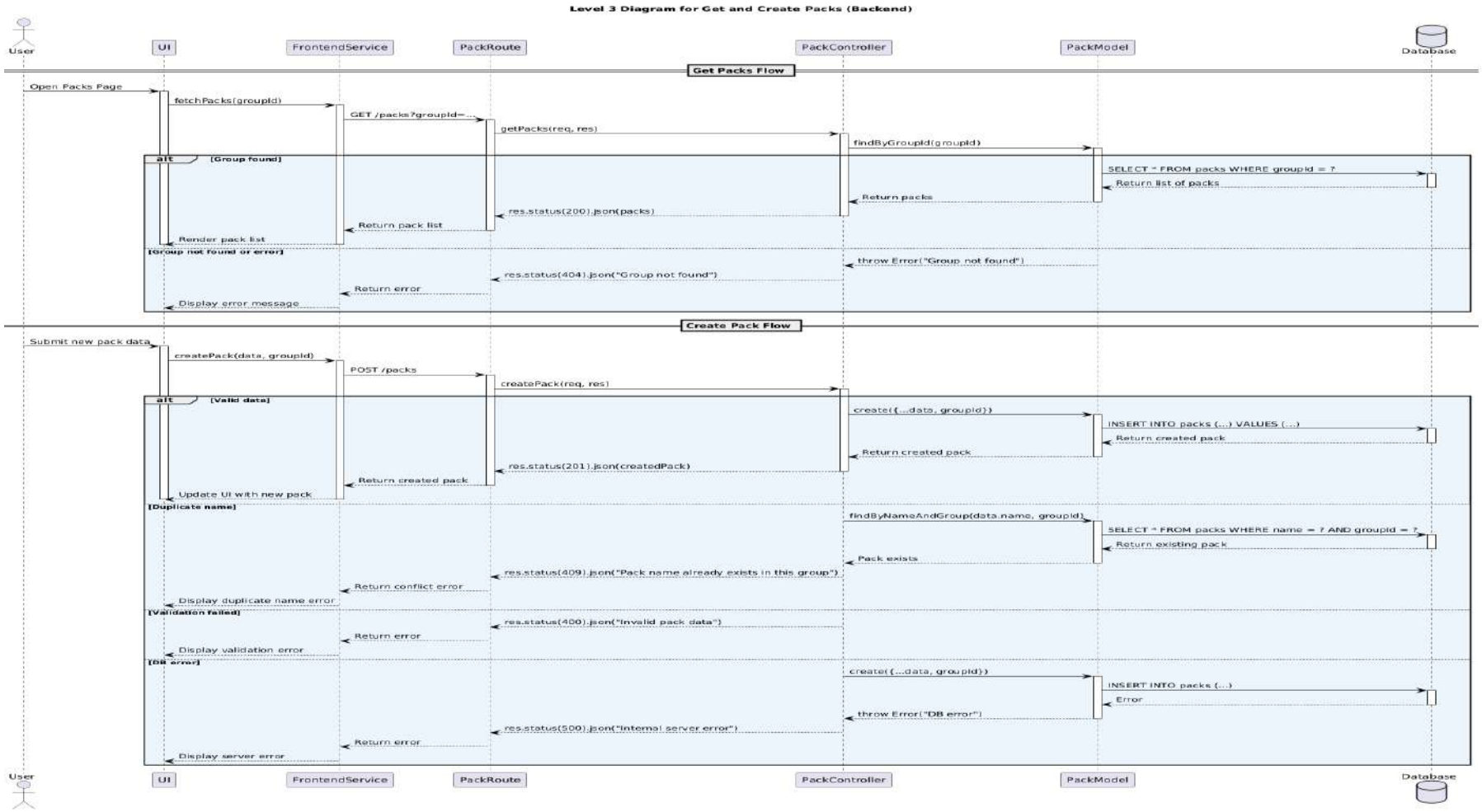

Figura 5.11: US05: Diagrama de Sequência do Backend

5.1.4 US06 - Como Administrador/Travel Designer quero criar uma nova proposta de viagem

Descrição: A criação de uma proposta de viagem é um processo complexo que envolve a gestão de vários componentes interligados, como viajantes, destinos, alojamento, transporte, etc. Esta user story, representa um *épico*, agrupa um conjunto de funcionalidades essenciais que permitem ao Administrador ou Travel Designer montar e personalizar uma proposta de forma modular e progressiva. As sub user stories associadas abordam as operações de criação, edição e remoção, algumas das quais dependem da seleção prévia de elementos de viagem extra.

Início do Processo: Para iniciar a criação de uma proposta, o utilizador deve clicar no botão *"New Proposal"*, disponível na barra lateral da dashboard (ver Figura 5.12).

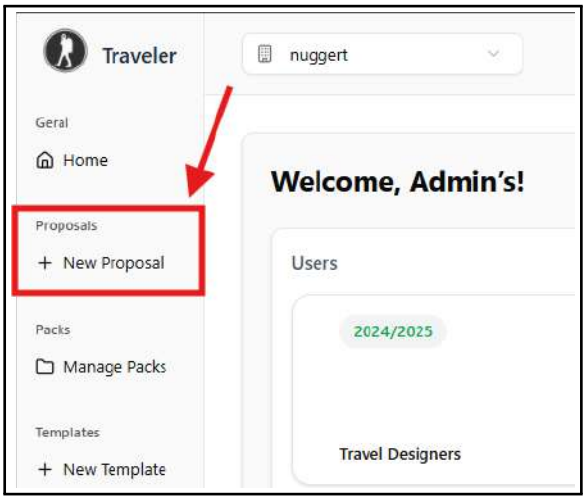

Figura 5.12: Botão que dá início ao processo de criação de uma proposta

Ao acionar esta funcionalidade, a aplicação executa a seguinte lógica:

- Envia um pedido ao backend para verificar se já existe alguma proposta com estado *In_Progress* associada ao grupo de trabalho do utilizador.
- Se existir, essa proposta é automaticamente atualizada para o estado *Archived*, preservando o seu conteúdo e histórico.
- De seguida, é criada uma nova proposta com estado *In_Progress*, a qual passa a representar a sessão de edição ativa.
- O `id` da nova proposta é então adicionado à rota da aplicação (URL), e o utilizador é redirecionado para a página de edição.

Este workflow garante a consistência da lógica de propostas, permitindo apenas uma proposta ativa por utilizador em simultâneo, ao mesmo tempo que mantém o registo de propostas anteriores que não foram finalizadas.

Estados da Proposta: A proposta pode assumir três estados distintos ao longo do seu ciclo de vida, conforme ilustrado na Figura 5.13:

- **In_Progress** – Proposta mais recente em edição.
- **Archived** – Propostas guardadas, mas ainda não finalizadas. Podem ser reabertas e continuar a ser editadas.
- **Submitted** – Propostas finalizadas e entregues ao cliente. Estado imutável.

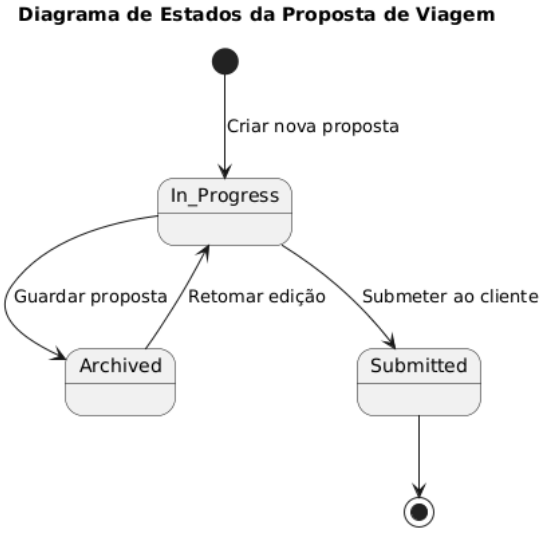

Figura 5.13: Diagrama de estados das propostas de viagem

Subdivisão Funcional: Dado que esta user story representa um épico, foi subdividida em várias sub-user stories.

- **US06.1 - Como Administrador/Travel Designer quero poder adicionar, editar e remover viajantes numa proposta**

  Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerirem os viajantes associados a uma proposta de viagem.

  Fluxo de Interação (Frontend): O utilizador acede à interface da proposta e interage com um formulário dinâmico onde pode introduzir os dados dos viajantes, como o nome, idade e documento de identificação. As operações de adição, edição e remoção de viajantes são geridas localmente através da store Zustand, sem qualquer comunicação imediata com o backend. A validação dos dados é realizada no frontend, garantindo que os campos obrigatórios estão corretamente preenchidos. Apenas quando o utilizador clica no botão "Próximo" é que os dados consolidados dos viajantes são enviados para o backend, utilizando uma requisição HTTP do tipo `POST` ou `PUT`, consoante se trate da criação ou atualização da proposta. (ver Figura A.3).

  Fluxo de Interação (Backend): O backend disponibiliza endpoints RESTful para suportar as operações de criação, edição, remoção e listagem de viajantes associados a uma proposta. Toda a lógica está centralizada no `TravelerController`, que recebe os pedidos provenientes do frontend e executa as ações correspondentes, com base no identificador único da proposta. Quando o pedido corresponde à criação de novos

viajantes, os dados previamente validados são inseridos na proposta. No caso de uma edição, o sistema atualiza os dados existentes com base no identificador do viajante. Para operações de remoção, os viajantes são eliminados da proposta após a devida confirmação. Por fim, a funcionalidade de listagem permite recuperar todos os viajantes associados a uma proposta específica, facilitando a visualização e eventual edição posterior.

Diagramas de Sequência: Dois diagramas de sequência representam a interação completa:

- **Frontend:** mostra como o utilizador interage com a UI e como o estado local é atualizado com Zustand (ver Figura A.1).
- **Backend:** representa a lógica da API no tratamento dos dados dos viajantes (ver Figura A.2).

Notas Técnicas:

- Utilização de Zustand para armazenar os viajantes temporariamente por `proposalId`.
- O backend disponibiliza endpoints `GET`, `POST`, `PUT` e `DELETE` para gerir viajantes.
- A comunicação com o backend apenas ocorre após confirmação o utilizador pressionar o botão "Próximo", de forma a não sobrecarregar com várias chamdas à API.

Interface e diagramas de sequência disponíveis no Anexo A.1.

- **US06.2 - Como Administrador/Travel Designer quero poder adicionar, editar e remover destinos numa proposta**

Descrição: Esta user story descreve a funcionalidade que permite a um utilizador com perfil de Administrador ou Travel Designer adicionar, editar e remover destinos de uma proposta de viagem. Cada destino representa uma etapa da viagem e está associado a um intervalo de datas (início e fim), sendo crucial para a definição das restantes componentes da proposta.

Fluxo de Interação (Frontend): O utilizador interage com a interface da proposta através de um formulário dinâmico que permite adicionar destinos, definindo o local, data de início e data de fim (ver Figura A.6). A gestão do estado é feita localmente usando Zustand, garantindo uma experiência fluida. Sempre que o utilizador adiciona, edita ou remove um destino, as alterações são refletidas no estado local da aplicação, mas não são imediatamente enviadas ao servidor. A validação dos dados, como a verificação das datas e do nome do destino, é realizada no frontend. Apenas quando o utilizador avança para a próxima etapa (clicando em "Próximo"), os dados são consolidados e enviados ao backend por requisições HTTP (`POST` ou `PUT`, conforme o caso).

Fluxo de Interação (Backend): O backend disponibiliza endpoints RESTful responsáveis por gerir os destinos das propostas. Estes endpoints são tratados pelo

`DestinationController`, que processa a criação, atualização, remoção e listagem dos destinos com base no `proposalId`. No cenário de criação, os dados dos novos destinos são validados e associados à proposta. Em caso de edição, as informações de destinos existentes são atualizadas conforme os identificadores fornecidos. A remoção elimina um destino específico da proposta, após validação. Por fim, o endpoint de listagem permite obter todos os destinos de uma proposta, o que facilita a visualização e edição no frontend.

Diagramas de Sequência:

- **Frontend:** mostra a interação entre o utilizador, o formulário de destinos e o serviço que gere o estado local (ver Figura A.4).
- **Backend:** representa o tratamento das requisições para adicionar, atualizar ou remover destinos na base de dados (ver Figura A.5).

Notas Técnicas:

- Utilizar Zustand para gerir o estado local dos destinos, agrupados por `proposalId`.
- Criar ou atualizar endpoints no backend para suportar as operações de `GET`, `POST`, `PUT` e `DELETE`.
- Validar localmente os campos obrigatórios, como o nome do destino e as datas.
- As alterações são persistidas apenas após a submissão via botão "Próximo"..

Interface e diagramas de sequência disponíveis no Anexo A.2.

- **US06.3 - Como Administrador/Travel Designer quero poder selecionar elementos de viagem extra numa proposta**

Descrição: Esta user story descreve a funcionalidade que permite a um utilizador com perfil de Administrador ou Travel Designer gerir os elementos de viagem extra associados a uma proposta. Estes elementos incluem: Transferes, Atividades, Seguros, Celebrações, Itinerários e Extras. A sua seleção permite personalizar ainda mais a proposta, definindo quais componentes adicionais deverão ser configuráveis nas fases seguintes.

Fluxo de Interação (Frontend): O utilizador acede à interface da proposta e é apresentado a uma lista de opções representadas por checkboxes correspondentes a cada tipo de elemento extra (ver Figura A.9). A seleção e desmarcação destas opções atualiza o estado local da aplicação, gerido com a biblioteca Zustand. A validação da escolha é feita no frontend. Os dados mantêm-se apenas no estado local até o utilizador clicar em "Próximo". Nesse momento, a seleção final é enviada ao backend através de uma requisição HTTP do tipo `PUT`, onde a estrutura do corpo da requisição reflete o estado booleano de cada elemento.

Fluxo de Interação (Backend): O backend disponibiliza endpoints RESTful para recuperar e atualizar os elementos extra de uma proposta, com base no `proposalId`. A lógica é gerida pelo `TripElementsController`, que processa o pedido de atualização

da configuração de elementos extra. Quando os dados são recebidos, são validados e armazenados na base de dados como parte integrante do modelo da proposta. O sistema também expõe um endpoint `GET` para permitir que a seleção anterior possa ser recuperada e apresentada ao utilizador em visitas futuras.

Diagramas de Sequência:

- **Frontend:** representa a interação entre o utilizador, a UI da seleção de elementos e a store Zustand (ver Figura A.7).
- **Backend:** mostra o processamento da atualização dos elementos extra e a sua persistência no modelo da proposta (ver Figura A.8).

Notas Técnicas:

- Utilizar Zustand para armazenar localmente a seleção dos elementos extra.
- Implementar endpoints RESTful no backend para `GET` e `PUT` com base no `proposalId`.
- A estrutura de dados enviada deve conter a seleção em formato booleano, por exemplo:

```
{
  elementsExtra: {
    transfer: true,
    atividades: false,
    seguros: true,
    celebracoes: false,
    itinerarios: true,
    extras: true
  }
}
```

- A persistência ocorre apenas após a validação e submissão ao clicar em "Próximo".

Interface e diagramas de sequência disponíveis no Anexo A.3.

- **US06.4 - Como Administrador/Travel Designer quero poder adicionar, editar e remover hotéis numa proposta**

Descrição: Esta user story descreve a funcionalidade que permite a um utilizador com perfil de Administrador ou Travel Designer gerir os hotéis associados a cada destino dentro de uma proposta. Esta funcionalidade contribui para a construção detalhada de cada pack de viagem, permitindo adaptar a experiência consoante o perfil e preferências do cliente.

Fluxo de Interação (Frontend): O utilizador inicia a configuração de hotéis após selecionar o pack pretendido (ver Figura A.12). O sistema apresenta um formulário que permite inserir dados como o nome do hotel, número de quartos, regime e datas de estadia. (ver Figura A.13). As operações de adicionar, editar ou remover são iniciadas no frontend e enviadas diretamente ao backend. Apenas após confirmação de sucesso

por parte da API, o estado local da aplicação (Zustand) é atualizado, garantindo a consistência entre cliente e servidor. O sistema permite editar hotéis existentes com base na interação com ícones de edição, e a remoção requer confirmação do utilizador. Ao clicar em "Próximo", é validado que existe pelo menos um hotel para cada destino antes de avançar.

Fluxo de Interação (Backend): O backend expõe endpoints RESTful para criação, edição, remoção e listagem de hotéis, implementados no `HotelController`. Cada operação é associada a uma proposta específica, destino e pack. O modelo de dados garante que um hotel está sempre relacionado com um `proposalId`, `destinationId` e um identificador de `pack`. As requisições do frontend são validadas e persistidas na base de dados. Em caso de sucesso, a resposta confirma a operação e permite sincronizar o estado no frontend.

Diagramas de Sequência:

- **Frontend:** representa o fluxo de interação entre o utilizador, a UI de gestão de hotéis e a lógica local via Zustand (ver Figura A.10).
- **Backend:** representa o processamento de cada operação (POST/PUT/DELETE/GET) de hotel associada a uma proposta (ver Figura A.11).

Notas Técnicas:

- O estado local (Zustand) só é atualizado após resposta positiva da API.
- Criar/atualizar endpoints RESTful no backend com suporte para `POST`, `PUT`, `DELETE` e `GET`.
- Garantir a associação correta entre hotel, destino, proposta e pack no modelo de dados.
- O payload enviado ao backend segue o seguinte formato:

```
{
  hotel:
    {
      name: "Hotel Exemplo",
      rooms: 2,
      regime: "All Inclusive",
      checkIn: "2025-07-01",
      checkOut: "2025-07-05",
      pack: "premium",
      destinationId: "xxxxxxxxxxxx"
    }
}
```

Interface e diagramas de sequência disponíveis no Anexo A.4.

- **US06.5 – Como Administrador/Travel Designer quero poder adicionar, editar e remover voos numa proposta**

  Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerir os voos associados a cada destino dentro de uma proposta.

  Fluxo de Interação (Frontend): O utilizador inicia a configuração de voos após selecionar o pack pretendido (ver Figura A.14). O sistema apresenta um formulário onde é possível adicionar informações como aeroporto de origem, aeroporto de destino, companhia aérea, datas de partida e chegada, classe e tipo de bagagem (ver Figura A.15). As operações de adição, edição e remoção de voos desencadeiam requisições HTTP para o backend. O estado local da aplicação, gerido com Zustand, é atualizado apenas após a confirmação de sucesso da operação por parte da API. A interface inclui suporte de autocompletar para os aeroportos, recorrendo a uma API pública de pesquisa.

  Fluxo de Interação (Backend): O backend expõe endpoints RESTful implementados no `FlightController`, responsáveis por tratar as operações de criação, atualização, remoção e listagem de voos. Cada voo é associado a uma proposta, destino e pack específicos, garantindo organização e estruturação no armazenamento dos dados. A API valida os dados recebidos, assegura a integridade relacional (com `destinationId`, `proposalId` e `pack`) e responde de forma apropriada ao frontend.

  Diagramas de Sequência:

  - **Frontend:** representa a sequência de ações entre o utilizador, o formulário e a lógica de estado local via Zustand (ver Figura A.16).
  - **Backend:** descreve o tratamento completo da requisição, desde o recebimento até à persistência dos dados na base de dados (ver Figura A.17).

  Notas Técnicas:

  - Zustand é utilizado para gerir a lista de voos localmente e garantir sincronização apenas após sucesso da operação backend.
  - Os endpoints implementam as operações `POST`, `PUT`, `DELETE` e `GET`.
  - A estrutura de dados enviada ao backend segue o seguinte formato:

```
{
  voo:
    {
      origin: "Aeroporto Francisco Sá Carneiro (OPO)",
      destination: "Dubai International Airport (DXB)",
      classe: "Economic",
      departureDate: "2025-07-01",
      arrivalDate: "2025-07-05",
      baggageOption: "dispatched",
      pack: "premium",
```

```
        destinationId: "xxxxxxxxxxxx"
      }
  }
```

- A funcionalidade de autocompletar os aeroportos é implementada através de uma API pública, que retorna os nomes e códigos conforme definido pela International Air Transport Association (IATA) com base na pesquisa do utilizador.

Interface e diagramas de sequência disponíveis no Anexo A.5.

- **US06.6 - Adicionar, editar e remover transferes por destino (caso selecionado na US06.3).**

Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerir os transferes associados a cada destino dentro de uma proposta. Esta funcionalidade apenas está disponível caso os transferes tenham sido selecionados previamente na fase de configuração inicial da proposta.

Fluxo de Interação (Frontend): O utilizador acede à página de gestão de transferes após a seleção do pack pretendido (ver Figura A.18). O sistema apresenta um formulário dinâmico onde é possível adicionar, editar ou remover transferes associados a um destino. Os dados preenchidos incluem o nome do serviço de transfere, data e hora, e, opcionalmente, notas adicionais (ver Figura A.19).

Todas as ações são tratadas localmente, com recurso à biblioteca Zustand, que mantém o estado atualizado na aplicação cliente. O envio dos dados para o backend apenas ocorre após o utilizador carregar no botão "Próximo". Nesse momento, todos os transferes consolidados são enviados numa única requisição batch para persistência no servidor (ver Figura A.20).

Fluxo de Interação (Backend): O backend expõe os endpoints necessários para gerir transferes. Ao receber o pedido batch com todos os transferes, a API remove previamente todos os transferes associados à proposta e insere os novos de forma agrupada. Esta estratégia garante a sincronização total entre o frontend e a base de dados. Toda a lógica de persistência encontra-se encapsulada no `TransferController`, que comunica com o `TransferModel` para executar as operações de escrita na base de dados (ver Figura A.21).

Diagramas de Sequência:

- **Frontend:** representa a sequência de ações entre o utilizador, os componentes da interface, o hook lógico e a store Zustand, até ao envio da requisição batch (ver Figura A.20).
- **Backend:** descreve a receção do pedido PUT em batch, a remoção dos transferes antigos e inserção dos novos na base de dados (ver Figura A.21).

Notas Técnicas:

- Zustand é utilizado para gerir o estado local dos transferes até ao envio final.
- A API implementa os métodos REST `GET` e `PUT` (em batch) via o endpoint `/transfers`.
- A estrutura de dados enviada no pedido batch segue o seguinte formato:

```
{
  transfers: [
    {
      name: "Transfers do Aeroporto",
      dateTime: "2025-07-01T00:00:00Z",
      notes: "Transfer noturno",
      pack: "premium",
      destinationId: "xxxxxxxxxxxx"
    },
    ...
  ]
}
```

- A remoção e inserção em batch no backend garante integridade e evita duplicação ou conflitos.

Interface e diagramas de sequência disponíveis no Anexo A.6.

- **US06.7 - Adicionar, editar e remover atividades por destino (caso selecionado na US06.3).**

Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerir as atividades associadas a cada destino dentro de uma proposta. Esta funcionalidade apenas está disponível caso as atividades tenham sido selecionadas previamente na fase de configuração inicial da proposta.

Fluxo de Interação (Frontend): O utilizador acede à página de gestão de atividades após a seleção do pack pretendido (ver Figura A.22). O sistema apresenta um formulário dinâmico onde é possível adicionar, editar ou remover atividades associadas a um destino. Os dados preenchidos incluem o nome da atividade, data, hora e notas opcionais (ver Figura A.23 e Figura A.24).

Todas as ações são tratadas localmente, com recurso à biblioteca Zustand, que mantém o estado atualizado na aplicação cliente. O envio dos dados para o backend apenas ocorre após o utilizador carregar no botão "Próximo". Nesse momento, todas as atividades consolidadas são enviadas numa única requisição batch para persistência no servidor (ver Figura A.25).

Fluxo de Interação (Backend): O backend expõe os endpoints necessários para gerir atividades. Ao receber o pedido batch com todas as atividades, a API remove

previamente todas as atividades associadas à proposta e insere as novas de forma agrupada. Esta estratégia garante a sincronização total entre o frontend e a base de dados. Toda a lógica de persistência encontra-se encapsulada no `ActivityController`, que comunica com o `ActivityModel` para executar as operações de escrita na base de dados (ver Figura A.26).

Diagramas de Sequência:

- **Frontend:** representa a sequência de ações entre o utilizador, os componentes da interface, o hook lógico e a store Zustand, até ao envio da requisição batch (ver Figura A.25).
- **Backend:** descreve a receção do pedido PUT em batch, a remoção das atividades antigas e inserção das novas na base de dados (ver Figura A.26).

Notas Técnicas:

- Zustand é utilizado para gerir o estado local das atividades até ao envio final.
- A API implementa os métodos REST `GET` e `PUT` (em batch) via o endpoint `/activities`.
- A estrutura de dados enviada no pedido batch segue o seguinte formato:

```
{
  activities: [
    {
      name: "Passeio de barco",
      duration: "2025-07-03T10:00:00Z",
      description: "Inclui almoço",
      price: "20",
      transferService: "true",
      pack: "premium",
      destinationId: "xxxxxxxxxxxx"
    },
    ...
  ]
}
```

- A remoção e inserção em batch no backend garante integridade e evita duplicação ou conflitos.

Interface e diagramas de sequência disponíveis no Anexo A.7.

- **US06.8 - Adicionar, editar e remover seguros por destino (caso selecionados).**

Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerir os seguros associados a cada destino dentro de uma proposta. Esta funcionalidade apenas está disponível caso

os seguros tenham sido selecionados previamente na fase de configuração inicial da proposta.

Fluxo de Interação (Frontend): O utilizador acede à página de gestão de seguros após a seleção do pack pretendido (ver Figura A.27). O sistema apresenta um formulário dinâmico onde é possível adicionar, editar ou remover seguros associados a um destino. Os campos incluem o tipo de seguro, a cobertura, e o preço de cada seguro. Para facilitar a configuração, são apresentados seguros pré-definidos que o utilizador pode selecionar diretamente, com a possibilidade de personalizar os detalhes, se necessário (ver Figura A.28).

Todas as ações são tratadas localmente através de Zustand. A submissão para o backend só ocorre após o utilizador carregar no botão "Próximo", altura em que todos os seguros são enviados numa única requisição batch (ver Figura A.29).

Fluxo de Interação (Backend): O backend expõe os endpoints necessários para gestão de seguros. Ao receber o pedido PUT batch, todos os seguros anteriores são removidos e os novos são inseridos. Esta abordagem evita inconsistências e garante uma sincronização total entre o cliente e o servidor (ver Figura A.30).

Diagramas de Sequência:

- **Frontend:** representa a interação com o formulário, Zustand e envio da requisição batch (ver Figura A.29).
- **Backend:** mostra o tratamento completo da requisição PUT /insurances/batch (ver Figura A.30).

Notas Técnicas:

- A API implementa os métodos `GET` e `PUT` no endpoint `/insurances`.
- O estado local é mantido com Zustand até à submissão.
- Exemplo de estrutura de envio:

```
{
  insurances: [
    {
      type: "Seguro de Viagem",
      description: "Cobertura total da viagem",
      price: "100",
      pack: "standard",
      destinationId: "xxxxxxxxxxxx"
    }
  ]
}
```

Interface e diagramas de sequência disponíveis no Anexo A.8.

- **US06.9 - Adicionar, editar e remover celebrações por destino (caso selecionadas).**

  Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerir celebrações (como aniversários, luas de mel, ocasiões especiais) associadas a um destino de uma proposta. Esta funcionalidade só é apresentada se as celebrações forem previamente selecionadas na configuração inicial.

  Fluxo de Interação (Frontend): Após selecionar o pack, o utilizador tem acesso à secção de celebrações (ver Figura A.31). A interface permite adicionar eventos com título, data e descrição (ver Figura A.33). Todas as alterações são geridas localmente com Zustand, sendo o envio realizado apenas ao clicar em "Próximo", numa única requisição batch (ver Figura A.32).

  Fluxo de Interação (Backend): O backend remove todas as celebrações existentes associadas à proposta antes de inserir as novas. Esta operação é tratada pelo `CelebrationController`, garantindo consistência na base de dados. O endpoint responsável é `PUT /celebrations/batch`.

  Diagramas de Sequência:

  - **Frontend:** detalha a interação do utilizador com os componentes de celebrações e a store Zustand (ver Figura A.34).
  - **Backend:** mostra o processo completo de sincronização via requisição batch (ver Figura A.35).

  Notas Técnicas:

  - A gestão do estado é feita com Zustand até ao envio batch.
  - Endpoints implementados: `GET /celebrations`, `PUT /celebrations/batch`.
  - Estrutura do payload:

```
{
  celebrations: [
    {
      title: "Aniversário de Casamento",
      date: "2025-07-06",
      description: "Jantar romântico à beira-mar",
      pack: "premium",
      destinationId: "xxxxxxxxxxxx"
    }
  ]
}
```

Interface e diagramas de sequência disponíveis no Anexo A.9.

- **US06.11 - Criar itinerários com recurso à inteligência artificial.**

  Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer gerar automaticamente um itinerário personalizado para cada destino de uma proposta, com o apoio de um modelo de Large Language Model (LLM). Esta funcionalidade encontra-se disponível apenas se os itinerários tiverem sido selecionados durante a fase de configuração inicial da proposta.

  Fluxo de Interação (Frontend): O utilizador acede à página de itinerários após a seleção do pack pretendido (ver Figura A.36). Caso pretenda gerar um novo itinerário, pode utilizar o botão "Gerar com Inteligência Artificial (IA)", o que ativa um formulário de confirmação (ver Figura A.37).

  Após confirmação, o frontend invoca diretamente o endpoint do backend responsável pela criação do itinerário com IA (`POST /itineraries/ai-generate`), enviando apenas os dados essenciais: `proposalId`, `destinationId`, `pack`, e outras informações de apoio (atividades, hotel, voo). O frontend não constrói nem envia o prompt, limitando-se a delegar essa responsabilidade ao servidor.

  A resposta do backend, contendo o itinerário gerado, é usada para atualizar o estado local com recurso à biblioteca Zustand, permitindo ao utilizador visualizar, editar ou remover o itinerário (ver Figura A.38).

  Fluxo de Interação (Backend): O backend expõe um endpoint específico para criação de itinerários com IA: `POST /itineraries/ai-generate`. Ao receber o pedido, o `ItineraryController` recolhe os dados relevantes e constrói um prompt estruturado.

  Este prompt é enviado para a API da Hugging Face[3], utilizando o modelo LLaMA[4]. A IA devolve uma resposta em formato JavaScript Object Notation (JSON), contendo o plano detalhado do itinerário, organizado por dias.

```
{
  itinerary: {
    title: string,
    totalEstimatedCost: number,
    days: [
      {
        title: string,
        description: string,
        estimatedCost: number,
        activities: [
          {
            time: string,
            description: string,
            location: string,
            costEstimate: number
          }
        ]
      }
```

[3] https://huggingface.co/
[4] https://www.llama.com/

```
    ]
  }
}
```

O backend valida e persiste este conteúdo na base de dados através do `ItineraryModel`, e devolve o itinerário completo ao frontend para exibição.

Diagramas de Sequência:

- **Frontend:** representa a sequência desde o clique no botão "Gerar com IA" até à sincronização do estado local com o itinerário devolvido pelo backend (ver Figura A.39).
- **Backend:** ilustra a construção do prompt, envio à Hugging Face com o modelo LLaMA, e persistência do itinerário (ver Figura A.40).

Notas Técnicas:

- Zustand é utilizado para armazenar e atualizar localmente os itinerários após sucesso da operação.
- A API utiliza um endpoint exclusivo: `/itineraries/ai-generate`.
- O prompt é construído no backend com os dados: nome do destino, nome do hotel, data de chegada (voo), atividades selecionadas, e tipo de pack.
- É utilizado o modelo LLaMA alojado na API da Hugging Face.
- A resposta da IA segue o formato JSON, contendo um itinerário estruturado por dias e atividades.
- A validação e persistência são realizadas no backend antes do envio da resposta ao frontend.
- Os itinerários gerados podem ser editados ou removidos diretamente pelo utilizador.
- As credenciais de acesso à API da Hugging Face devem estar armazenadas no ficheiro de variáveis de ambiente, de forma a garantir a segurança e confidencialidade das mesmas.

Interface e diagramas de sequência disponíveis no Anexo A.10.

5.1.5 US07 – Enviar proposta por email com recurso a SMTP

Descrição: Esta user story descreve a funcionalidade que permite aos utilizadores com perfil de Administrador ou Travel Designer submeter uma proposta final e envia-la ao cliente por email. Esta operação representa a última fase do processo de configuração da proposta.

Fluxo de Interação (Frontend): O utilizador acede à página de revisão da proposta, onde tem acesso ao botão "Enviar Proposta" (ver Figura **??**). Após pressionar o botão, é apresentado um formulário onde o utilizador insere o email do cliente, bem como a data de início e de fim da validade da proposta.

Após preenchimento do formulário, o sistema realiza as validações locais, nomeadamente se o email tem um formato válido e se as datas introduzidas são válidas. Caso as validações sejam bem-sucedidas, é efetuada uma chamada à API através do endpoint `POST /send-proposal-email`. Se o envio for realizado com sucesso, o sistema procede à atualização da proposta com as datas indicadas e o estado `submitted`, através de um pedido `PUT /proposals/:id`. O utilizador recebe então uma notificação visual de sucesso e é automaticamente redirecionado para a **dashboard**. Em caso de erro no envio ou na atualização, é apresentada uma mensagem de erro na interface e o processo é interrompido.

Fluxo de Interação (Backend): O backend tem o endpoint `POST /send-proposal-email`, tratado pelo `EmailController`. Quando recebe o pedido, o controlador valida os campos obrigatórios: destinatário, nome da proposta, nome do grupo e datas de validade.

De seguida, o sistema constrói um link personalizado que aponta para o Frontoffice, permitindo ao cliente aceder à proposta. É então gerado o corpo do email em HTML, contendo as informações da proposta e um botão com o link de acesso. O envio do email é feito utilizando o `nodemailer`[5], configurado com transporte SMTP (via Google[6]).

Se o envio for bem-sucedido, é devolvido um código `200 OK`. Após esta confirmação, o frontend invoca o endpoint `PUT /proposals/:id` para atualizar o estado da proposta para `submitted` e guardar as datas de validade. Em caso de falha no envio, é retornado um erro `500 Internal Server Error` e o estado da proposta não é alterado.

Diagramas de Sequência:

- **Frontend:** representa a interação entre o formulário de envio, validações, chamadas à API e redirecionamento final (ver Figura 5.14).
- **Backend:** descreve o processamento do envio de email e posterior atualização da proposta (ver Figura 5.15).

Notas Técnicas:

- Utilização da biblioteca `nodemailer` com transporte SMTP configurado para Gmail.
- As credenciais de autenticação SMTP estão protegidas em variáveis de ambiente.
- O corpo do email é enviado em HTML e inclui: nome da proposta, grupo responsável, intervalo de validade e link para a proposta.
- O envio do email é responsabilidade exclusiva do `EmailController`.
- A proposta apenas é marcada como submetida (`submitted`) após confirmação de envio com sucesso.

[5] https://nodemailer.com/
[6] https://developers.google.com/workspace/gmail/imap/imap-smtp?hl=pt-br

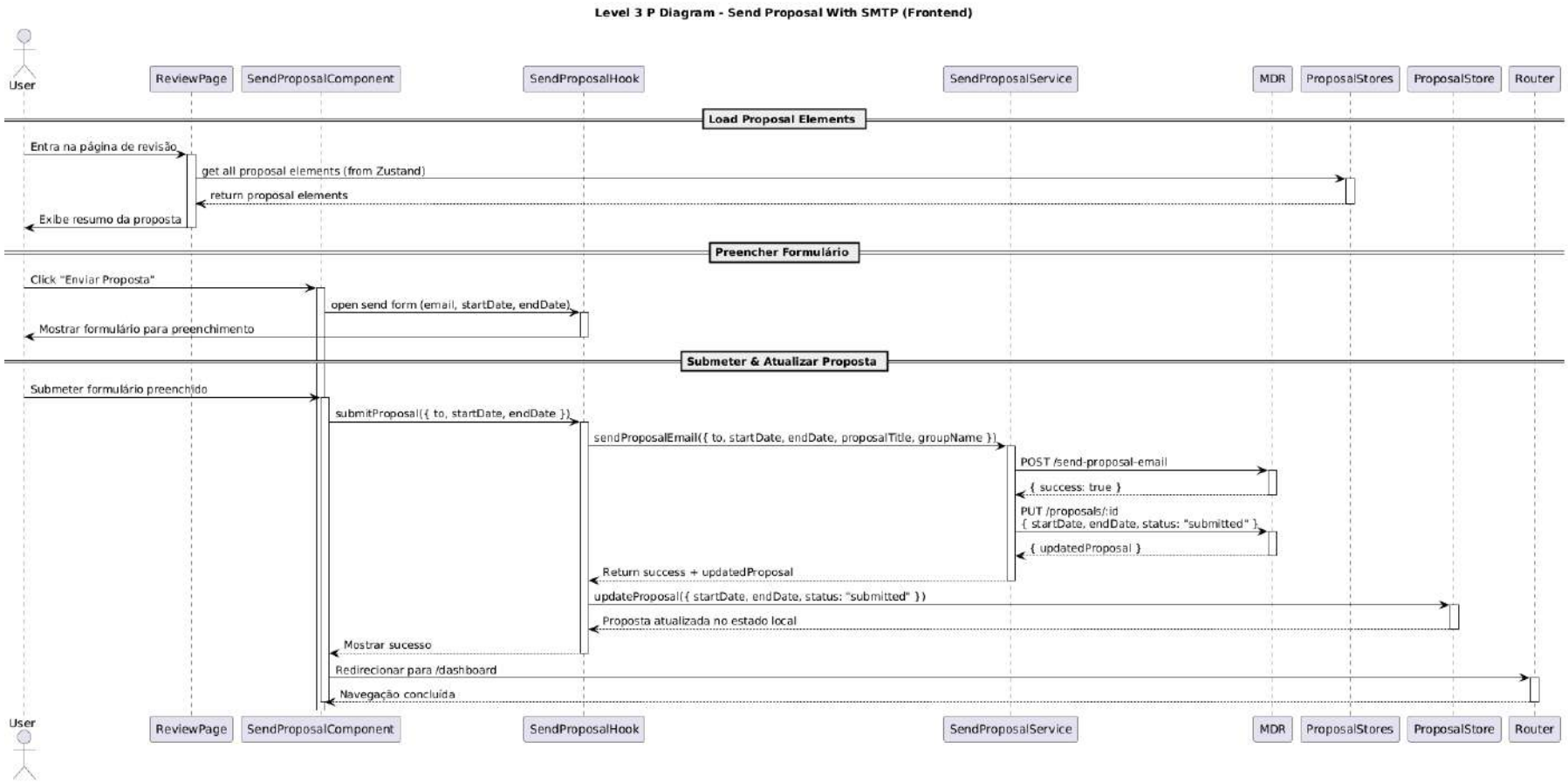

Figura 5.14: US7 - Diagrama de sequência (Frontend)

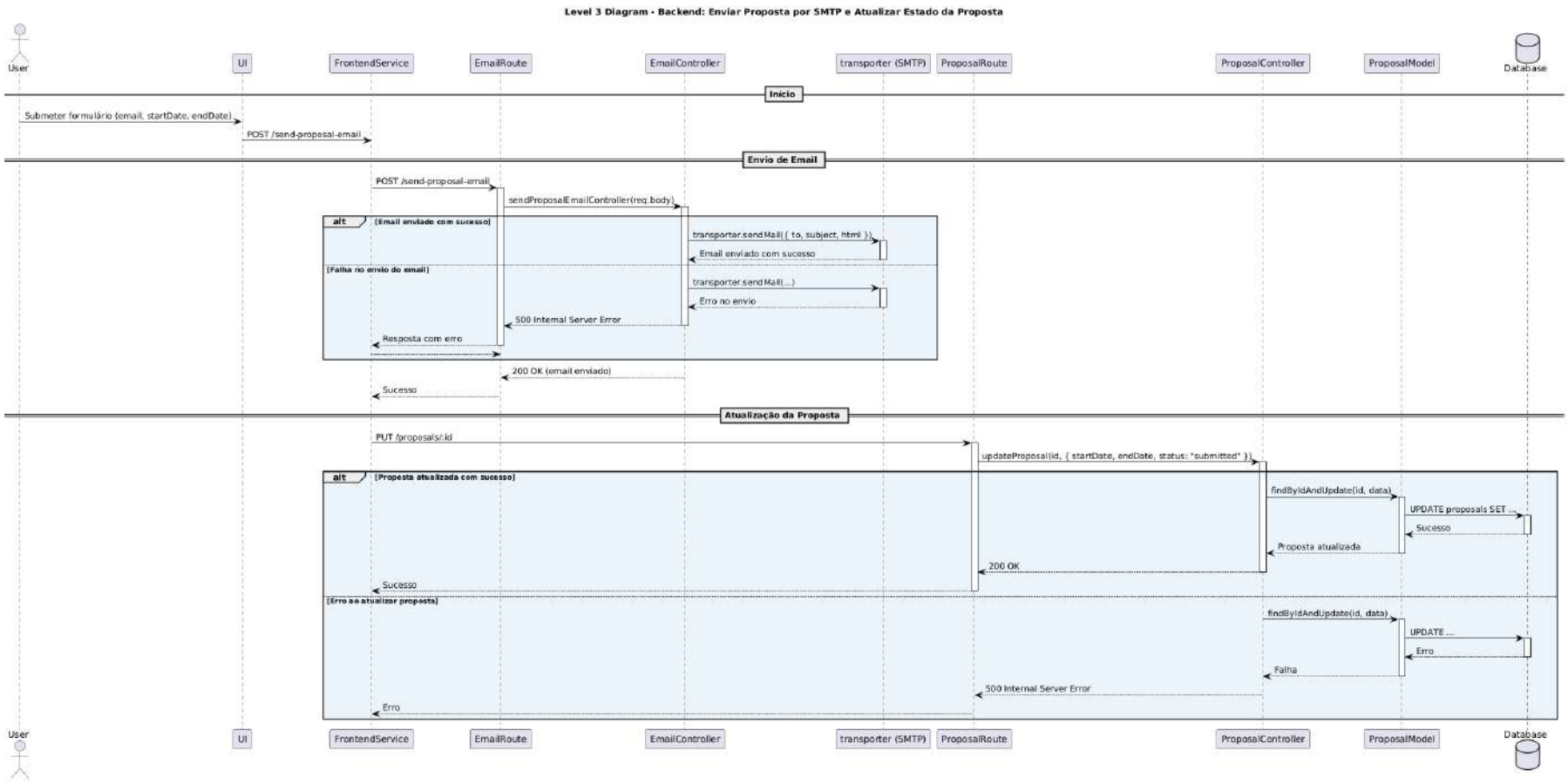

Figura 5.15: US7 - Diagrama de sequência (Backend)

5.2 Deploy da Aplicação

Nesta secção, é descrito o processo de *deploy* da aplicação, detalhando as ferramentas, os serviços e as configurações utilizadas para disponibilizar o sistema num ambiente acessível aos utilizadores finais.

O ambiente de produção foi dividido entre dois serviços principais: um servidor `Ubuntu Server`, alojado na infraestrutura da Amazon Web Services (AWS), destinado à execução do backend, e a plataforma `Vercel`[7], responsável pela publicação do frontend. Para assegurar a execução contínua do backend, foi utilizado o gestor de processos `PM2`[8], permitindo manter a aplicação ativa e monitorizada de forma eficiente. O encaminhamento das requisições HTTP e a distribuição dos conteúdos foi garantido através do `Nginx`[9], configurado como *reverse proxy*. A persistência de dados foi assegurada com recurso ao serviço de base de dados `MongoDB`.

O processo de disponibilização do backend consistiu na compilação do projeto com o compilador `tsc`, seguida da instalação das dependências necessárias via `npm install`, culminando na inicialização da aplicação com o comando `pm2 start`. Esta sequência permitiu garantir que o servidor estivesse operacional e estável no ambiente de produção.

No caso do frontend, optou-se por utilizar a plataforma Vercel, que simplifica o processo de *deploy* ao integrar-se diretamente com o repositório Git do projeto. A Vercel executa automaticamente o comando `npm run build` sempre que é feito um *push* para o repositório, disponibilizando de imediato a versão atualizada da aplicação num domínio personalizado.

Por fim, para tornar a aplicação acessível de forma segura e com um domínio próprio, foi realizada a configuração dos registos `Domain Name System (DNS)` no gestor de domínios, complementada pela emissão de certificados Secure Sockets Layer (SSL) através da ferramenta `Certbot`[10], com base no serviço `Let's Encrypt`[11]. O servidor `Nginx` foi então configurado para redirecionar automaticamente o tráfego HTTP para HTTPS, assegurando uma ligação encriptada entre os utilizadores e a aplicação.

5.2.1 Links da Aplicação

Abaixo encontram-se os links de acesso às instâncias públicas da aplicação desenvolvida:

- **Frontend**: `https://traveler-vercel.vercel.app/login`
- **Backend**: `https://traveler.itspossible.tech/api-docs/`

5.3 Testes

Para garantir o requisito não funcional de confiabilidade, a aplicação foi submetida a diversos testes unitários, de integração e de ponta-a-ponta (*end-to-end*), recorrendo a várias ferramentas apropriadas para cada camada do sistema:

- **Postman:** Foi criada uma *collection* com os vários pedidos efetuados ao servidor backend, incluindo casos de sucesso e de erro. Estes testes permitiram validar a

[7] `https://vercel.com/`
[8] `https://pm2.keymetrics.io/`
[9] `https://nginx.org/`
[10] `https://certbot.eff.org/`
[11] `https://letsencrypt.org/`

resposta da API, nomeadamente os códigos de estado HTTP, mensagens de erro, e comportamento esperado perante dados inválidos.

- **Jest (Backend):** A framework `Jest` foi utilizada para testes unitários e de integração nos controladores e utilitários do backend. Estes testes asseguraram que as funções do servidor funcionavam corretamente de forma isolada e em conjunto com a base de dados.
- **Cypress (End-to-End):** A ferramenta `Cypress` foi utilizada para realizar testes de ponta-a-ponta, simulando o comportamento do utilizador na aplicação completa (frontend + backend). Este tipo de teste garantiu que os fluxos de interação funcionam corretamente na prática.
- **Jest + React Testing Library (Frontend):** A combinação de `Jest` com `React Testing Library` foi usada para testar os componentes React do frontend. Estes testes focaram-se na renderização, validação de formulários, interações com o utilizador e navegação entre páginas.

5.4 Avaliação da Solução

Para efeitos de validação da solução desenvolvida, foram definidos, desde o início, um conjunto de critérios de aceitação que visam assegurar o cumprimento dos principais requisitos expectáveis numa aplicação web moderna.

Os critérios estabelecidos foram os seguintes:

- **Desempenho:** Capacidade de resposta da aplicação face às interações do utilizador, bem como o tempo de carregamento das diferentes páginas.
- **Usabilidade:** Facilidade de utilização da interface, garantindo que os utilizadores tenham uma experiência positiva e fluída.
- **Escalabilidade:** Capacidade da aplicação de suportar múltiplos utilizadores e pedidos simultâneos sem comprometer o desempenho.
- **Responsividade:** Capacidade da interface de se adaptar corretamente a diferentes tamanhos de ecrã e dispositivos.
- **Manutenibilidade:** Facilidade com que a aplicação pode ser evoluída, corrigida ou adaptada ao longo do tempo.

Durante o desenvolvimento, realizaram-se sessões regulares de testes e demonstrações com o orientador de estágio, com o objetivo de perceber o grau de conformidade da solução face aos critérios definidos. Adicionalmente, foi adotada a metodologia de trabalho *Gitflow*, permitindo a realização de submissões de código organizadas e sujeitas a validação por parte do supervisor. Este processo favoreceu uma revisão contínua da aplicação por parte da equipa da Its Possible Tech, permitindo a deteção atempada de falhas e a introdução de melhorias com base no feedback recolhido no final de cada sessão, e de cada Pull Request.

Em suma, a avaliação da solução foi feita de forma iterativa, através de revisões contínuas, validação dos critérios definidos e ajustamentos baseados no feedback obtido. Esta abordagem permitiu garantir que a aplicação cumpria os objetivos propostos e estava apta a ser utilizada num ambiente real.

Capítulo 6

Conclusões

Este capítulo apresenta uma reflexão final sobre o trabalho desenvolvido. São identificadas os objetivos realizados, reconhecidas algumas limitações e perspetivadas futuras melhorias, terminando com uma apreciação pessoal sobre a relevância e impacto da solução construída.

6.1 Objetivos Concretizados

Os objetivos delineados no início do projeto foram plenamente concretizados, culminando numa solução funcional e alinhada com as necessidades da *It's Possible Tech*, bem como com as melhores práticas de desenvolvimento de software e segurança da informação.

A aplicação resultante — denominada Traveler — foi concebida com base nos princípios da *Clean Architecture*, promovendo a separação de responsabilidades e facilitando a manutenção e escalabilidade futura do sistema.

O backend foi desenvolvido seguindo práticas robustas de organização de código, modularidade e testes, enquanto o frontend, baseado em React e Next.js, respeitou os princípios de reutilização de componentes, gestão eficiente de estado e acessibilidade.

A Tabela 6.1 resume o grau de concretização de cada um dos objetivos definidos no início do projeto:

Tabela 6.1: Concretização dos objetivos do projeto

Objetivo Específico	**Estado**
Conceber a arquitetura da solução e respetiva documentação técnica.	Concretizado
Desenvolver o backend da aplicação com base em boas práticas de engenharia de software.	Concretizado
Desenvolver o frontend da aplicação com foco na experiência do utilizador e na modularidade dos componentes.	Concretizado
Realizar a integração entre componentes e efetuar testes funcionais e de integração.	Concretizado
Validar a aplicação junto dos utilizadores e efetuar o respetivo *deploy* para ambiente de produção.	Concretizado

6.2 Limitações e trabalho futuro

Apesar de a aplicação desenvolvida ter cumprido os objetivos definidos no início do projeto, existem limitações relevantes que importa salientar e que poderão ser abordadas em iterações futuras.

Uma limitação foi a ausência de um sistema de reservas em tempo real. A aplicação não foi integrada com um sistema de distribuição global (GDS), o que impossibilita uma ligação direta com prestadores de serviços como companhias aéreas e hotéis. Futuramente, seria interessante fazer a conexão com um destes sistemas, de forma a automatizar ainda mais a criação das propostas.

Outra limitação identificada é a ausência de suporte multilingue e de funcionalidade multi-moeda, o que reduz a acessibilidade da aplicação a utilizadores internacionais. Neste sentido, uma possível melhoria seria implementar um sistema de internacionalização (i18n), que permitisse recorrer a um dicionário para alternar entre idiomas. Complementarmente, a introdução de suporte para diferentes moedas permitiria também ajustar automaticamente os preços apresentados à preferência ou a localização do utilizador.

Outra melhoria diz respeito ao sistema de apoio ao utilizador. Devido à limitação temporal do projeto, a dashboard destinada às equipas de suporte apresenta apenas funcionalidades básicas. Uma evolução significativa seria a implementação de um sistema de mensagens interno, que possibilitasse uma comunicação direta entre os utilizadores e a equipa de suporte, facilitando o esclarecimento de dúvidas e a resolução de problemas de forma eficiente.

Em suma, as limitações identificadas decorrem sobretudo do tempo disponível para o desenvolvimento do projeto.

6.3 Apreciação Final

Considero que o projeto desenvolvido cumpriu os objetivos que foram definidos antes do início do trabalho, permitindo aplicar de forma concreta os conhecimentos adquiridos ao longo da licenciatura. Para além da aplicação de boas práticas de programação, o sistema foi concebido com especial atenção à segurança e à proteção dos dados dos utilizadores, um requisito cada vez mais relevante no desenvolvimento de software moderno.

Este trabalho foi também uma oportunidade para aumentar os meus conhecimentos técnicos. Durante a licenciatura, tive apenas um contacto breve com Angular, no entanto, este projeto deu-me a oportunidade de trabalhar com tecnologias modernas como React e Next.js, aprofundando o meu conhecimento com estas ferramentas. O desafio consistiu não apenas em aprender novas tecnologias, mas em aplica-las num contexto real, com exigências concretas e prazos definidos, algo que considero essencial para o meu crescimento profissional.

Bibliografia

[1] I. Travefy, *Itinerary Management For Travel Professionals | Travefy*, 2025. URL: https://travefy.com/.

[2] L. Ltd, *Software for Tour Operators, Travel Agencies and DMCs | Lemax*. URL: https://www.lemax.net/.

[3] V. Pc, *Online Travel Agency Software - Back Office Solutions | TravelWorks*. URL: https://travelworkssolution.com/.

[4] OVHcloud. «Bases de dados relacionais vs não relacionais.» Acedido em junho de 2025. (2025), URL: https://www.ovhcloud.com/pt/learn/relational-vs-non-relational-databases/.

[5] O. Hutsulyak, *NodeJS vs .NET*, jan. de 2025. URL: https://www.techmagic.co/blog/node-js-vs-net-what-to-choose.

[6] C. Hitesh, *Angular vs Next.js: A Detailed Comparison - DEV Community*, dez. de 2024. URL: https://dev.to/hitesh_developer/angular-vs-nextjs-a-detailed-comparison-34ki.

Apêndice A

Anexos

A.1 Diagrama de Sequência & Interface – Viajantes

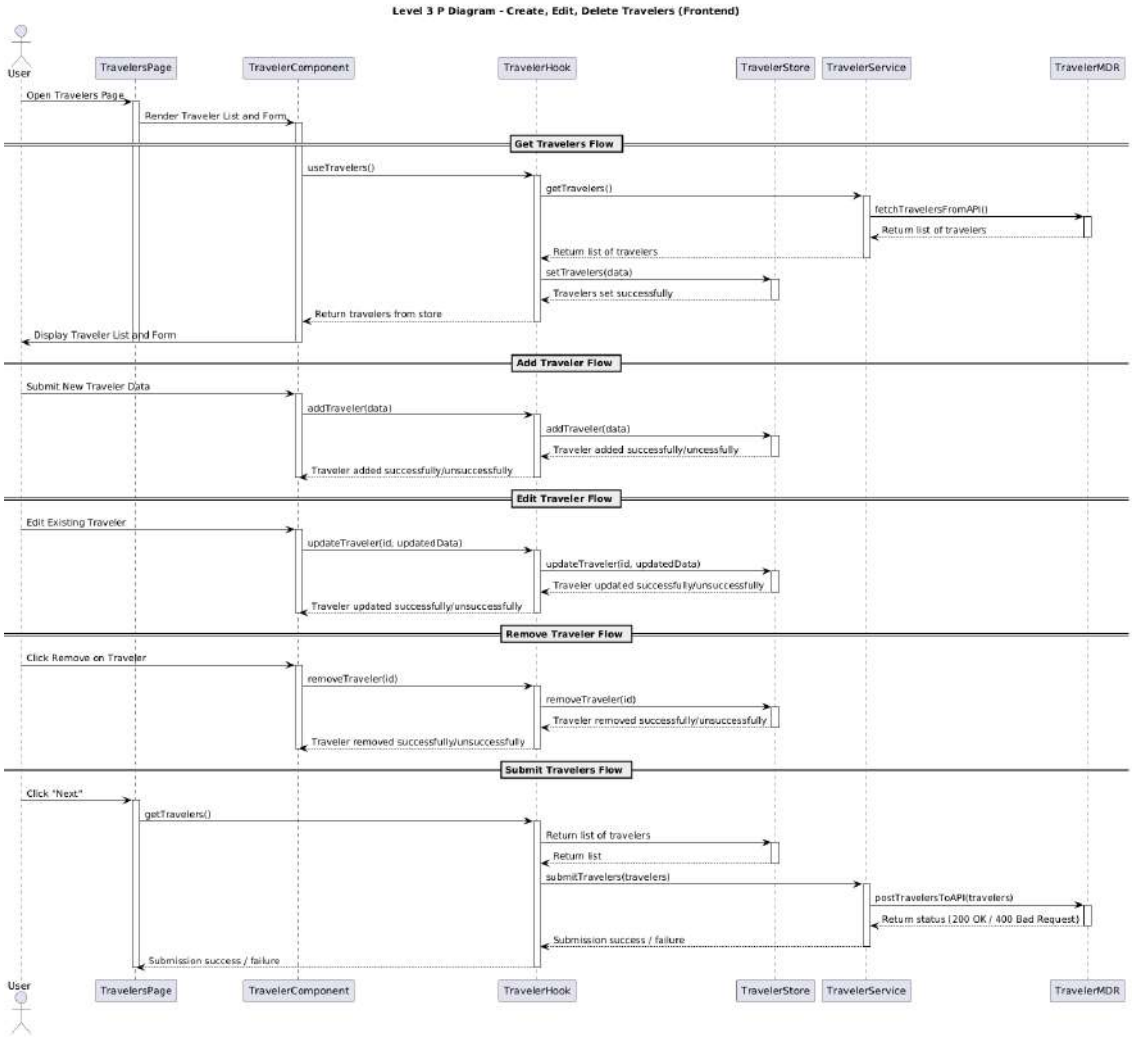

Figura A.1: Diagrama de sequência (frontend) – Gestão de viajantes

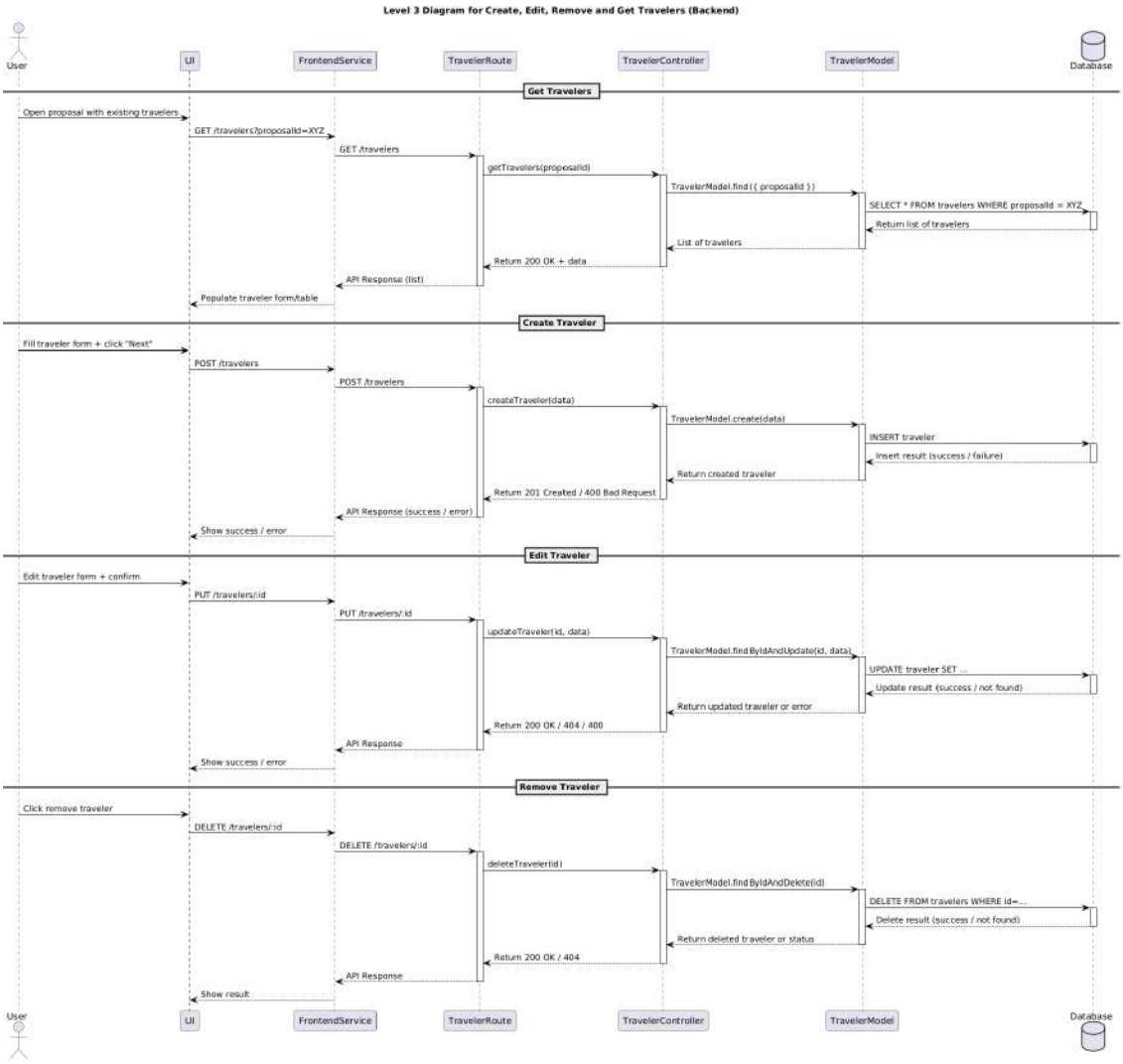

Figura A.2: Diagrama de sequência (backend) – Gestão de viajantes

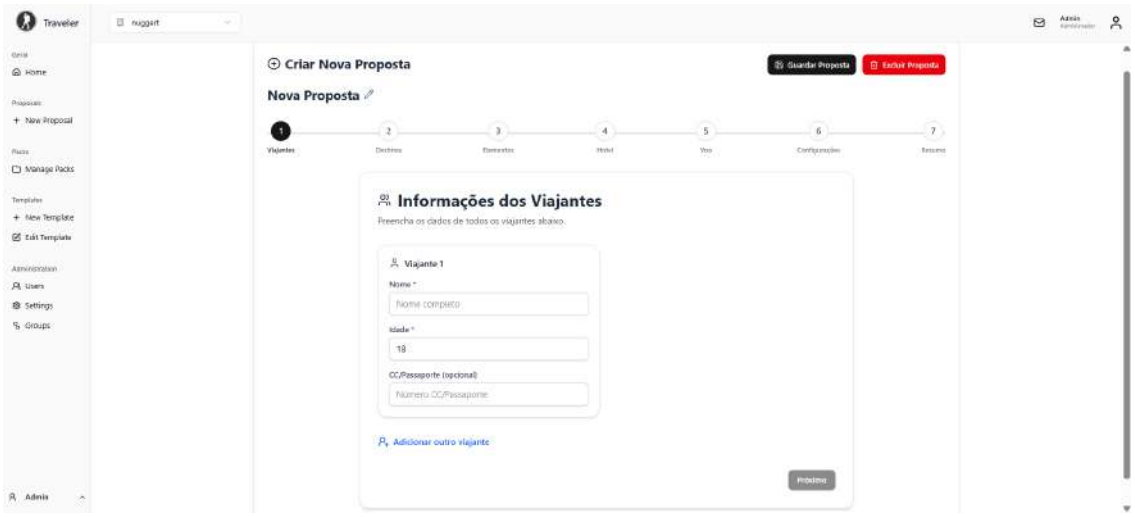

Figura A.3: Interface da página de viajantes

A.2 Diagrama de Sequência & Interface – Destinos

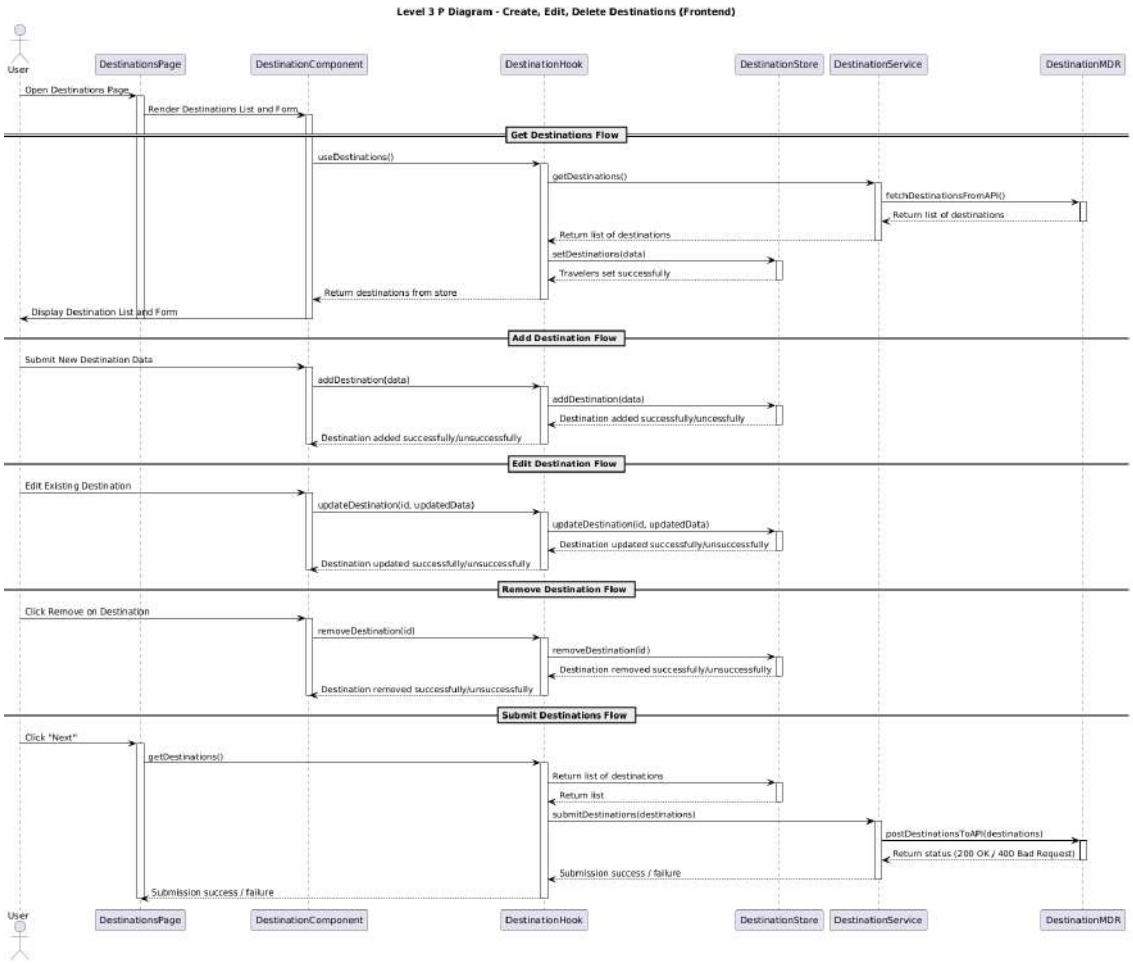

Figura A.4: Diagrama de sequência (frontend) – Gestão de destinos

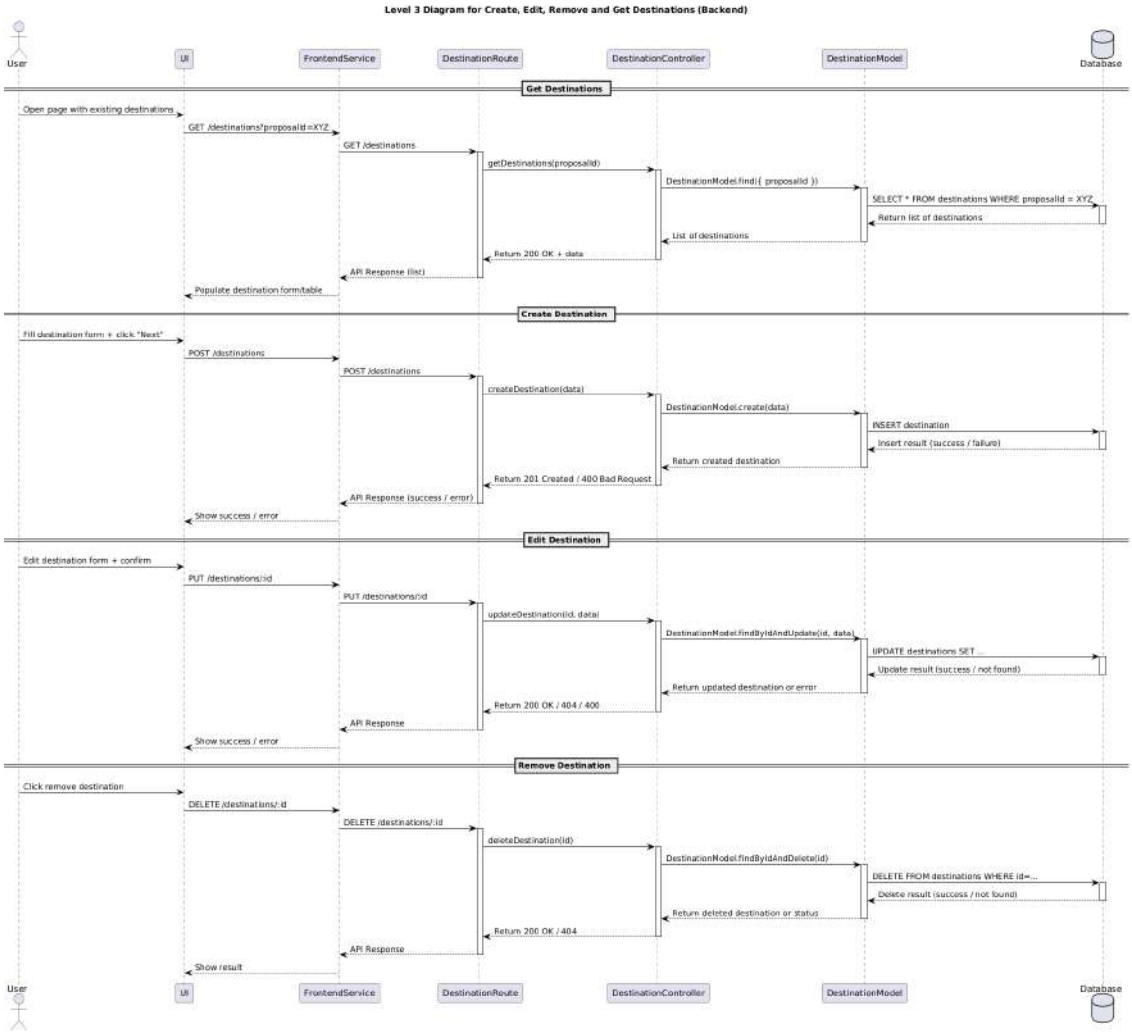

Figura A.5: Diagrama de sequência (backend) – Gestão de destinos

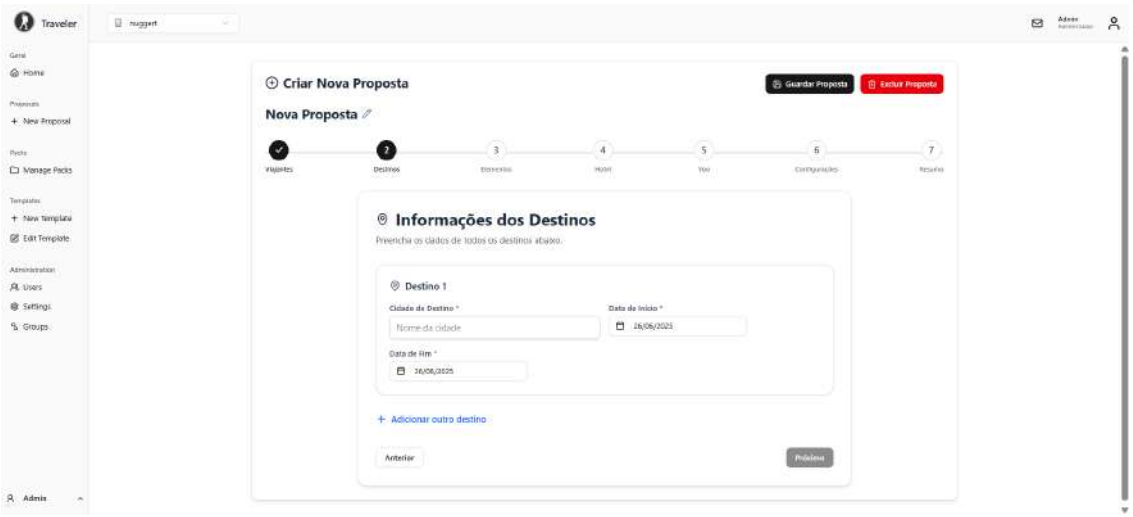

Figura A.6: Interface da página de destinos

A.3 Diagrama de Sequência & Interface – Elementos Extra

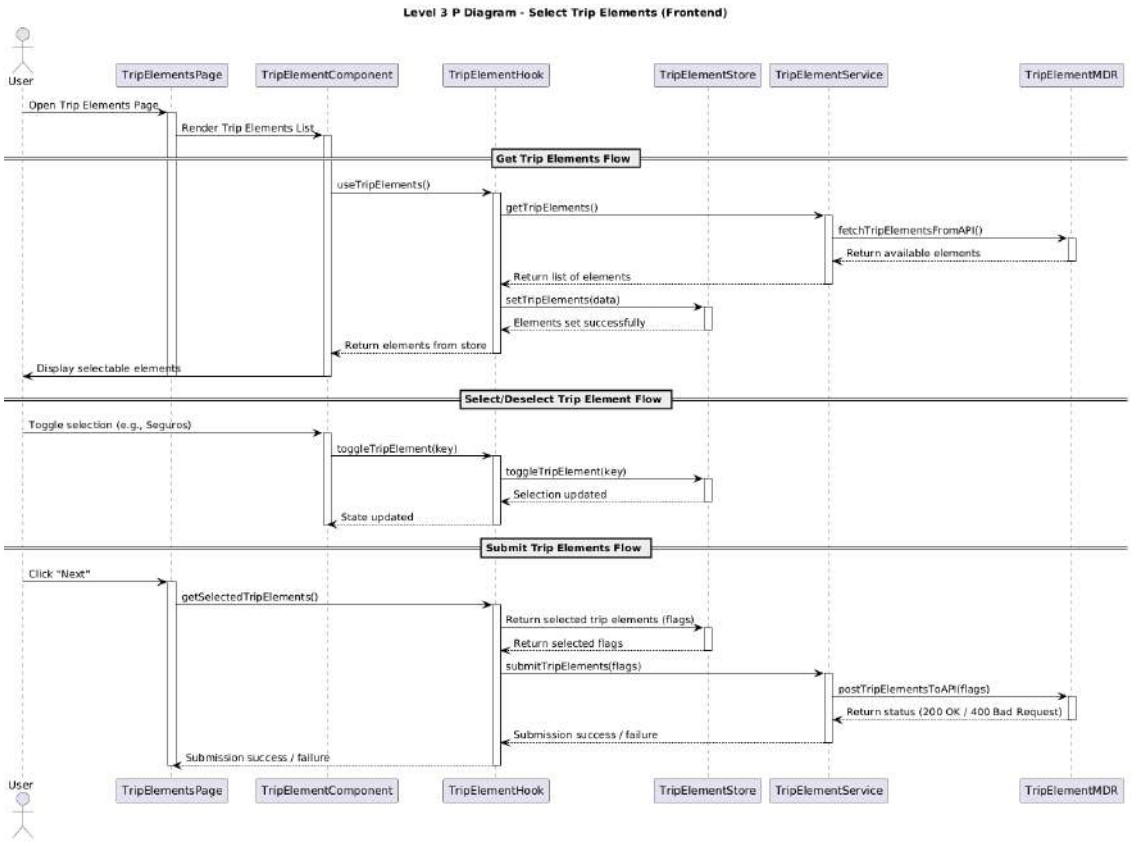

Figura A.7: Diagrama de sequência (frontend) – Gestão de elementos extra

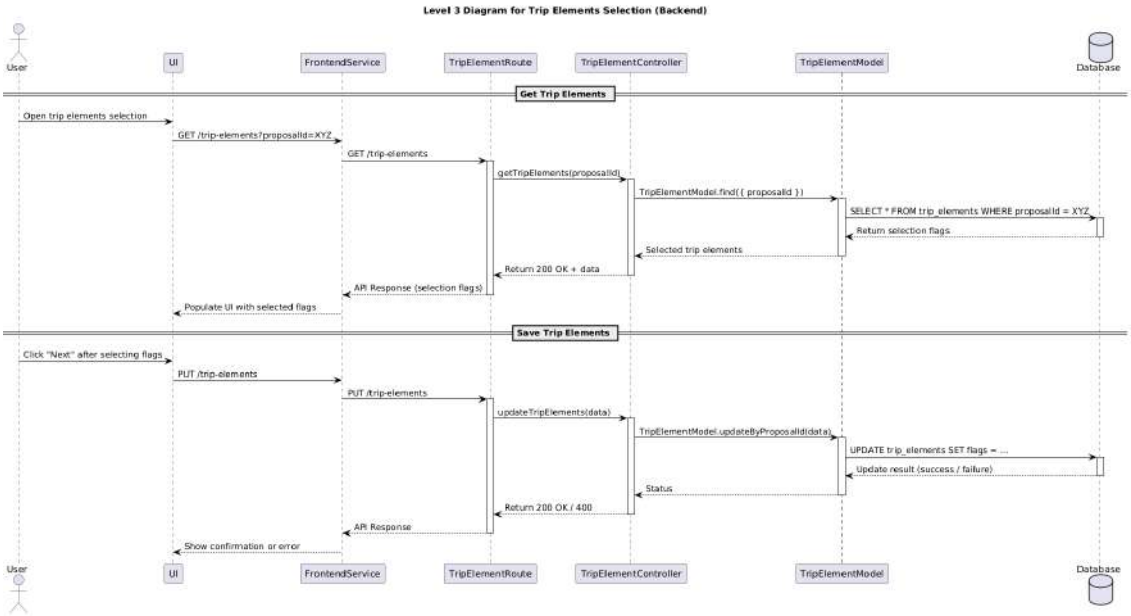

Figura A.8: Diagrama de sequência (backend) – Gestão de elementos extra

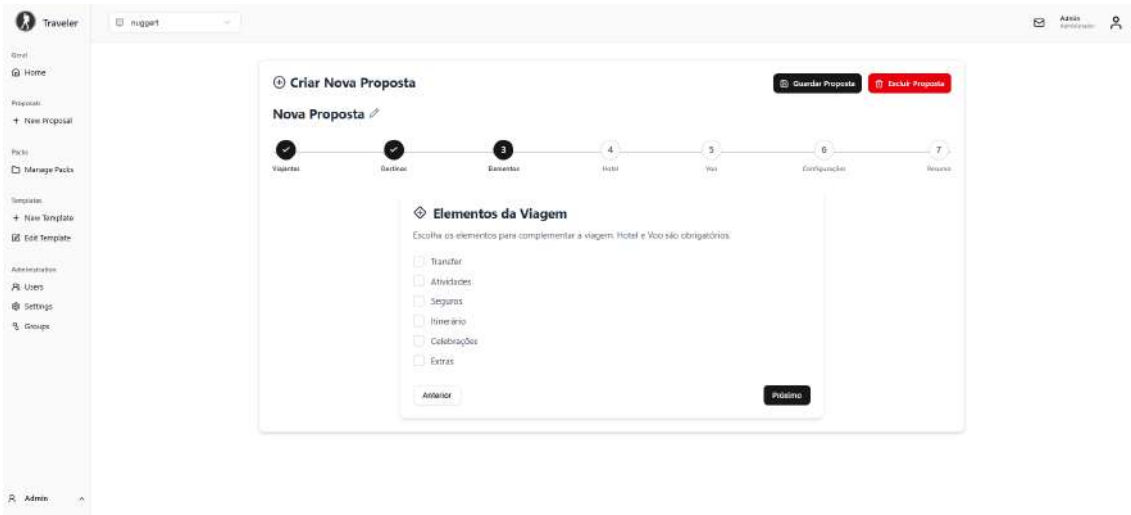

Figura A.9: Interface da página de elementos extra

A.4 Diagrama de Sequência & Interface – Hotéis

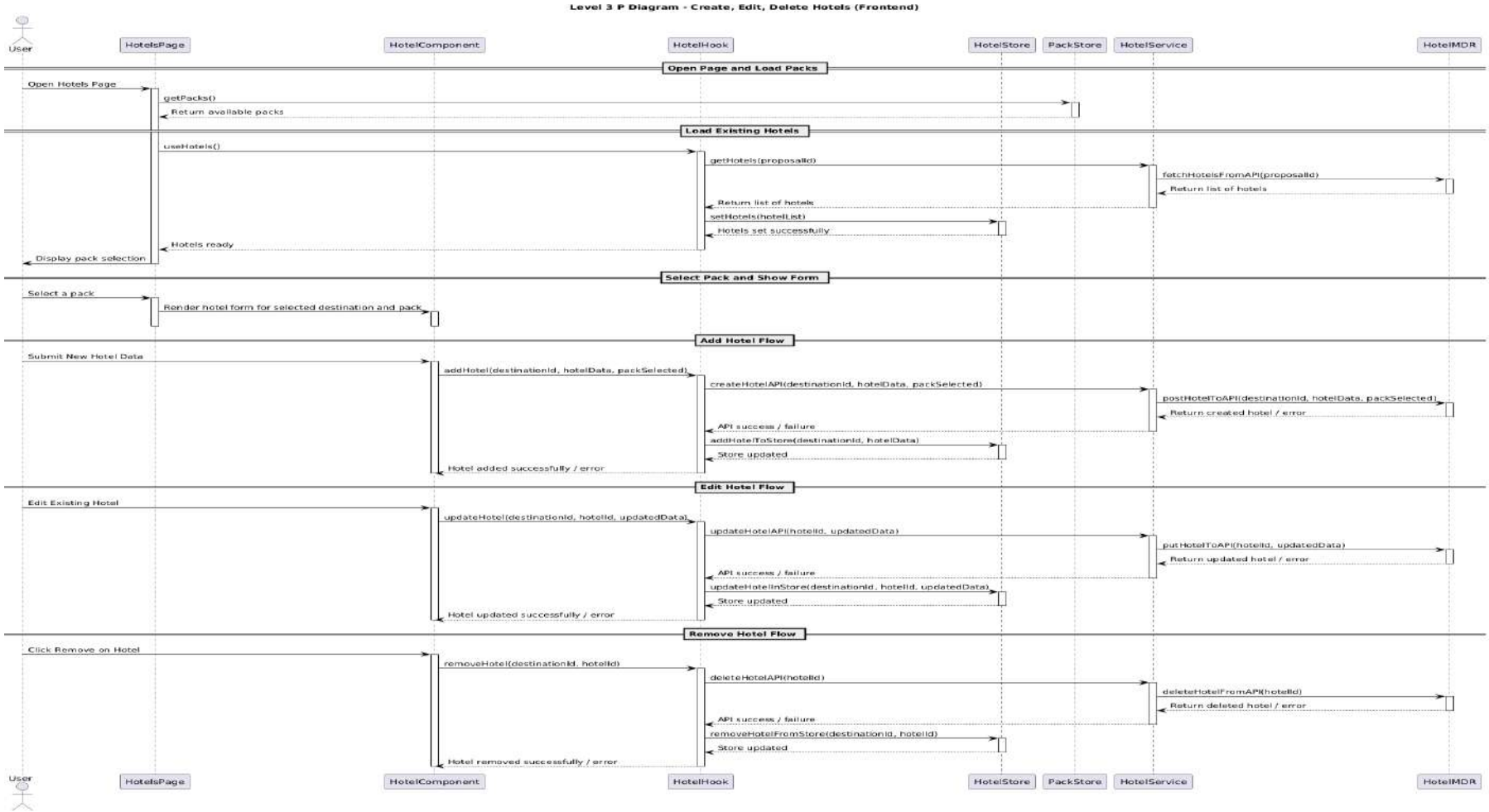

Figura A.10: US06.4 - Diagrama de sequência (frontend)

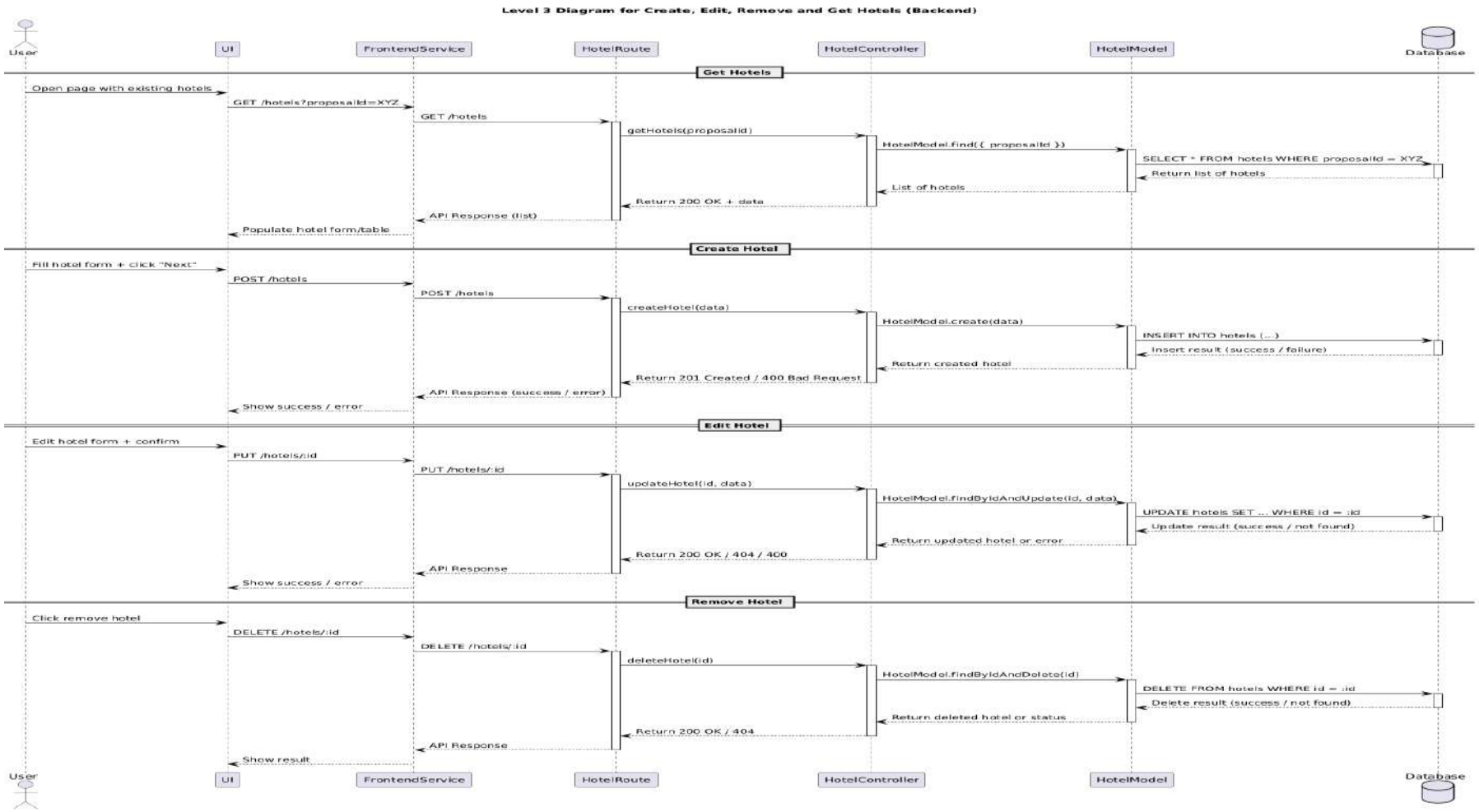

Figura A.11: US06.4 - Diagrama de sequência (backend)

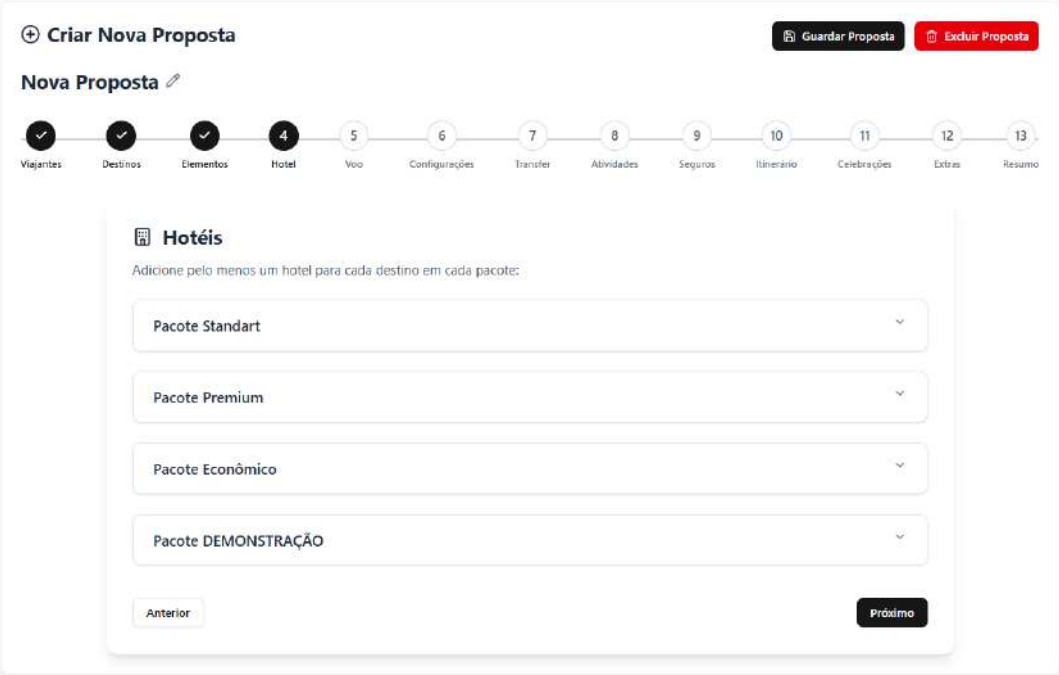

Figura A.12: Interface da página de Hotéis: Seleção de um pack

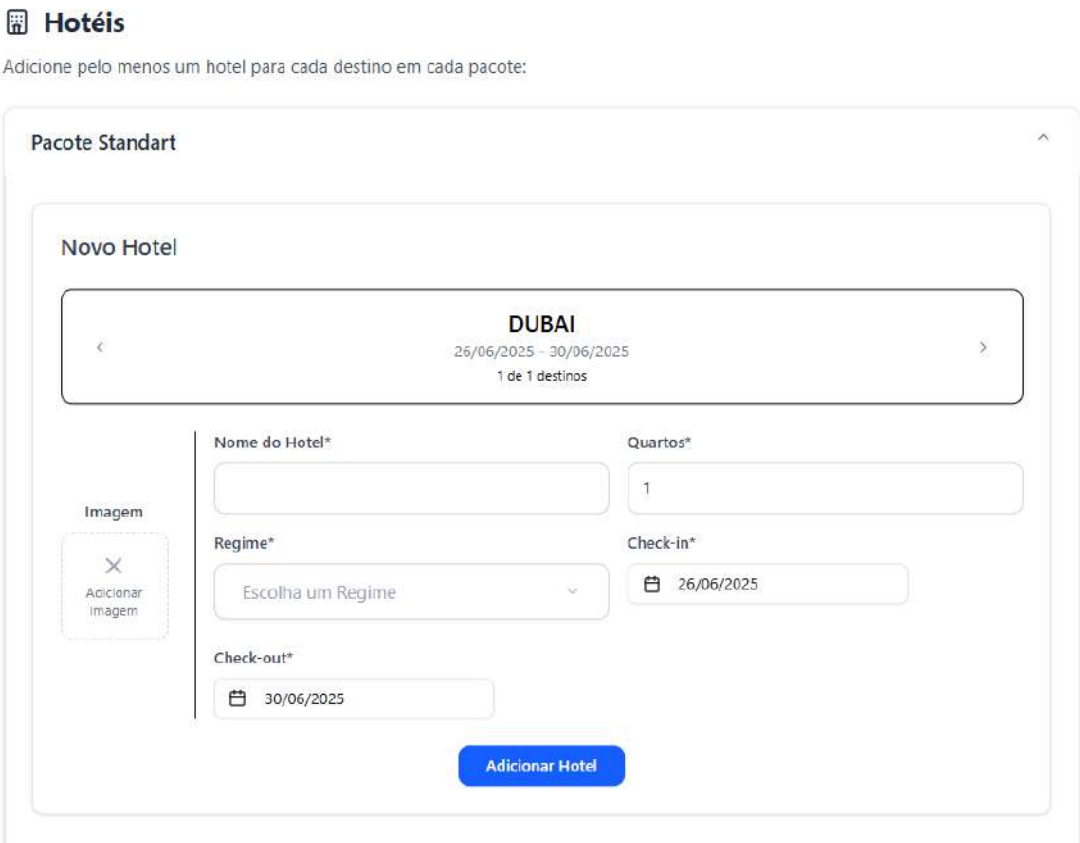

Figura A.13: Formulário para adicionar hotel

A.5 Diagrama de Sequência & Interface – Voos

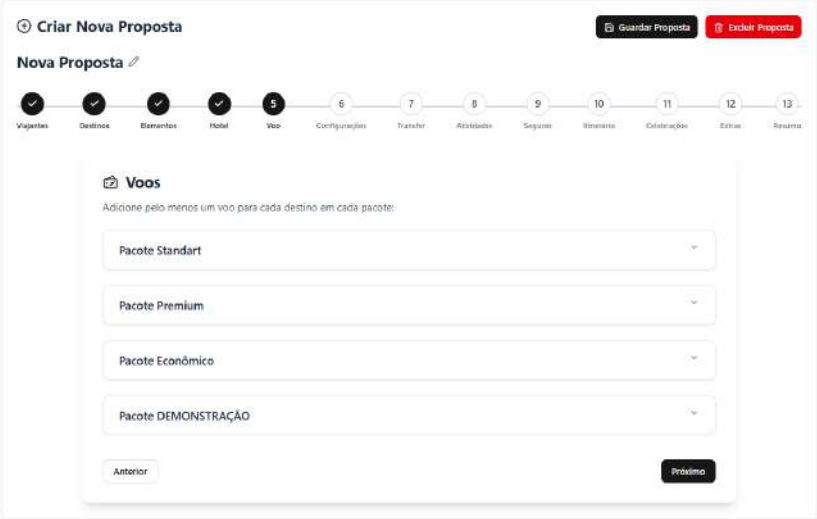

Figura A.14: Interface da página de Hotéis: Seleção de um pack

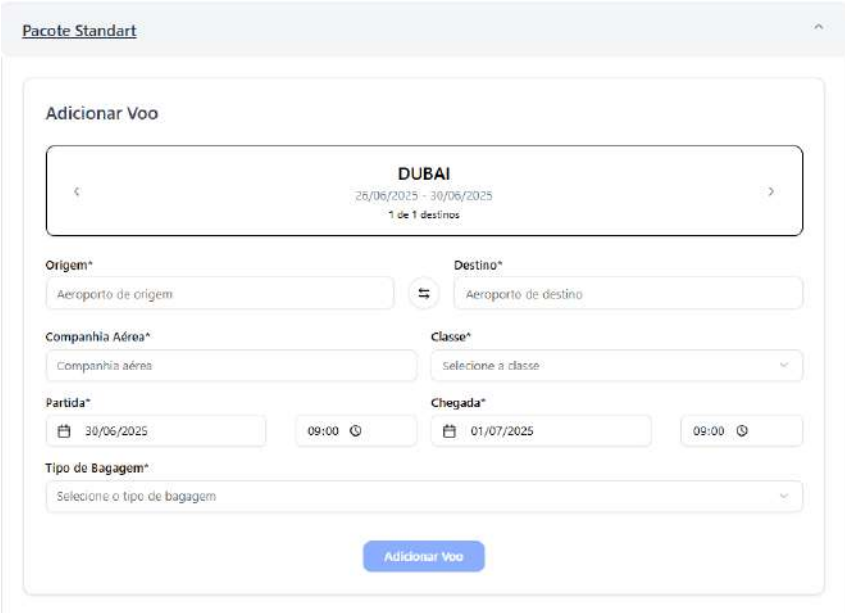

Figura A.15: Formulário para adicionar hotel

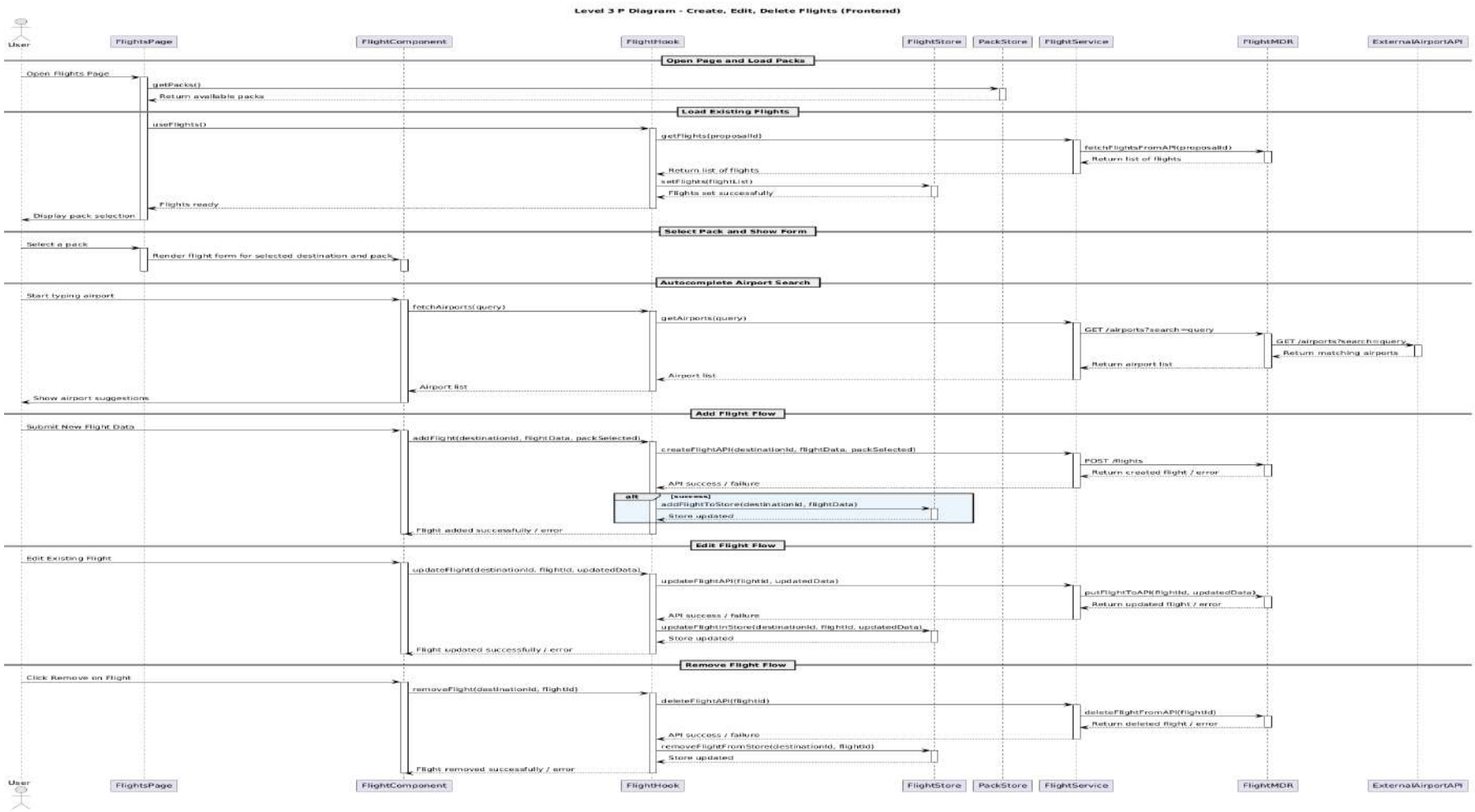

Figura A.16: US06.5 - Diagrama de sequência (frontend)

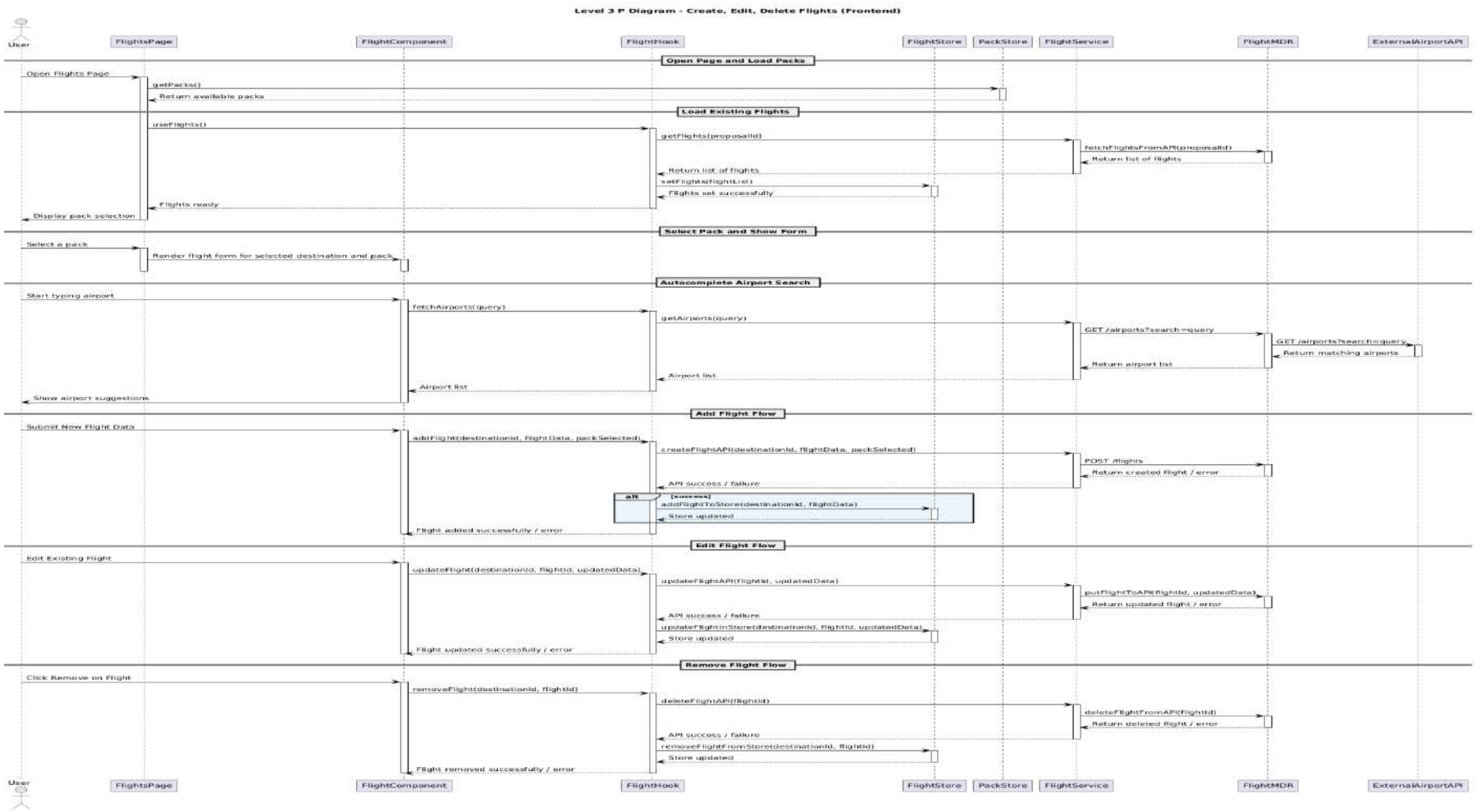

Figura A.17: US06.5 - Diagrama de sequência (backend)

A.6 Diagrama de Sequência & Interface – Transferes

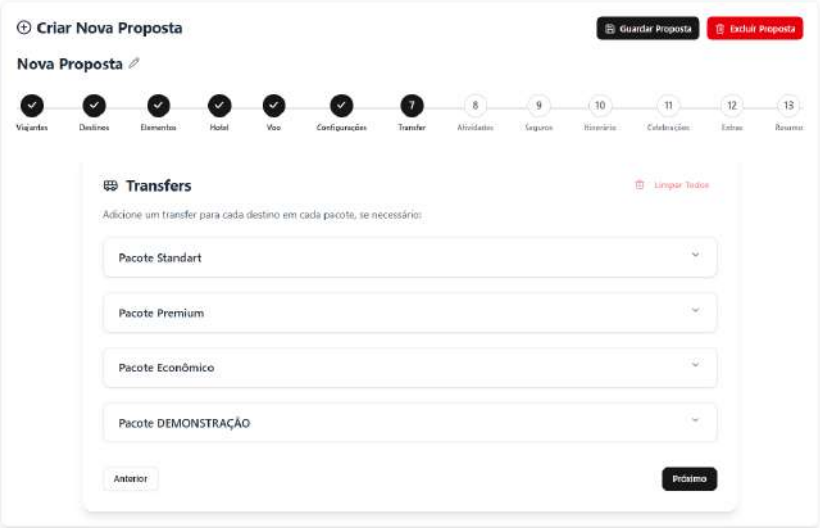

Figura A.18: Interface da página de Transferes: Seleção de um pack

Figura A.19: Formulário para adicionar transfere

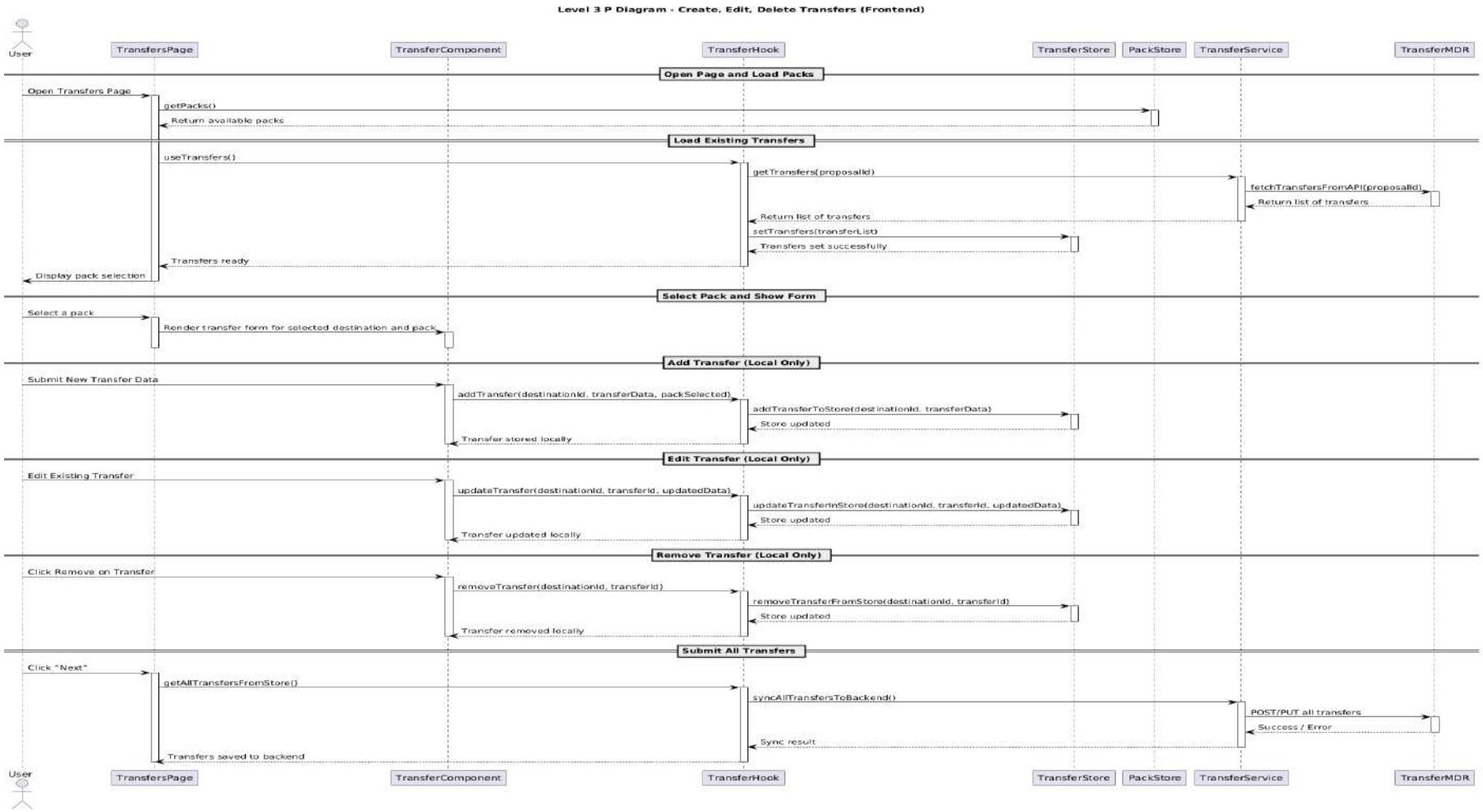

Figura A.20: US06.6 - Diagrama de sequência (frontend)

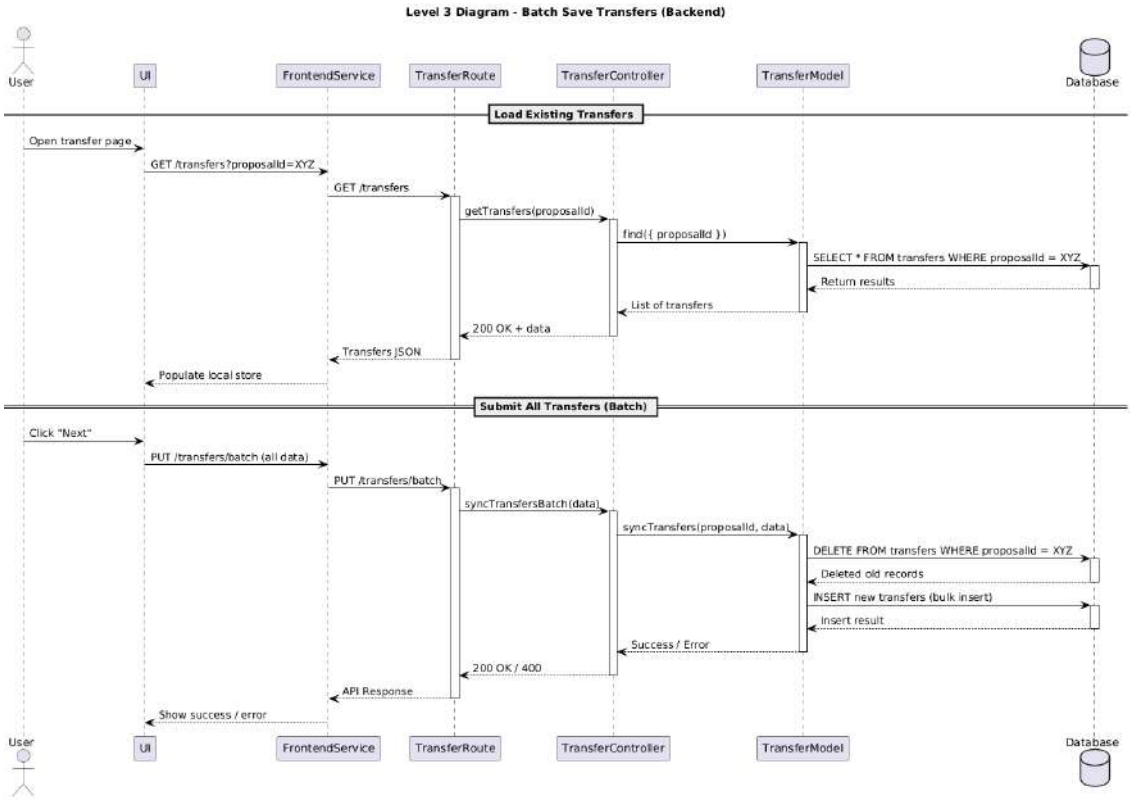

Figura A.21: US06.6 - Diagrama de sequência (backend)

A.7 Diagrama de Sequência & Interface – Atividades

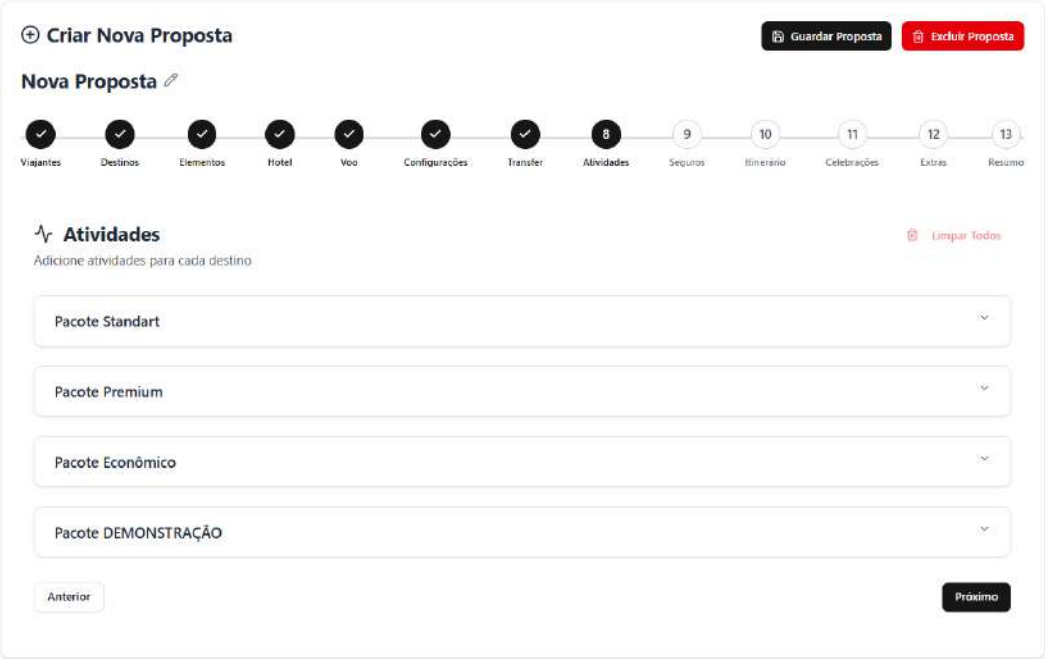

Figura A.22: Interface da página de Atividades: Seleção de um pack

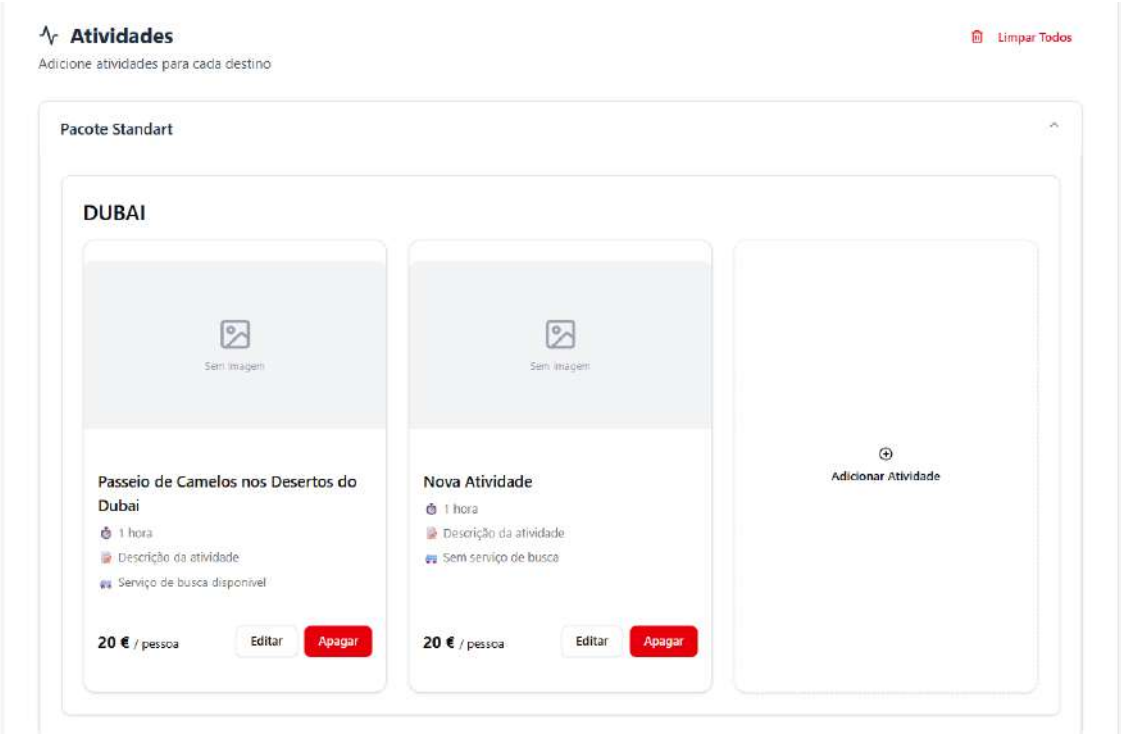

Figura A.23: Lista das Atividades

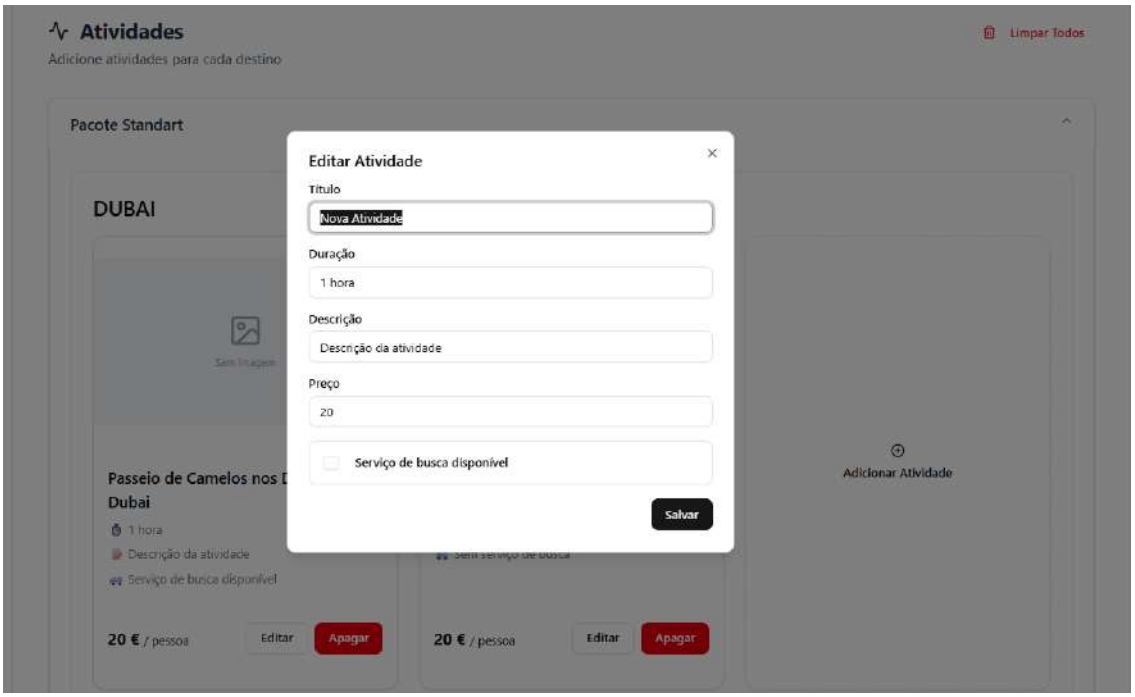

Figura A.24: Formulário de Atividade

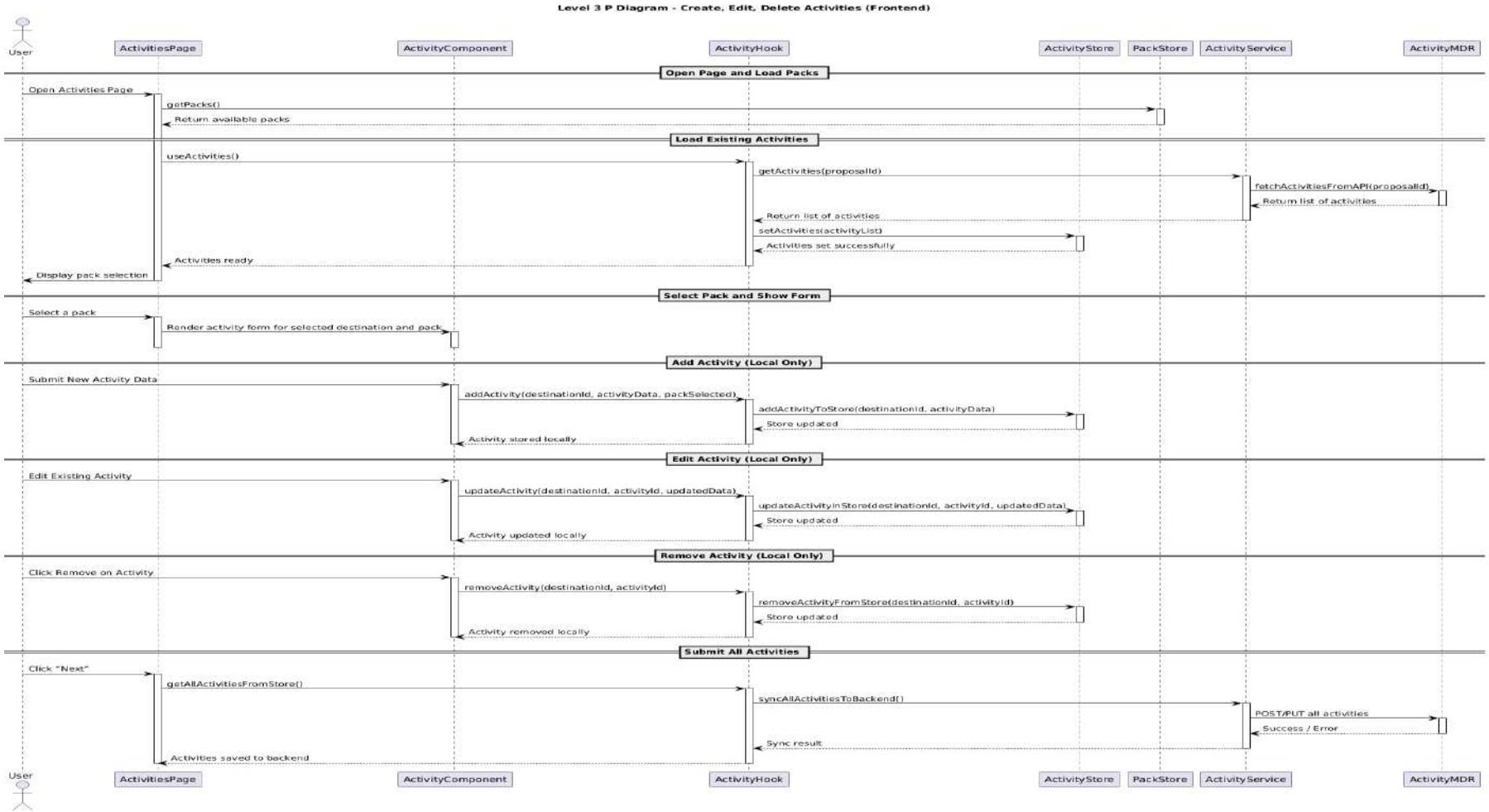

Figura A.25: US06.7 - Diagrama de sequência (frontend)

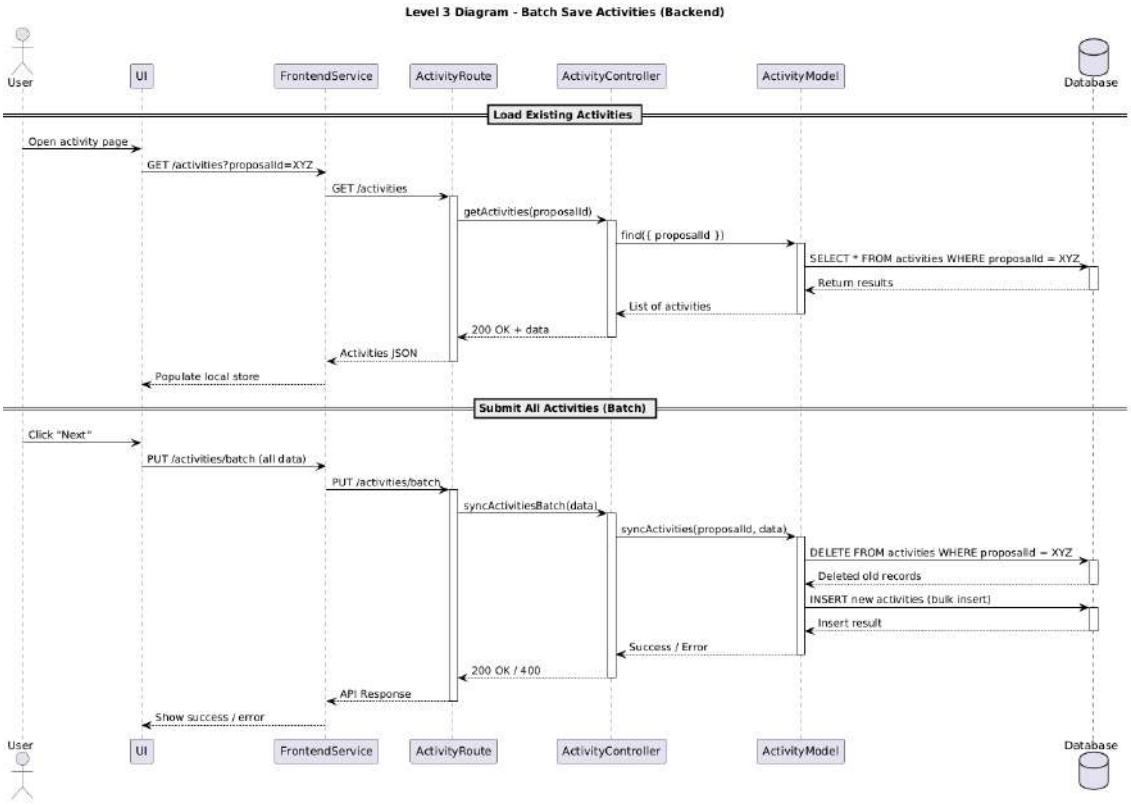

Figura A.26: US06.7 - Diagrama de sequência (backend)

A.8 Diagrama de Sequência & Interface – Seguros

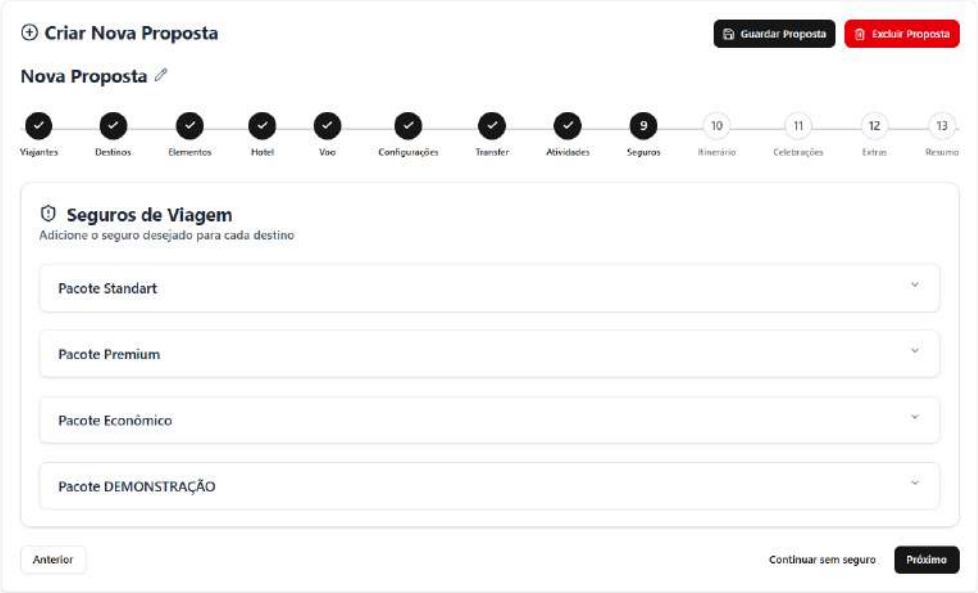

Figura A.27: Interface da página de Seguros: Seleção de um pack

Figura A.28: Seleção de um Seguro

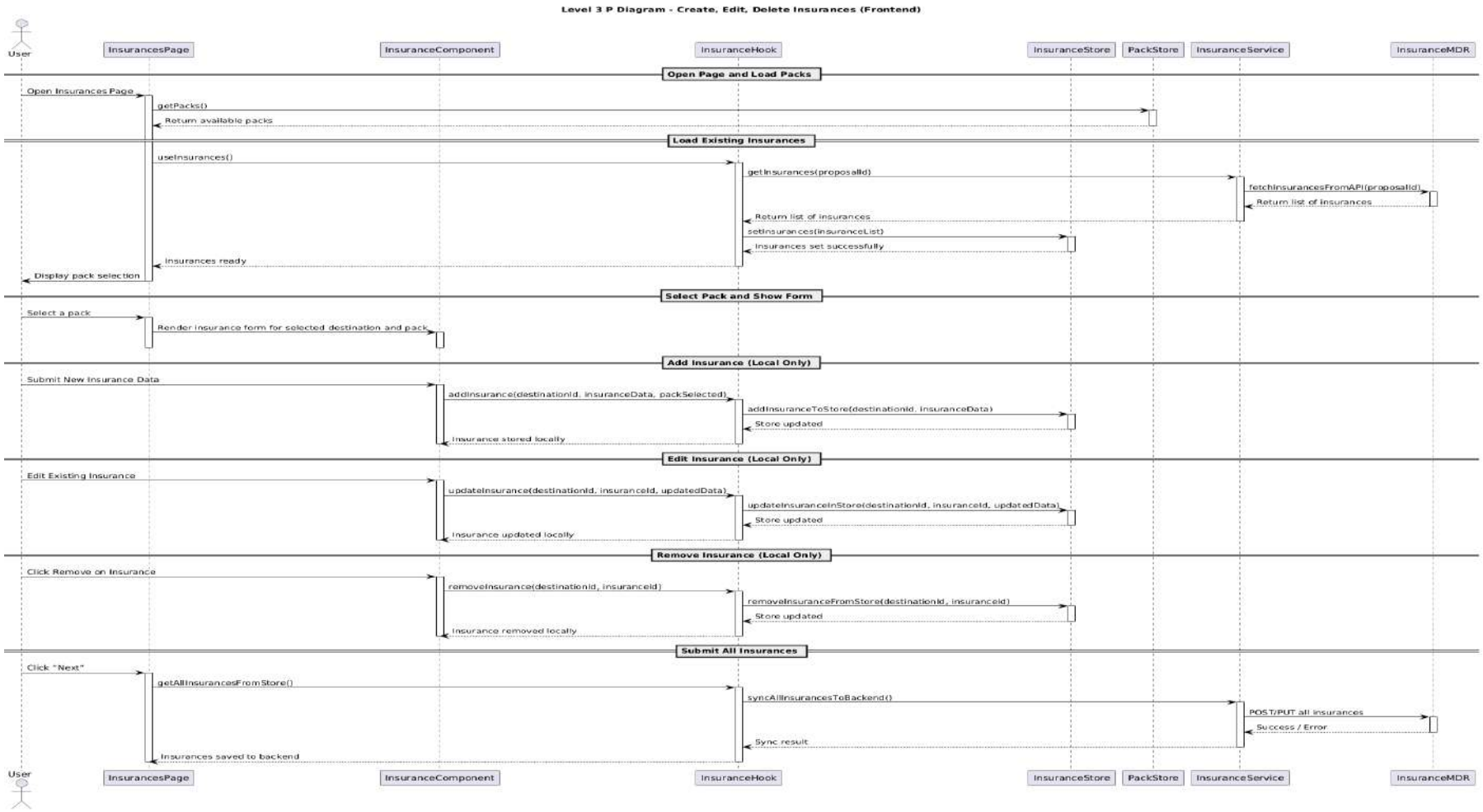

Figura A.29: US06.8 - Diagrama de sequência (frontend)

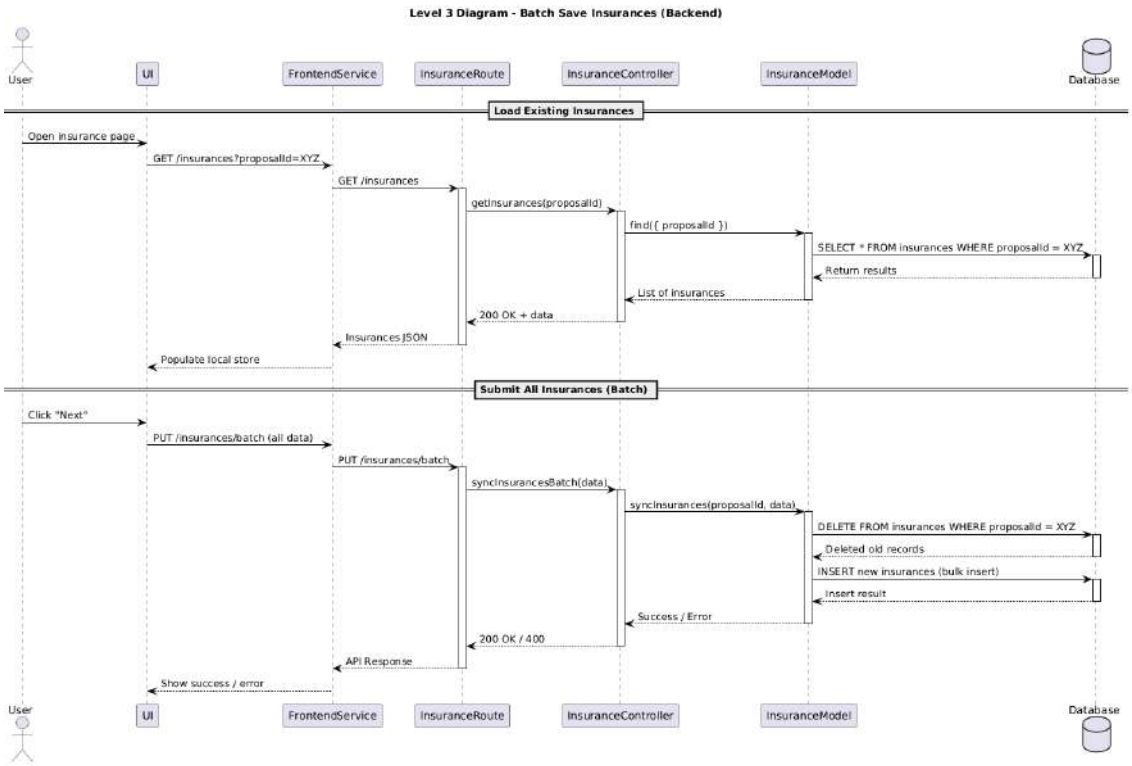

Figura A.30: US06.8 - Diagrama de sequência (backend)

A.9 Diagrama de Sequência & Interface – Celebrações

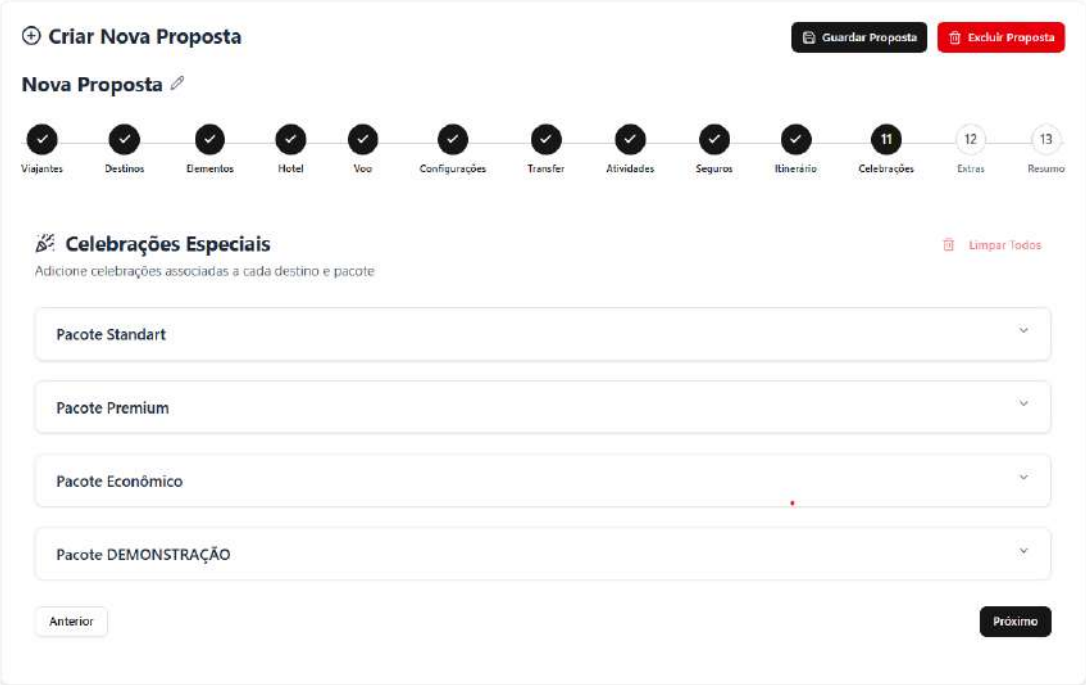

Figura A.31: Interface da página de Celebrações: Seleção de um pack

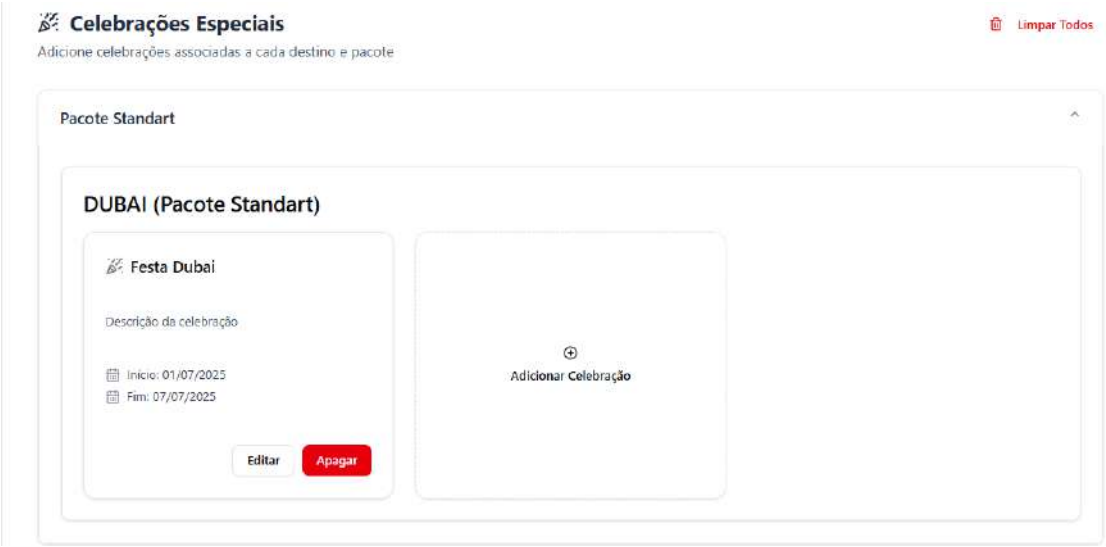

Figura A.32: Listagem de Celebrações

Figura A.33: Formulário para adicionar/editar celebração

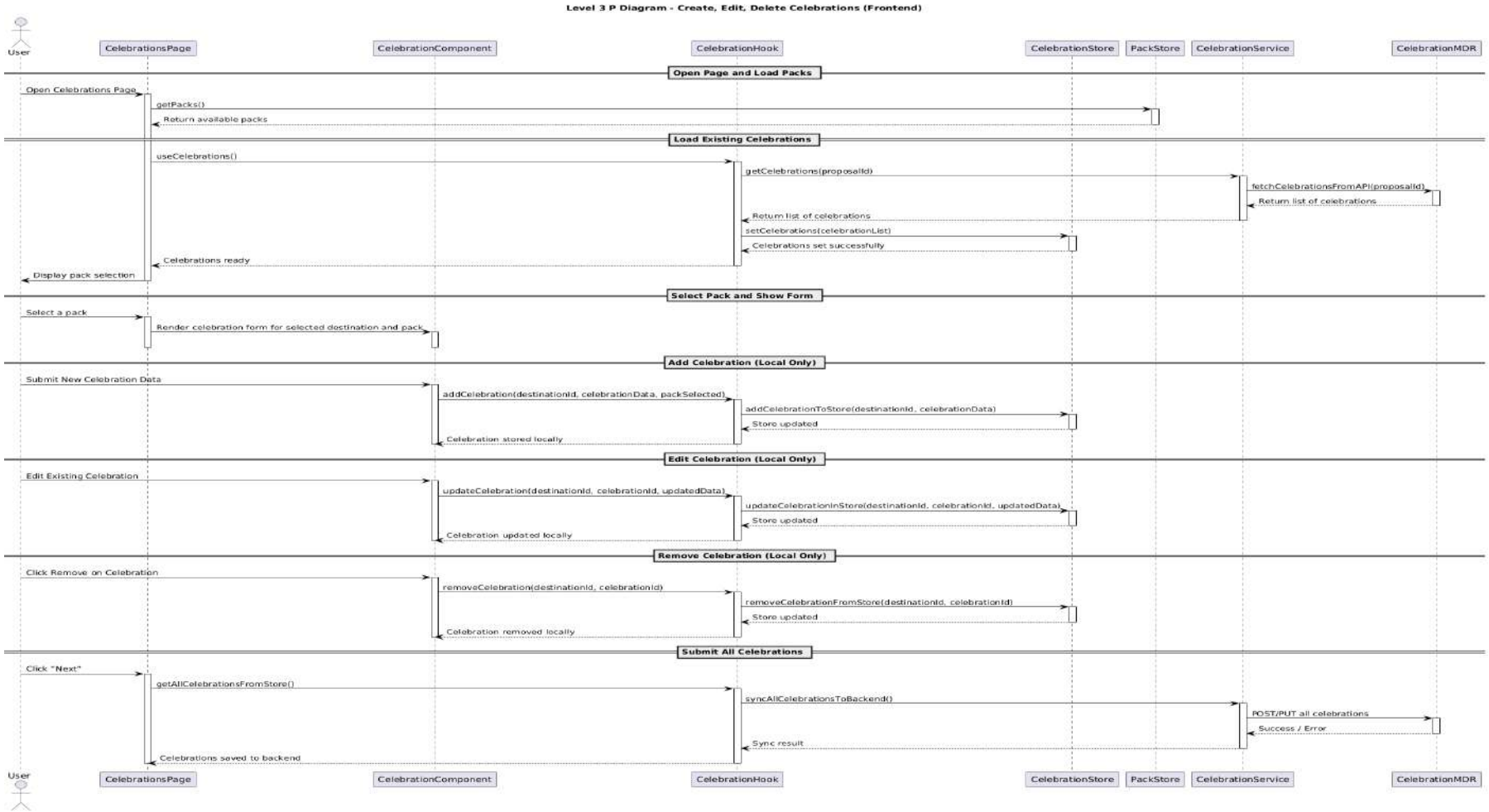

Figura A.34: US06.9 - Diagrama de sequência (frontend)

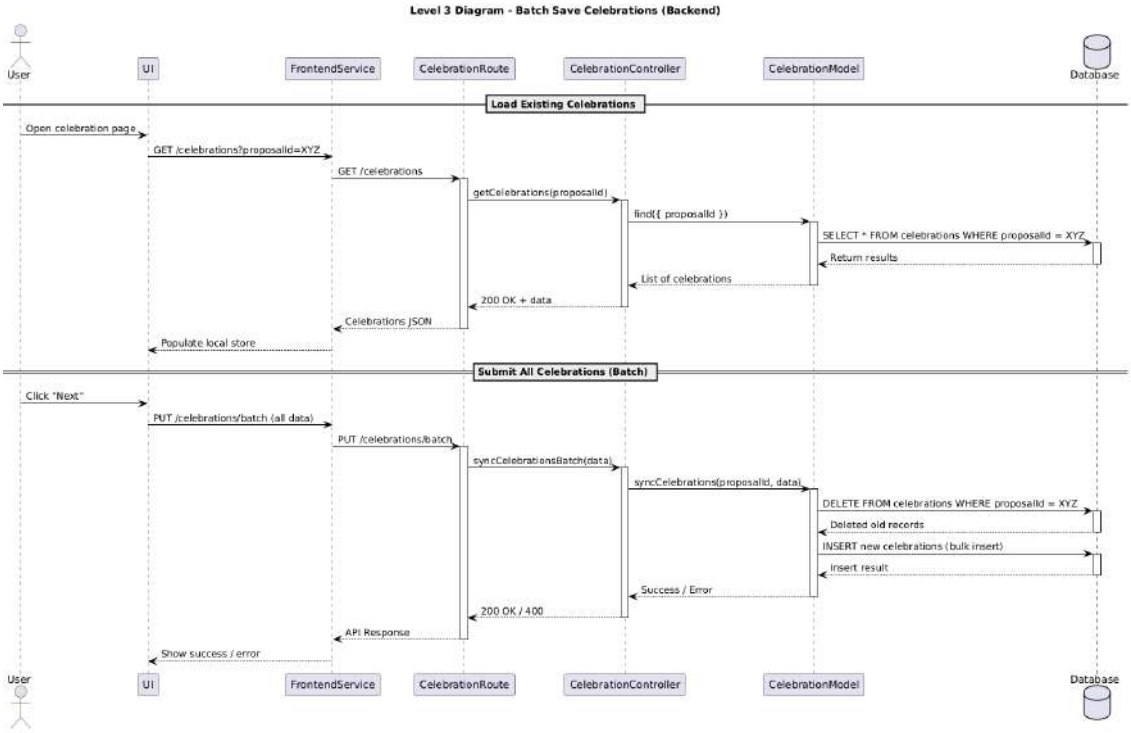

Figura A.35: US06.9 - Diagrama de sequência (backend)

A.10 Diagrama de Sequência & Interface – Itinerários

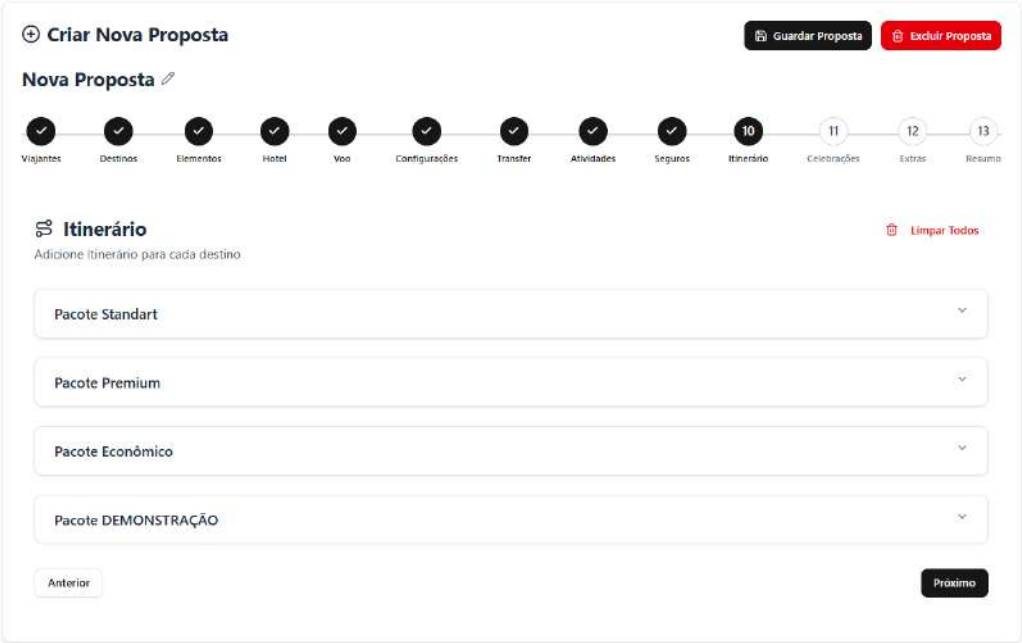

Figura A.36: Interface da página de Itinerários: Seleção de um pack

Figura A.37: Botão para criar itinerário com IA

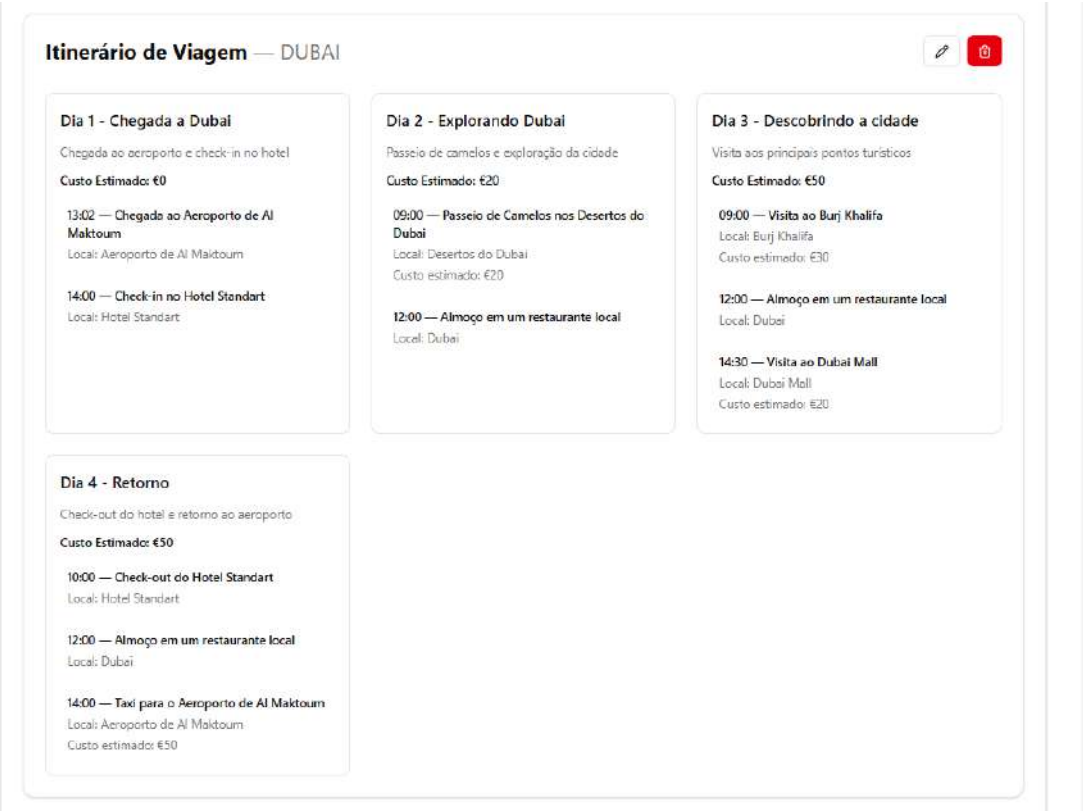

Figura A.38: Exemplo Itinerário criado com IA

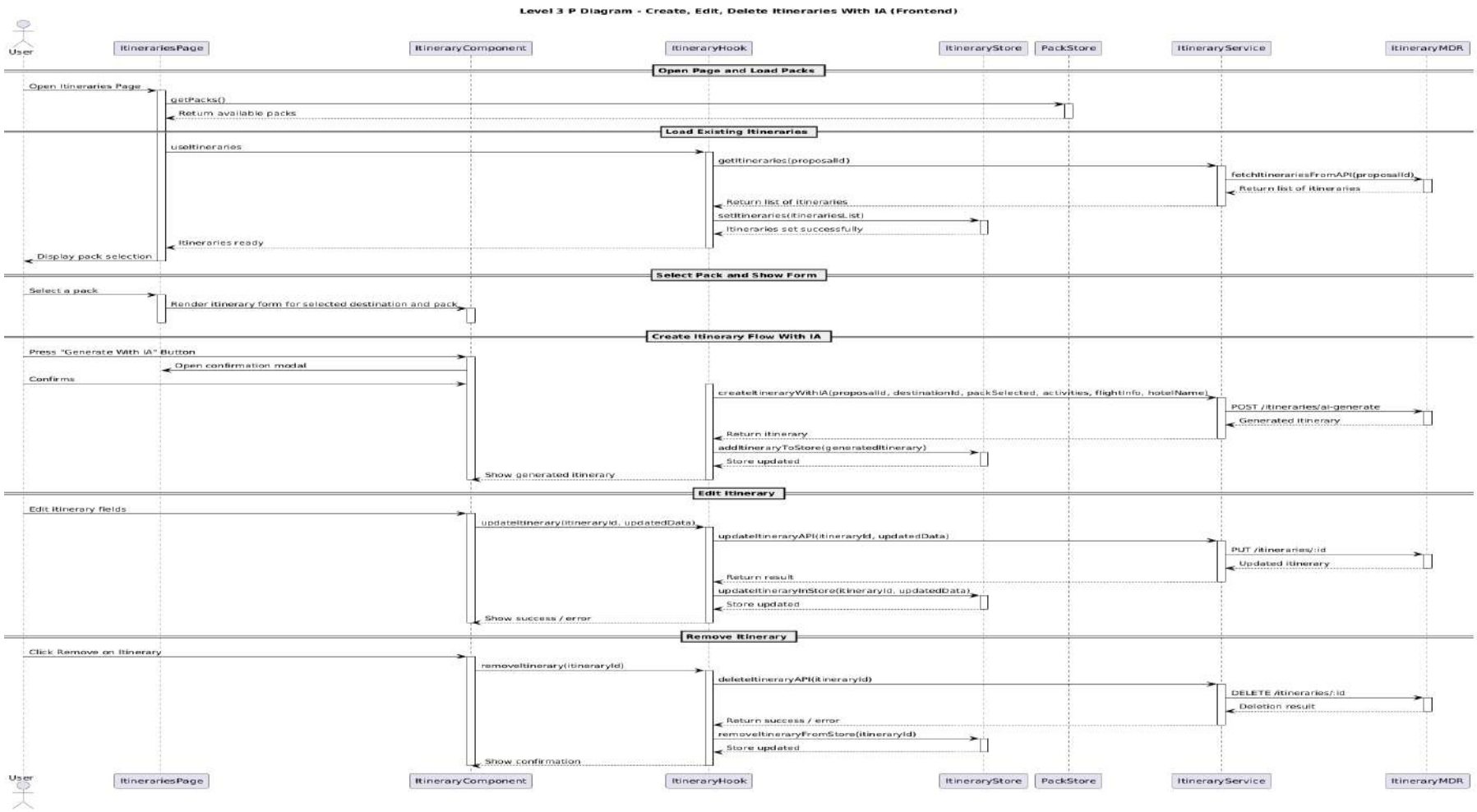

Figura A.39: US06.11 - Diagrama de sequência (frontend)

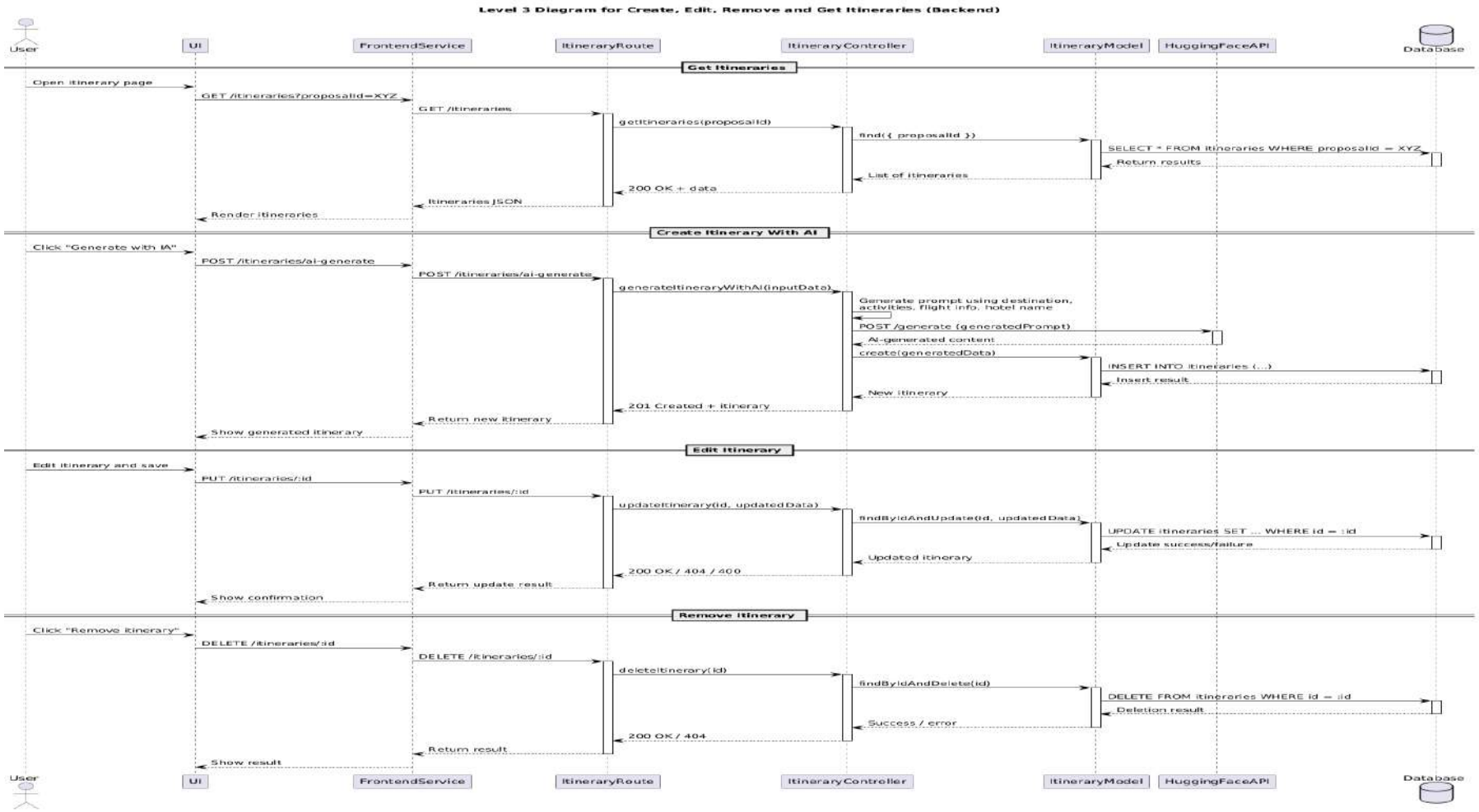

Figura A.40: US06.11 - Diagrama de sequência (backend)